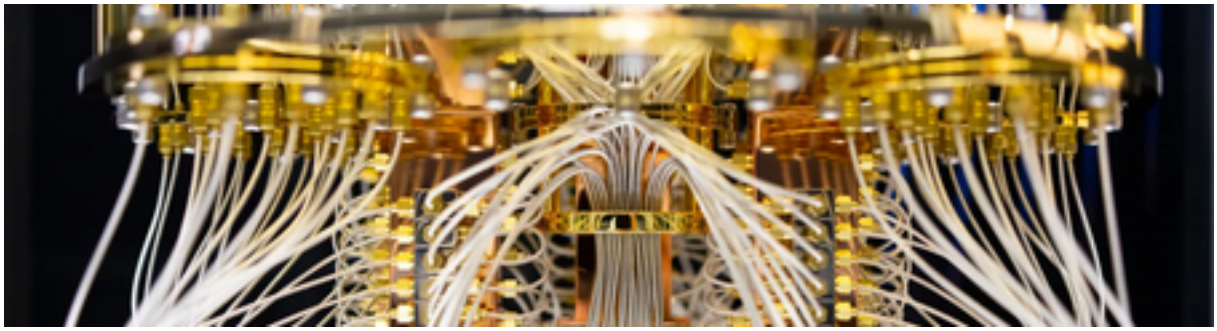
TÉCNICO LISBOA



# Accelerating Quantum Circuit Simulation with NVIDIA GPUs and tools editing

**Maria Madalena Reforço Osório de Barros**

Thesis to obtain the Master of Science Degree in

## Electrical and Computer Engineering

Supervisors: Prof. Leonel Augusto Pires Seabra de Sousa
Prof. Aleksandar Ilić

### Examination Committee

Chairperson: Prof. Name of the Chairperson
Supervisor: Prof. Leonel Augusto Pires Seabra de Sousa
Members of the Committee: Prof. Name of First Committee Member
Dr. Name of Second Committee Member
Eng. Name of Third Committee Member

**October 2026**

# Declaration of Originality

I declare that this document is an original work of my own authorship and that it fulfils all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

# Declaration of Artificial Intelligence Tools Use

I declare that generative Artificial Intelligence (AI) and AI-assisted tools were used in this dissertation. Nevertheless, I take full responsibility for the content and integrity of this work. In compliance with the Guide for the Responsible Use of Artificial Intelligence at Técnico, I declare that the following tools were used:

1) For linguistic revision: Anthropic. 2026. Claude [Large language model]. https://claude.ai. OpenAI. 2026. ChatGPT [Large language model]. https://chat.openai.com. Superhuman. 2026. Grammarly [AI writing assistant]. https://www.grammarly.com
2) As a coding assistant: OpenAI. 2026. ChatGPT [Large language model]. https://chat.openai.com. Anthropic. 2026. Claude Code [AI coding assistant]. https://claude.com/claude-code

# Acknowledgments

I would like to thank my parents and my brother. None of this would have been possible without your constant support, and I owe you far more than these pages can hold.
To the rest of my family, thank you for the constant encouragement.
To my partner, thank you for your encouragement in this pursuit.
I am grateful to my supervisors for their guidance.

# Abstract

Classical simulation supports the development and validation of quantum algorithms while quantum hardware remains limited in scale and by noise. Therefore, it is important not only to know how fast each simulator runs, but also to understand why, since published comparisons frequently rank runtimes without clarifying the conditions under which a backend excels. To that end, this dissertation benchmarks six state-vector backends, Qiskit Aer, CUDA-Q, Qulacs, mpiQulacs, Qibo and QuEST, across three circuit families on CPUs and GPUs, and for the Quantum Fourier Transform from a single CPU thread to $64$ nodes and sixteen GPUs on the EuroHPC supercomputer Deucalion. Performance models, roofline analysis and top-down profiling then relate the measured execution times to hardware utilisation, while stabiliser, tensor-network and density-matrix simulation complete the comparison. No backend consistently outperforms the others; the leading backend depends on the circuit, the qubit count and the hardware configuration. Memory capacity constrains the state size, whereas memory bandwidth, gate dispatch, vectorisation, gate fusion and distributed-state layout account for the primary performance differences. These results support a two-level selection framework, in which the circuit structure and the required output determine the simulation method, and the hardware and the state size determine the backend.

# Keywords

Quantum Circuit Simulation; GPU Acceleration; High-Performance Computing; State-Vector Simulation; Quantum Simulator Benchmarking

# Resumo

A simulação clássica apoia o desenvolvimento e a validação de algoritmos quânticos, enquanto o *hardware* quântico continua sujeito a limitações de escala e a efeitos de ruído. Importa, por isso, compreender as diferenças de desempenho entre simuladores, já que as comparações publicadas se centram frequentemente nos tempos de execução sem explicar em que condições cada um se destaca. Esta dissertação compara seis simuladores de vetor de estado, Qiskit Aer, CUDA-Q, Qulacs, mpiQulacs, Qibo e QuEST, em três famílias de circuitos em CPUs e GPUs. Para a transformada de Fourier quântica, a avaliação estende-se de uma *thread* de CPU a $64$ nós e dezasseis GPUs no supercomputador EuroHPC Deucalion. Modelos de desempenho, análise *roofline* e perfis *top-down* relacionam os tempos de execução medidos com o *hardware*. A comparação inclui também simulação por estabilizadores, redes tensoriais e matriz de densidade. Nenhum simulador é o mais rápido em todos os casos, pois a liderança depende do circuito, do número de qubits e da configuração do *hardware*. A capacidade da memória limita a dimensão do estado, enquanto a largura de banda da memória, o despacho de portas, a vetorização, a fusão de portas e a distribuição do estado explicam as principais diferenças de desempenho. Estes resultados sustentam um guia de seleção em dois níveis, em que a estrutura do circuito e a saída pretendida orientam a escolha do método de simulação, e o *hardware* e a dimensão do estado orientam a escolha do simulador.

# Palavras Chave

Simulação de Circuitos Quânticos; Aceleração por GPU; Computação de Alto Desempenho; Simulação por Vetor de Estado; *Benchmarking* de Simuladores Quânticos

# Contents


**1 Introduction** **1**

1.1 Context and Motivation . . . 2

1.2 Objectives . . . 3

1.3 Outline . . . 4

**2 Introduction to Quantum Computing** **5**

2.1 Formalism Underpinning Quantum Computation . . . 6

2.2 Quantum Computing Fundamentals . . . 7

2.2.1 Qubits and Superposition . . . 7

2.2.2 Bloch Sphere . . . 8

2.2.3 Entanglement . . . 9

2.2.4 Quantum Measurement . . . 9

2.2.5 Strong and Weak Simulation . . . 10

2.2.6 Quantum Gates . . . 10

2.3 Quantum Circuits . . . 12

2.3.1 Circuit Model . . . 12

2.3.2 Universal Gate Sets . . . 13

2.3.3 Clifford Gates . . . 13

2.4 Noise in Quantum Systems . . . 14

2.4.1 Density-Matrix Formalism . . . 14

2.4.2 Decoherence Mechanisms . . . 15

2.4.3 Noise Channels . . . 16

2.5 Summary . . . 18

**3 State of the Art: Classical Quantum Circuit Simulation** **19**

3.1 Classical Simulation Approaches . . . 20

3.1.1 State-Vector Simulation . . . 20

3.1.2 Path-Summation and Hybrid Methods . . . 21

3.1.3 Tensor-Network Simulation . . . 22

3.1.4 Stabiliser Simulation . . . 24
3.2 Simulation Tools and Backends . . . 24
3.3 Noise Simulation Strategies . . . 25
3.4 The Cost of State-Vector Simulation . . . 26
3.5 Architectural Optimisations . . . 28
3.5.1 Vectorisation . . . 28
3.5.2 Gate Fusion . . . 29
3.5.3 Thread Placement and Memory Affinity . . . 30
3.5.4 GPU Execution . . . 31
3.5.5 Distributed Execution . . . 32
3.6 Prior Studies and the Remaining Gap . . . 33
3.7 Summary . . . 34
**4 Benchmarking Framework and Methodology** **35**
4.1 Performance Analysis Framework . . . 36
4.2 Benchmark Circuits . . . 38
4.2.1 The Quantum Fourier Transform . . . 39
4.2.2 The Hardware-Efficient Ansatz . . . 40
4.2.3 The Clifford Circuit . . . 41
4.3 Experimental Setup . . . 41
4.3.1 Simulators and Builds . . . 42
4.3.2 Execution Model . . . 45
4.4 Measurement and Analysis . . . 47
4.4.1 Comparisons and Timing . . . 47
4.4.2 Roofline Measurements . . . 48
4.4.3 Accuracy Metrics . . . 50
4.5 Summary . . . 51
**5 Results** **53**
5.1 Single-Node Performance Landscape . . . 54
5.2 QFT Thread Scaling on One Node . . . 58
5.3 Multi-Node CPU Scaling . . . 60
5.4 GPU Acceleration . . . 64
5.4.1 Within One Node . . . 64
5.4.2 Across Nodes . . . 66
5.5 Vectorisation and Gate Fusion . . . 69
5.6 Noise-Aware Density-Matrix Simulation . . . 70

5.7 Beyond State-Vector Simulation . . . 73
5.8 Simulator-Selection Framework . . . 75
5.9 Summary . . . 77

**6 Conclusions and Future Work** **79**
6.1 Conclusions . . . 80
6.2 Limitations . . . 81
6.3 Future Work . . . 82

**Bibliography** **83**

**A Supporting Evidence for the Results** **92**
A.1 Variation Between Repetitions . . . 92
A.2 Single-Thread Execution-Path Profiling . . . 93
A.3 Top-Down Microarchitecture Analysis for the QFT . . . 94
A.4 Process Layout of the Multi-Node CPU Runs . . . 96
A.5 GPU Activity and Data Exchange in the Multi-GPU Runs . . . 97
A.6 The CUDA IPC Transport in the Qiskit Aer Runs . . . 100

# List of Figures


2.1 Bloch sphere representation of a single qubit. . . . . 8
2.2 Two-qubit circuit for Bell-state preparation. . . . . 12
2.3 Controlled-$Z$ gate from Hadamard and CNOT gates. . . . . 13
2.4 Stabiliser simulation of the Bell-state circuit. . . . . 14

3.1 Amplitude pairs updated by a single-qubit gate. . . . . 21
3.2 Schrödinger–Feynman decomposition of a cut gate. . . . . 22
3.3 Matrix product state of five qubits. . . . . 23
3.4 Tensor-network and state-vector runtime in Qibo. . . . . 23
3.5 Stabiliser tableau of the Bell-state circuit. . . . . 24
3.6 Roofline analysis of state-vector gate kernels. . . . . 27
3.7 Gate fusion reduces the passes over the state vector. . . . . 29
3.8 NUMA topology and state-vector placement on a dual-socket node. . . . . 31
3.9 State-vector distribution across ranks via index bits. . . . . 32

4.1 Cache-aware roofline model and its three regions. . . . . 38
4.2 Circuit diagram for the $n$-qubit QFT. . . . . 40
4.3 Structure of the HEA benchmark. . . . . 40
4.4 Structure of the Clifford benchmark. . . . . 41
4.5 Exchange schemes of the distributed backends. . . . . 46
4.6 Data paths of the multi-GPU option of CUDA-Q. . . . . 47

5.1 Single-node state-vector simulation time across three regimes and three circuits. . . . . 55
5.2 Thread scaling and parallel efficiency of the QFT at $n = 30$ on one node. . . . . 59
5.3 Multi-node QFT strong scaling at $n = 30$, each backend in its best layout. . . . . 61
5.4 Multi-node QFT weak scaling with $28$ qubits per node. . . . . 63
5.5 CUDA-Q multi-GPU time against $n$ within one node, double precision. . . . . 64
5.6 Multi-GPU QFT time at $n = 30$ within and across nodes. . . . . 66

5.7 Multi-GPU QFT time from $n = 30$ to $n = 35$ within and across nodes. . . . . 67
5.8 Roofline comparison of vectorisation and fusion. . . . . 69
5.9 Cost and fidelity of density-matrix simulation of the QFT with depolarising noise. . . . . 71
5.10 Accuracy of the QFT under four noise channels. . . . . 72
5.11 Stabiliser vs. state-vector runtime for random Clifford circuits. . . . . 73
5.12 Tensor-network vs. state-vector runtime, by circuit family and entanglement. . . . . 74
5.13 Two-level selection of a simulation method and a backend. . . . . 78

A.1 QFT single-thread kernel share, QULACS vs. QISKIT AER. . . . . 94
A.2 Top-down pipeline-slot breakdown for QFT at $n = 28$. . . . . 95
A.3 Estimated GPU-active and inactive time, QFT from $n = 30$ to $n = 34$. . . . . 98
A.4 Swap bandwidth between GPU buffers within and across nodes. . . . . 99
A.5 QISKIT AER QFT time with and without the CUDA IPC transport. . . . . 100

# List of Tables


2.1 Common single- and multi-qubit gates. . . . 11
2.2 Coherence times and two-qubit gates of the main qubit technologies. . . . 16

3.1 Comparison of classical quantum circuit simulation paradigms. . . . 25
3.2 State-vector memory by qubit count and matching hardware. . . . 26
3.3 CPU-side hardware-optimisation support across state-vector simulators. . . . 33

4.1 Benchmark circuits. . . . 39
4.2 Experimental platforms. . . . 42
4.3 Simulator versions and configurations. . . . 45
4.4 Run specification of each experiment. . . . 49
4.5 Floating-point operation counts at $n = 28$. . . . 49
4.6 Feature comparison of the benchmarked simulators and the broader landscape. . . . 52

5.1 QFT thread-scaling summary at $n = 30$. . . . 59
5.2 Karp–Flatt fraction of the QFT thread sweep at $n = 30$. . . . 60
5.3 Multi-node QFT strong scaling at $n = 30$ on Deucalion x86. . . . 61
5.4 Karp–Flatt fraction of the multi-node QFT strong scaling at $n = 30$. . . . 62
5.5 CUDA-Q multi-GPU time within one node, double precision. . . . 65
5.6 Speedup and Karp–Flatt fraction of CUDA-Q within one node. . . . 65
5.7 Incremental speedup of the multi-GPU QFT runs within and across nodes. . . . 68
5.8 Roofline estimates and runtimes at $n = 28$. . . . 70
5.9 Simulation paradigm by circuit structure, first level of the selection framework. . . . 75
5.10 Dense state-vector backend by execution regime, second level of the selection framework. . . . 76

A.1 Spread between repetitions per experiment group. . . . 93
A.2 Single-thread execution-path summary across backends and circuits. . . . 94
A.3 Top-down QFT diagnostics on the i9-13900KS platform at $n = 26$ and $n = 28$. . . . 96
A.4 QFT time of mpiQulacs and Qiskit Aer in the two process layouts. . . . 97

# Acronyms

**AVX2** Advanced Vector Extensions 2

**AVX-512** Advanced Vector Extensions 512

**CARM** Cache-aware Roofline Model

**CPTP** Completely Positive, Trace-Preserving

**CPU** Central Processing Unit

**CUDA** Compute Unified Device Architecture

**DBI** Dynamic Binary Instrumentation

**DFT** Discrete Fourier Transform

**DRAM** Dynamic Random-Access Memory

**FMA** Fused Multiply-Add

**FPGA** Field Programmable Gate Array

**GHZ** Greenberger–Horne–Zeilinger

**GPU** Graphics Processing Unit

**HEA** Hardware-Efficient Ansatz

**HPC** High-Performance Computing

**HSP** Hidden Subgroup Problem

**IPC** Inter-Process Communication

**MPI** Message Passing Interface

**MPS** Matrix Product State

**NISQ** Noisy Intermediate-Scale Quantum

**NUMA** Non-Uniform Memory Access

**PCIe** Peripheral Component Interconnect Express

**PMU** Performance Monitoring Unit

**QEC** Quantum Error Correction

**QFT** Quantum Fourier Transform

**RQC** Random Quantum Circuit

**SDK** Software Development Kit

**SIMD** Single Instruction, Multiple Data

**SMT** Simultaneous Multithreading

**TMA** Top-down Microarchitecture Analysis

**UCX** Unified Communication X

**VQE** Variational Quantum Eigensolver

# 1

# Introduction

## Contents


1.1 Context and Motivation . . . . . 2

1.2 Objectives . . . . . 3

1.3 Outline . . . . . 4

This dissertation examines the classical simulation of quantum circuits, its cost on current hardware and the performance of the simulators that implement it. Section 1.1 sets the context, Section 1.2 presents the research questions and objectives, and Section 1.3 outlines the document structure.

## 1.1 Context and Motivation

Quantum algorithms provide computational speedups over the best known classical methods for problems such as integer factorisation [1]. Currently available processors are Noisy Intermediate-Scale Quantum (NISQ) devices, limited by noise and by the number of physical qubits [2]. Running such algorithms at scale requires *fault-tolerant quantum computing*, in which quantum error correction suppresses logical errors. Error correction below the surface-code threshold has been demonstrated on a $105$-qubit superconducting processor [3]. Building on this progress, a machine capable of executing about a million error-corrected operations has been proposed as a next milestone [4].

Superconducting circuits, trapped ions, neutral atoms, photonic systems and semiconductor spins are the main physical approaches to building quantum computers. The eight quantum computers procured or co-funded by the EuroHPC Joint Undertaking for European High-Performance Computing (HPC) centres span the first four, plus a superconducting quantum annealer [5], with some in operation and others planned. All of them face the same obstacles. Qubits lose their coherence and gates introduce errors, which limits the depth of the circuits that run reliably. The number of qubits and their connectivity are also limited, so circuits must be mapped onto the interactions a device offers [6]. Furthermore, error correction needs many physical qubits for each logical one, with the surface code requiring physical error rates below about $0.6$ to $1\%$ per operation [7]. Section 2.4 gives typical coherence times and gate errors for each technology.

While these obstacles remain, quantum algorithms are developed and tested on classical simulators. Simulation is used to verify algorithms, to characterise noise and to set the classical baselines that a quantum device must beat to claim an advantage [2], but its cost grows exponentially with the number of qubits [8]. The state-vector representation of an $n$-qubit system requires $2^n$ complex amplitudes, so every additional qubit doubles the memory. A $30$-qubit state vector in double-precision floating-point format occupies $16$ GiB, and a $50$-qubit one $16$ PiB. Applying a general single-qubit gate performs $28$ floating-point operations for every $64$ bytes it reads and writes, so its arithmetic intensity, the number of operations per byte moved, is $0.4375$ FLOP/byte [9]. This intensity is far below the point at which current processors become compute-bound, so the workload is bound by memory bandwidth [10]. The exponential growth of the state and the limit set by memory bandwidth are the two constraints that this dissertation examines.

Wall-clock runtime alone is too coarse to characterise a backend or to guide its use. Two simulators can share asymptotic scaling yet differ in how they fuse gates, vectorise their kernels[1], dispatch each gate and

[1] In this dissertation, a *kernel* is the numerical routine of a simulator that applies gates to the state vector. The term does not refer to the operating-system kernel.

lay out a distributed state vector. These differences decide whether a run is bound by memory capacity, memory bandwidth, computation or communication. This dissertation therefore benchmarks state-vector simulators across a common set of circuits and platforms, and relates the measured runtimes and scaling to the architecture of each backend and execution regime. The comparison can then guide the choice of simulator for a given circuit family, qubit range and hardware configuration.

## 1.2 Objectives

The primary objective of this dissertation is to benchmark and compare quantum-circuit simulators on classical high-performance hardware, with an emphasis on how computational performance, the memory subsystem and parallel scalability interact. Building on the preceding integrated project (*Projeto Integrador de 2º Ciclo*, PIC), which compared Qiskit Aer and CUDA-Q on the Quantum Fourier Transform (QFT), this dissertation broadens both the simulator set and the analysis. This objective is decomposed into three research questions.

**RQ1.** How do the state-vector simulators examined in this dissertation differ in performance across circuit families, qubit counts and execution regimes spanning single-thread, full-node, multi-node, single-GPU and multi-GPU configurations?

**RQ2.** To what extent can the observed performance differences be attributed to architectural factors such as memory capacity, memory bandwidth, gate fusion, vectorisation, gate dispatch cost and the layout of a distributed state vector?

**RQ3.** Where are the practical boundaries of dense state-vector simulation, and under what circumstances do alternative paradigms, such as stabiliser or tensor-network methods, become preferable?

These questions are addressed through four objectives. The first objective is to broaden the comparison. Five open-source frameworks, Qiskit Aer, CUDA-Q, Qulacs (with its Message Passing Interface (MPI) build, mpiQulacs), Qibo and QuEST, spanning six backends, are evaluated on three circuit families, the QFT, the Hardware-Efficient Ansatz (HEA) and a shallow Clifford circuit, defined in Section 4.2. The three were chosen for their different structures, all-to-all couplings in the QFT, nearest-neighbour layers in the HEA and a Clifford circuit that stabiliser methods simulate efficiently. Benchmark suites for quantum software include the QFT and hardware-efficient ansätze among their standard circuits [11], so these two circuits also allow the results to be compared with other studies, whereas further circuits of the same structures would repeat the comparison without adding a new case. The circuits run in single-thread, full-node, single-GPU, multi-node and multi-GPU regimes, which follow the levels of parallel hardware on which a state-vector simulator runs, from one core to several nodes and Graphics Processing Units (GPUs). Multi-node execution is measured for the QFT, and multi-GPU execution for the QFT and the HEA.

The second objective is to explain the observed performance. Wall-clock behaviour is interpreted against state-vector memory scaling, where capacity sets the ceiling on the reachable qubit count and bandwidth limits throughput. A cache-aware roofline then places the kernels of QULACS relative to the bandwidth and compute ceilings of one core, and shows how gate fusion moves their operating point and how vectorisation raises throughput at a fixed arithmetic intensity.
The third objective is to characterise parallel and distributed scalability. Shared-memory thread scaling is diagnosed with the Karp–Flatt effective serial fraction. Distributed-memory MPI strong and weak scaling, and multi-GPU scaling across the NVLink and InfiniBand fabrics, distinguish the cases in which distribution accelerates a fixed problem from those in which it only extends the tractable size.
The fourth objective is to map the boundaries of dense state-vector simulation and synthesise the results into practical guidance. The stabiliser and tensor-network paradigms are evaluated relative to state-vector baselines to determine under which conditions they extend the achievable qubit count. Additionally, a secondary analysis quantifies how density-matrix noise modelling contracts it, assessing accuracy using state fidelity, purity and population error. The consolidated findings provide recommendations regarding the most suitable simulator, backend and execution regime for specific circuit types and qubit ranges.

## 1.3 Outline

Chapter 2 introduces the quantum-computing background required for the rest of the work, including the formalism, qubits, quantum circuits and noise in quantum systems, as well as typical coherence times and gate errors of the main hardware technologies. Chapter 3 reviews the classical simulation approaches, the landscape of simulator tools and the hardware-level optimisations that state-vector engines exploit, concluding with a review of prior benchmarking studies and the research gap addressed in this dissertation. Chapter 4 presents the performance models used to interpret the measurements, defines the benchmark circuits, simulators and platforms, and details the experimental design and the metrics connecting results to hardware characteristics. Chapter 5 reports the results, covering single-node performance, thread and multi-node scaling, GPU scaling, roofline analysis, the noise study and the comparison of simulation paradigms, and synthesises these findings into a selection framework. Chapter 6 summarises the main findings, discusses the limitations of the work and proposes directions for future research. Appendix A contains the evidence behind the results, including the variation between repetitions, Central Processing Unit (CPU) profiles and the data exchanges of the multi-GPU experiments.

# 2

# Introduction to Quantum Computing

## Contents


2.1 Formalism Underpinning Quantum Computation . . . . . 6
2.2 Quantum Computing Fundamentals . . . . . 7
2.3 Quantum Circuits . . . . . 12
2.4 Noise in Quantum Systems . . . . . 14
2.5 Summary . . . . . 18

This chapter introduces the quantum-mechanical formalism that underlies the simulation workloads studied in this dissertation. It focuses on the concepts and notation that directly determine the computational cost and memory requirements of classical simulators, namely how states are represented, how gates transform those representations and how density matrices represent noisy states. Section 2.4 includes typical coherence times and gate errors of the main qubit technologies (Table 2.2). Chapter 4 defines the benchmark circuits, including the Quantum Fourier Transform (QFT), and the accuracy metrics used in the evaluation.

## 2.1 Formalism Underpinning Quantum Computation

Quantum computation rests on four postulates of quantum mechanics. Pure states are represented by complex unit vectors in Hilbert spaces whose dimension grows exponentially with the number of qubits, and unitary operators describe their evolution in closed systems. The postulates are presented below in the form applicable to qubit registers and circuits, following Nielsen and Chuang [12].

**Postulate 2.1** (State Space)**.** *An isolated quantum register of $n$ qubits is described by a unit state vector $|\Psi\rangle$ in the $2^n$-dimensional Hilbert space $\mathcal{H}_{2^n}$. The computational basis $\{|x\rangle : x \in \{0,1\}^n\}$ plays the role of the standard basis in $\mathbb{C}^{2^n}$. Global phase is physically unobservable, so $|\Psi\rangle$ and $e^{i\phi}|\Psi\rangle$ represent the same physical state. For a single qubit ($n = 1$), the computational basis $\{|0\rangle, |1\rangle\}$ spans $\mathbb{C}^2$, and states are commonly visualised on the Bloch sphere.*

This first postulate has a direct consequence for simulations, given that representing the state of $n$ qubits requires storing $2^n$ complex amplitudes. In the double-precision (64-bit) floating-point format, each amplitude occupies 16 bytes, so the memory doubles with every added qubit [10]. This exponential scaling is the main obstacle facing classical simulation and motivates the GPU-accelerated and approximate methods examined in Chapter 3.

**Postulate 2.2** (Observables as Hermitian Operators)**.** *Every measurable physical quantity corresponds to a Hermitian (self-adjoint) operator $\hat{O} = \sum_i o_i |o_i\rangle\langle o_i|$, whose eigenvectors $\{|o_i\rangle\}$ form an orthonormal basis and whose real eigenvalues $\{o_i\}$, which may repeat, are the only possible measurement outcomes. In quantum computing, the Pauli operators $X$, $Y$, $Z$ and their tensor products are the most commonly used observables.*

Hermitian operators are the quantum equivalent of real symmetric matrices in classical linear algebra, as they have real spectra and orthogonal eigenbases. The postulate therefore establishes that measurement outcomes correspond to eigenvalues and the state is projected onto the corresponding eigenspace after measurement.

**Postulate 2.3** (Measurement (Born Rule))**.** *Measuring the observable $\hat{O}$ on state $|\Psi\rangle$ yields the outcome $o_i$ with probability*

$$\Pr(o_i) = |\langle o_i|\Psi\rangle|^2, \tag{2.1}$$

*after which the state collapses irreversibly to $|o_i\rangle$; when several eigenvectors share an eigenvalue, their probabilities add and the state is projected onto the space that they span.*

The Born rule connects the continuous, deterministic evolution of quantum amplitudes to the discrete, probabilistic outcomes observed in experiment. In terms of simulation, it specifies what a simulator must ultimately produce, either exact probabilities (strong simulation) or samples drawn from the distribution $\{|\langle o_i|\Psi\rangle|^2\}$ (weak simulation), the two types of simulation of Section 2.2.5.

**Postulate 2.4** (Time Evolution (Unitary Dynamics))**.** *Between measurements, a closed quantum system evolves according to the Schrödinger equation $i\hbar\frac{d}{dt}|\Psi(t)\rangle = \hat{H}|\Psi(t)\rangle$. For a time-independent Hamiltonian, this continuous evolution is governed by the unitary operator $U(t) = e^{-i\hat{H}t/\hbar}$. The circuit model of quantum computation approximates this continuous evolution by a finite sequence of unitary gates $U = U_m \cdots U_2 U_1$ acting on the qubits.*

Unitary operators ($U^\dagger U = I$) have two properties that both quantum algorithms and their simulation rely on. Every gate has the inverse $U^\dagger$, so quantum computation is reversible up to the point of measurement, and $\langle\Psi|U^\dagger U|\Psi\rangle = 1$, so probability amplitudes remain normalised throughout the computation. Unitary operators are thus the complex analogue of orthogonal matrices, since they preserve the $\ell^2$-norm of the state vector just as orthogonal transformations preserve Euclidean length.

## 2.2 Quantum Computing Fundamentals

A classical simulator stores and transforms the objects defined in this section, namely qubit registers and their amplitudes, entangled states that cannot be factored into single-qubit states, the probabilities that measurement returns and the gates that transform the state.

### 2.2.1 Qubits and Superposition

In classical computing, an $n$-bit register occupies exactly one of the $2^n$ configurations in $\{0,1\}^n$ at any given time. Quantum computing replaces classical bits with *quantum bits* (qubits), whose state space is the two-dimensional complex Hilbert space $\mathcal{H}_2 = \mathbb{C}^2$ (Postulate 2.1). The computational basis of a single qubit is the orthonormal pair $\{|0\rangle, |1\rangle\}$, and the most general pure state is a normalised superposition

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle, \qquad \alpha,\beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1, \tag{2.2}$$

where $\alpha$ and $\beta$ are called *probability amplitudes*. By the Born rule (Postulate 2.3), measurement in the computational basis yields outcome $0$ with probability $|\alpha|^2$ and outcome $1$ with probability $|\beta|^2$. These probabilities alone do not characterise the state, however, since the relative phase between the amplitudes also determines the subsequent evolution and interference.

For a register of $n$ qubits, the joint state space is the tensor product $\mathcal{H}_{2^n} = (\mathbb{C}^2)^{\otimes n}$, spanned by the $2^n$ computational-basis vectors $|x\rangle$ with $x \in \{0,1\}^n$. A general $n$-qubit pure state is therefore

$$|\Psi\rangle = \sum_{x\in\{0,1\}^n} c_x\,|x\rangle, \qquad \sum_x |c_x|^2 = 1, \tag{2.3}$$

where the $c_x \in \mathbb{C}$ are the probability amplitudes of the basis states.

### 2.2.2 Bloch Sphere

Up to a physically unobservable global phase (Postulate 2.1), any single-qubit pure state can be parameterised as

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle, \qquad \theta \in [0,\pi],\ \varphi \in [0,2\pi). \tag{2.4}$$

Each pure state, up to global phase, corresponds to a unique point on the surface of the *Bloch sphere* (Figure 2.1). The angles $(\theta,\varphi)$ define Cartesian coordinates $(x,y,z) = (\sin\theta\cos\varphi,\ \sin\theta\sin\varphi,\ \cos\theta)$ on the unit sphere. The north and south poles correspond to $|0\rangle$ and $|1\rangle$, respectively. Points on the equator represent equal-weight superpositions, such as $|\pm\rangle = \frac{1}{\sqrt{2}}(|0\rangle \pm |1\rangle)$ along the $x$-axis and $\frac{1}{\sqrt{2}}(|0\rangle \pm i|1\rangle)$ along the $y$-axis.

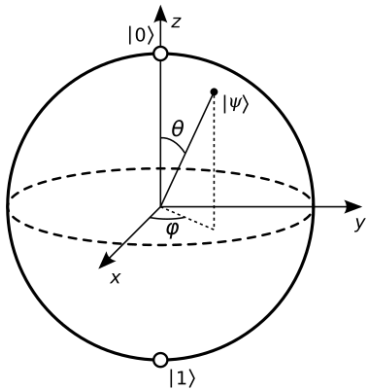



**Figure 2.1:** Bloch sphere, a geometric representation of a two-level quantum system (adapted from Smite-Meister [13]).

On the Bloch sphere, single-qubit unitary gates act as rotations (Section 2.2.6). Mixed states, which can arise through noise or interactions with the environment, lie inside the sphere and are described by the density-matrix formalism introduced in Section 2.4.1.

### 2.2.3 Entanglement

The joint state of two qubits $A$ and $B$ is described within the tensor product space $\mathcal{H}_A \otimes \mathcal{H}_B$. If it can be written as a product of a state for each qubit, i.e. $|\Psi_{AB}\rangle = |\psi_A\rangle \otimes |\psi_B\rangle$, the state is *separable*. A pure joint state that *cannot* be written as such a product ($|\Psi_{AB}\rangle \neq |\psi_A\rangle \otimes |\psi_B\rangle$) is *entangled*, and the individual qubits then have mixed reduced states rather than pure-state vectors of their own.

A standard example is the Bell state

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\big(|00\rangle + |11\rangle\big), \tag{2.5}$$

in which measuring both qubits in the computational basis gives perfectly correlated outcomes, $00$ or $11$ with probability $1/2$ each, regardless of the distance between the qubits. The four Bell states,

$$|\Phi^\pm\rangle = \frac{1}{\sqrt{2}}\big(|00\rangle \pm |11\rangle\big), \qquad |\Psi^\pm\rangle = \frac{1}{\sqrt{2}}\big(|01\rangle \pm |10\rangle\big), \tag{2.6}$$

form an orthonormal basis of maximally entangled two-qubit states. These states are widely used in quantum teleportation, superdense coding and error-correction protocols [12].

For simulation, entanglement prevents the joint pure state from being stored as one state vector per qubit. Tensor-network methods can still represent such states compactly, but the bond dimension required for an exact representation grows with the entanglement across each cut of the network (Section 3.1.3). Circuits that generate substantial entanglement, such as deep random circuits, are therefore costly for these methods. Dense state-vector simulation stores all $2^n$ amplitudes regardless of the entanglement, and its runtime depends on the number and type of gates and on how the simulator applies them.

### 2.2.4 Quantum Measurement

Measurement extracts classical information from a quantum state, with outcome probabilities given by the Born rule (Postulate 2.3). For the pure state in Equation (2.3), a projective measurement in the computational basis yields outcome $|x\rangle$ with probability

$$\Pr(x) = |c_x|^2. \tag{2.7}$$

Immediately after the outcome $x$ is obtained, the quantum state irreversibly collapses into the corresponding basis state $|x\rangle$. Repeating the measurement in the same basis, without intervening evolution, returns $x$ with certainty.

### 2.2.5 Strong and Weak Simulation

Classical simulation of a quantum circuit takes two forms [14], distinguished by what the simulator returns. A quantum computer yields only samples from its output distribution, since the state collapses upon measurement (Postulate 2.3), whereas a classical simulator can also compute the probabilities and amplitudes behind that distribution.

*Strong simulation* computes the amplitudes $c_x$ or the probabilities $|c_x|^2$ directly. For a circuit implementing a unitary operation $U$ on $n$ qubits, a strong simulator returns quantities such as the probability that the first qubit is measured as $0$,

$$\Pr(0) = \mathrm{Tr}\big[\big(|0\rangle\langle 0| \otimes I\big)\, U|0^n\rangle\langle 0^n|U^\dagger\big], \tag{2.8}$$

and, more generally, any amplitude $\langle x|U|0^n\rangle$, where $|0^n\rangle = |0\rangle^{\otimes n}$ and $I$ is the identity on the other $n-1$ qubits. Computing these amplitudes exactly for generic circuits is #P-hard. Specific families of circuits bypass this limit; in particular, Clifford (stabiliser) circuits admit efficient strong simulation through the Gottesman–Knill theorem (Section 2.3.3).

*Weak simulation* produces what a quantum computer itself returns, namely samples drawn from the output distribution $\{|c_x|^2\}$ of computational-basis measurements [15]. A weak simulator does not need to build the full state vector, and methods such as path summation and tensor-network contraction can sample from circuits whose state vector would not fit in memory (Sections 3.1.2 and 3.1.3).

In weak simulation, each *shot* represents one sampled measurement outcome, and more shots reduce the sampling uncertainty of estimated probabilities. How the runtime grows with the number of shots depends on how the simulator generates the samples. A state-vector simulator applies the circuit once and keeps the final state in memory, so every shot is drawn from the same stored probabilities and the runtime grows little with the shot count (Section 3.1.1). For noisy circuits, the trajectory method instead samples the noise channel at random while applying the circuit to a pure state (Section 2.4.3), and since each shot usually needs its own application of the circuit, the runtime grows roughly linearly with the shot count. Section 3.3 compares the costs of these approaches.

### 2.2.6 Quantum Gates

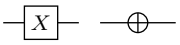
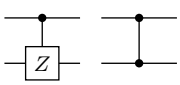
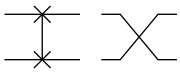
Postulate 2.4 makes the evolution of a closed system between measurements unitary, $|\psi\rangle \mapsto U|\psi\rangle$ with $U^\dagger U = I$, and the circuit model decomposes it into a discrete sequence of elementary operations called *quantum gates*. Linearity also yields the *no-cloning theorem*, which states that no physical operation can perfectly copy an arbitrary unknown quantum state [16]. Such a state can be moved or entangled with other qubits but not copied, which constrains algorithm and circuit design.

#### Single-Qubit Gates

Single-qubit gates are $2 \times 2$ unitary matrices acting on the state space $\mathbb{C}^2$ of a qubit. Up to an unobservable global phase, any single-qubit unitary is a rotation of the Bloch vector on the Bloch sphere (Section 2.2.2).

The left half of Table 2.1 lists common single-qubit gates with their circuit symbols and matrices.

**Table 2.1:** Common single-qubit gates (left) and multi-qubit gates (right), with their circuit symbols and their matrices in the computational basis. Where two symbols are shown, the second is an alternative in common use. For the two-qubit gates, the upper wire is the first qubit and the basis is ordered as $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

| Single-qubit gates | | | Multi-qubit gates | | |
|---|---|---|---|---|---|
| **Gate** | **Symbols** | **Matrix** | **Gate** | **Symbols** | **Matrix** |
| Pauli-$X$ |  | $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ | CNOT (CX) |  | $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$ |
| Pauli-$Y$ |  | $\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$ | | | |
| Pauli-$Z$ |  | $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ | Controlled-$Z$ (CZ) |  | $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$ |
| Hadamard ($H$) |  | $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ | | | |
| Phase ($S$) |  | $\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$ | SWAP |  | $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ |
| $\pi/8$ ($T$) |  | $\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$ | | | |

The Pauli operators $X$, $Y$ and $Z$ correspond, up to a global phase, to rotations by $\pi$ about the $x$-, $y$- and $z$-axes of the Bloch sphere. The Pauli-$X$ gate therefore acts as a bit-flip, exchanging the computational-basis states, $X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$. The Pauli-$Y$ gate exchanges them as well but adds phase factors, $Y|0\rangle = i|1\rangle$ and $Y|1\rangle = -i|0\rangle$. The Pauli-$Z$ gate leaves $|0\rangle$ unchanged and maps $|1\rangle$ to $-|1\rangle$, so it acts as a phase-flip.

Unlike the Pauli gates, which map each basis state to a basis state up to a phase, the Hadamard gate $H$ maps them to the equal superpositions $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$. Many quantum algorithms therefore use it to change between the computational basis and the $\{|+\rangle, |-\rangle\}$ basis.

### Multi-Qubit Gates

Unlike single-qubit gates, multi-qubit gates act on the joint state space of two or more qubits. Gates such as the controlled-NOT (CNOT) and controlled-$Z$ (CZ) can generate entanglement, whereas the SWAP gate only exchanges the states of two qubits. Their symbols and matrices appear in the right half of Table 2.1.

The CNOT gate acts on a control qubit and a target qubit and flips the target if and only if the control qubit is in the state $|1\rangle$. Its action on computational-basis states is given by

$$\text{CNOT}\,|c,t\rangle = |c,\ t \oplus c\rangle, \qquad c,t \in \{0,1\}, \tag{2.9}$$

where $\oplus$ denotes addition modulo 2. When combined with single-qubit gates, the CNOT gate enables the preparation of entangled states. For example, applying a Hadamard gate to the first qubit of $|00\rangle$, followed

by a CNOT with that qubit as the control, produces the Bell state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ (Figure 2.2).

## 2.3 Quantum Circuits

Quantum algorithms are expressed as circuits of the gates above. This section defines the circuit model, the gate sets that make it universal and the Clifford gates, whose circuits admit efficient classical simulation.

### 2.3.1 Circuit Model

The circuit model is the standard computational framework for expressing quantum algorithms [12]. In this model, a quantum computation is represented as a sequence of quantum gates acting on qubits arranged along horizontal wires, with time advancing from left to right. Figure 2.2 shows a two-qubit circuit that prepares a Bell state.

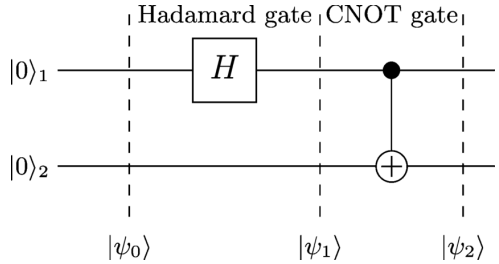



**Figure 2.2:** A two-qubit circuit illustrating Bell-state preparation. Starting from the initial state $|\psi_0\rangle = |0\rangle_1 |0\rangle_2$, a Hadamard gate is applied to the first qubit, producing $|\psi_1\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$. A CNOT gate is then applied with qubit 1 as control and qubit 2 as target, yielding the entangled Bell state $|\psi_2\rangle = |\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

If a circuit consists of the gates $U_1, U_2, \ldots, U_m$, the matrix product $U = U_m \cdots U_2 U_1$ is applied to the initial state. Each gate $U_j$ acts on one or a few qubits and is embedded into the full Hilbert space by tensoring it with identity operators on the remaining qubits.

Three quantities describe the size of a circuit, namely the number of qubits $n$, often called the circuit *width*, the total number of gates $m$ and the circuit *depth*, which is the number of sequential layers of gates when operations acting on different qubits can be executed in parallel. Applying $m$ gates that each act on a fixed number of qubits gives a state-vector simulation cost of $\mathcal{O}(m \cdot 2^n)$ before optimisations such as gate fusion (Section 3.5.2).

Measurement is represented in the circuit but treated separately from the unitary gates, since it is not itself a gate. The computational-basis measurements used in this dissertation are described by projectors (Postulate 2.3) and return classical outcomes with the probabilities of the Born rule, leaving the measured qubits in the corresponding basis states.

### 2.3.2 Universal Gate Sets

A gate set is *universal* if any unitary operation on $n$ qubits can be approximated to arbitrary precision, up to a global phase, by a finite sequence of its gates. A standard example is $\{H, T, \text{CNOT}\}$, in which the Hadamard and $T$ gates approximate any single-qubit unitary and the CNOT gate supplies the entangling operation that multi-qubit unitaries require [12]. The set $\{H, \text{Toffoli}\}$, where the Toffoli gate is a NOT controlled by two qubits, is universal in a weaker, computational sense. Its circuits can simulate any quantum computation, but both gates are real matrices and therefore cannot approximate every unitary [17].
Any other unitary gate can be built from $\{H, T, \text{CNOT}\}$, exactly or to the required precision. The controlled-$Z$ gate of Table 2.1, for instance, is equivalent to a CNOT preceded and followed by a Hadamard gate on its target qubit (Figure 2.3),

$$\text{CZ} = (I \otimes H)\,\text{CNOT}\,(I \otimes H) = |0\rangle\langle 0| \otimes HH + |1\rangle\langle 1| \otimes HXH = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes Z, \tag{2.10}$$

since $HXH = Z$ turns the controlled bit-flip into a controlled phase-flip.

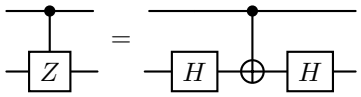



**Figure 2.3:** Controlled-$Z$ gate (left) and its decomposition into a CNOT preceded and followed by a Hadamard gate on the target qubit (right).

### 2.3.3 Clifford Gates

Clifford circuits are built from the Hadamard ($H$), phase ($S$) and CNOT gates of Table 2.1 [18], and the Bell-state circuit of Figure 2.2 is one of them. Products of these three gates give, up to a global phase, every other Clifford gate, among them the Pauli gates, $S^\dagger = S^3$ and the controlled-$Z$ gate of Figure 2.3. The $T$ gate, however, cannot be obtained from them. What sets Clifford gates apart is their action on Pauli operators, the tensor products $P$ of $I$, $X$, $Y$ and $Z$. Conjugating any such $P$ by a Clifford gate $U$ gives $UPU^\dagger = \pm P'$ for another Pauli operator $P'$ [12]. The Hadamard gate, for example, maps $X$ to $Z$, $Z$ to $X$ and $Y$ to $-Y$.[1] Although Clifford circuits can create entangled states, as Figure 2.2 shows, they are not universal, and adding the $T$ gate gives the universal set $\{H, T, \text{CNOT}\}$ of Section 2.3.2.

**Theorem 1** (Gottesman–Knill [19])**.** *Any Clifford circuit acting on the state $|0\rangle^{\otimes n}$, followed by measurements in the computational basis, can be classically simulated in polynomial time, including the computation of exact probabilities and amplitudes.*

[1] A gate $U$ maps the operator $P$ to $P'$ when $UPU^\dagger = P'$, or equivalently $UP = P'U$, so applying $P$ before the gate has the same effect as applying $P'$ after it. For the Hadamard gate, $HX = ZH$.

The theorem follows from the conjugation property. A Clifford circuit acting on $|0\rangle^{\otimes n}$ prepares a stabiliser state, which is defined uniquely by $n$ commuting Pauli operators called its stabiliser generators, and each gate in the circuit maps these generators to other Pauli operators by conjugation. A simulator can therefore track $n$ Pauli operators of $n$ letters each instead of $2^n$ amplitudes, which avoids the exponential memory of a full state vector and underlies the stabiliser methods of Section 3.1.4 [18].

Figure 2.4 follows the Bell-state circuit of Figure 2.2 in the stabiliser representation. The input state $|00\rangle$ is stabilised by $Z \otimes I$ and $I \otimes Z$. The Hadamard gate turns the first generator into $X \otimes I$. The CNOT gate, which copies an $X$ from its control to its target and a $Z$ from its target to its control, then maps the two generators to $X \otimes X$ and $Z \otimes Z$, which stabilise the Bell state $|\Phi^+\rangle$. At each step, the simulator tracks these two Pauli operators instead of four amplitudes.

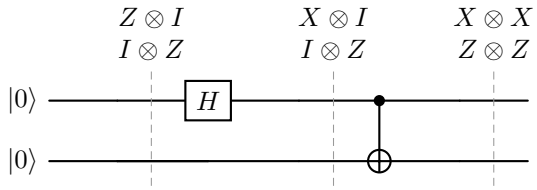



**Figure 2.4:** The Bell-state circuit of Figure 2.2 in the stabiliser representation. Above each dashed line are the two Pauli operators that stabilise the state at that point. Each gate updates them by conjugation, and the final pair stabilises $|\Phi^+\rangle$.

Clifford operations make up most of the operations in Quantum Error Correction (QEC) protocols and appear as components of larger quantum algorithms, which is why a Clifford circuit is one of the three benchmark families of this dissertation (Section 4.2). Circuits that add only a few non-Clifford gates, such as $T$ gates, can often still be simulated more efficiently than general circuits, since their Clifford parts keep a structure that simulation algorithms exploit [20].

## 2.4 Noise in Quantum Systems

The circuits of Section 2.3 evolve a closed system by unitary gates. However, on real hardware, the qubits also couple to their surroundings, so the state after a noisy gate is no longer a single vector, and a simulator of noisy circuits has to carry a different object and a model of what the hardware does to it. The object is the density matrix of Section 2.4.1. The two decoherence processes of Section 2.4.2 set the coherence times of Table 2.2, and the four channels of Section 2.4.3 are the noise models that the noise study of Section 5.6 applies.

### 2.4.1 Density-Matrix Formalism

When a quantum system interacts with its environment or undergoes imperfect operations, its state can no longer be described by a single pure state vector $|\psi\rangle$. Instead, the appropriate description is the *density*

*operator* (or density matrix), which captures statistical mixtures of pure states.

A pure state is represented by the rank-one projector $\rho = |\psi\rangle\langle\psi|$. A *mixed state* describes a classical probability distribution over an ensemble of pure states $\{|\psi_i\rangle, p_i\}$ and is written as

$$\rho = \sum_i p_i \, |\psi_i\rangle\langle\psi_i|, \qquad \sum_i p_i = 1, \quad p_i \geq 0. \tag{2.11}$$

The density operator has three defining properties. It is Hermitian ($\rho^\dagger = \rho$), positive semi-definite ($\langle\phi|\rho|\phi\rangle \geq 0$ for all $|\phi\rangle$) and normalised to unit trace ($\mathrm{Tr}(\rho) = 1$). A state is pure if and only if $\mathrm{Tr}(\rho^2) = 1$, and otherwise $\mathrm{Tr}(\rho^2) < 1$ and the state is mixed.

For a single qubit, any density matrix can be written in terms of the Bloch vector $\mathbf{r} = (r_x, r_y, r_z)$ as

$$\rho = \frac{1}{2}\big(I + r_x \, \sigma_x + r_y \, \sigma_y + r_z \, \sigma_z\big), \qquad r_x^2 + r_y^2 + r_z^2 \leq 1, \tag{2.12}$$

where $\sigma_x, \sigma_y, \sigma_z$ are the Pauli matrices. Pure states correspond to $|\mathbf{r}| = 1$, the surface of the Bloch sphere of Figure 2.1, and the maximally mixed state $\frac{1}{2}I$ sits at the origin. Noise generally moves a pure state into the interior of the sphere and lowers its purity, $\mathrm{Tr}(\rho^2) = (1 + |\mathbf{r}|^2)/2$ for one qubit. The overlap of a state with $|+\rangle$, $\langle+|\rho|+\rangle = (1 + r_x)/2$, depends only on the coherence between $|0\rangle$ and $|1\rangle$.

The density matrix also changes the simulation cost. Whereas a pure state of $n$ qubits requires $2^n$ complex amplitudes, a general density matrix is a $2^n \times 2^n$ Hermitian matrix with $\mathcal{O}(4^n)$ entries. Because $4^n = 2^{2n}$, the exponent is twice that of a pure state, so full density-matrix simulation reaches about half as many qubits as a state vector in the same memory. In double precision, the $242$ GiB available to a job on one x86 node used in this dissertation holds a density matrix of at most $16$ qubits, which motivates the trajectory-based alternative at the end of this section and the comparison of Section 3.3.

### 2.4.2 Decoherence Mechanisms

Decoherence is the process by which a quantum system loses its quantum properties due to uncontrolled coupling with its environment. In physical qubits, decoherence appears mainly as energy relaxation and dephasing, which are characterised by the timescales $T_1$ and $T_2$.

The *relaxation time* $T_1$ describes *amplitude damping*, that is, the irreversible decay of the excited state $|1\rangle$ towards the ground state $|0\rangle$. This process dissipates energy from the qubit into the environment and redistributes the population in the computational basis. By contrast, the *dephasing time* $T_2$ describes the loss of phase coherence between basis states without necessarily involving energy exchange. In this case, the populations of the basis states may remain unchanged, but the off-diagonal elements of the density matrix decay, reducing the system's ability to exhibit interference effects. The two times satisfy $T_2 \leq 2T_1$, with equality only in the absence of additional pure dephasing.

In current superconducting quantum processors, typical coherence times are of the order of tens to hundreds of microseconds [21]. The $105$-qubit Willow processor, for instance, reports mean values of $68$ µs for $T_1$ and $89$ µs for $T_2$ [3]. Table 2.2 gives typical values for the main qubit technologies. Gates and measurements must finish well within these times, which is what bounds the depth of the circuits a device runs reliably.

**Table 2.2:** Coherence times and two-qubit gate fidelities reported for recent processors of the main qubit technologies. $T_1$ is the relaxation time and $T_2$ the dephasing time, given with the pulse sequence used to measure it (CPMG, Carr–Purcell–Meiboom–Gill), and a dash marks a value that the cited work does not report. Photonic qubits have no $T_1$ or $T_2$ in this sense, and photon loss takes their place.

| Technology | Processor | Coherence | Two-qubit gate |
|---|---|---|---|
| Superconducting circuits | Google Willow, $105$ qubits [3] | $T_1 = 68$ µs and $T_2 = 89$ µs (CPMG), means over the qubits | Operates below the surface-code threshold |
| Trapped ions | Quantinuum Helios, $98$ qubits [22] | – | Infidelity $7.9 \times 10^{-4}$, fidelity $99.92\%$ |
| Neutral atoms | Rubidium arrays of Harvard, MIT and QuEra | $T_2 = 1.5$ s with dynamical decoupling [23] | CZ fidelity $99.5\%$ on up to $60$ atoms in parallel [24] |
| Semiconductor spins | Silicon quantum dots [25] | $T_2$ up to about $30$ µs with a Hahn echo | CNOT fidelity $99.51\%$ |
| Photonic | PsiQuantum dual-rail platform [26] | Limited by photon loss | Fusion fidelity $99.22\%$, conditional on photon detection and excluding loss |

Beyond the decoherence of idle qubits, gate implementations themselves are imperfect. Control errors, calibration drift, crosstalk and other hardware non-idealities can introduce both coherent and incoherent gate errors. A standard modelling approach is therefore to represent an imperfect operation as the composition of an ideal unitary transformation with an additional noise channel acting before, after or during the gate.

### 2.4.3 Noise Channels

A *quantum channel* is a Completely Positive, Trace-Preserving (CPTP) linear map $\mathcal{E}$ acting on density matrices. Any CPTP map admits a Kraus representation [12]

$$\rho \;\mapsto\; \mathcal{E}(\rho) = \sum_k E_k \rho E_k^\dagger, \qquad \sum_k E_k^\dagger E_k = I, \tag{2.13}$$

where $\{E_k\}$ are the Kraus operators of the channel. Each $E_k$ represents one event the noise can cause, applied to the state with probability $\mathrm{Tr}(E_k \rho E_k^\dagger)$, and the second condition makes these probabilities sum to one. The number and structure of the Kraus operators set the cost of simulating the channel. For a single qubit, every channel maps the Bloch ball[2] into an ellipsoid inside it [12], and the shape of the ellipsoid is what distinguishes the four channels used in Section 5.6.

[2]The Bloch ball is the set of single-qubit states, $|\mathbf{r}| \leq 1$ in Equation (2.12), that is, the sphere of Figure 2.1 together with its interior. Pure states lie on the surface and mixed states inside.

**Depolarising channel.** The depolarising channel models isotropic noise by leaving the qubit unchanged with probability $1-p$ and, with probability $p$, applying a uniformly random Pauli error,

$$\mathcal{E}_{\mathsf{dep}}(\rho) = (1-p)\rho + \frac{p}{3}\big(X\rho X + Y\rho Y + Z\rho Z\big), \tag{2.14}$$

with the four Kraus operators $\sqrt{1-p}\,I$, $\sqrt{p/3}\,X$, $\sqrt{p/3}\,Y$ and $\sqrt{p/3}\,Z$. It shrinks the Bloch vector by the factor $1-4p/3$ in every direction, and this symmetry makes it the usual generic model of gate imperfections. The same channel is also written as the replacement of the state by the maximally mixed state $I/2$ with probability $p$, which applies each Pauli error with probability $p/4$ [12]. The two forms describe the same channel when the replacement probability is $4/3$ of the Pauli-error probability, and simulation frameworks differ in which of them they call depolarising with parameter $p$ (Section 5.6).

**Bit-flip channel.** The bit-flip channel is the quantum analogue of a classical binary symmetric channel. With probability $p$ the qubit undergoes a Pauli-$X$ error, and with probability $1-p$ it remains unchanged,

$$\mathcal{E}_{\mathsf{bf}}(\rho) = (1-p)\rho + p\,X\rho X. \tag{2.15}$$

It keeps the $x$ component of the Bloch vector and shrinks the other two by $1-2p$, so the states $|\pm\rangle$ on the $x$-axis are left unchanged while $|0\rangle$ and $|1\rangle$ are mixed.

**Amplitude-damping channel.** The amplitude-damping channel models irreversible energy relaxation, the process behind $T_1$. Its Kraus operators are

$$E_0 = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{bmatrix}, \qquad E_1 = \begin{bmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{bmatrix}, \tag{2.16}$$

where $\gamma \in [0,1]$ is the damping parameter, which takes the place of the probability $p$ of the two channels above. It shrinks the two equatorial components of the Bloch vector by $\sqrt{1-\gamma}$ and the vertical one by $1-\gamma$, and shifts the whole ball towards $|0\rangle$, which it leaves fixed. It is the only one of the four channels that moves the maximally mixed state, a property called non-unitality, and as $\gamma \to 1$ it drives every state to $|0\rangle$.

**Phase-damping channel.** The phase-damping channel models pure dephasing, the process behind $T_2$, without any exchange of energy with the environment. Its Kraus operators are

$$E_0 = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{bmatrix}, \qquad E_1 = \begin{bmatrix} 0 & 0 \\ 0 & \sqrt{\gamma} \end{bmatrix}, \tag{2.17}$$

with $\gamma \in [0,1]$. It leaves the populations, and with them the vertical component of the Bloch vector, unchanged, and shrinks the two equatorial components by the same factor $\sqrt{1-\gamma}$ as amplitude damping.

The two damping channels therefore act alike on the coherences and differ only in the vertical component, which amplitude damping moves towards $|0\rangle$.

Noisy evolution is simulated either by *density-matrix propagation*, which applies Equation (2.13) to the full density matrix and yields ensemble-averaged observables in a single run [12], or by *trajectory-based simulation*, also called the Monte Carlo wave-function or quantum-jump method [27], which propagates a pure state vector, samples the action of the channel at random and repeats the run for many samples. Section 3.3 compares the cost of the two strategies, and the noise evaluation of Section 5.6 uses density-matrix propagation.

## 2.5 Summary

This chapter introduced the quantum formalism underlying classical circuit simulation. A pure state of $n$ qubits is represented by $2^n$ complex amplitudes, which evolve under unitary gates and determine measurement probabilities through the Born rule. First, the chapter distinguished strong simulation, which computes those amplitudes or probabilities, from weak simulation, which draws samples from the output distribution. Then, it showed that entanglement prevents a pure state from being written as a product of individual qubit states, but does not by itself make simulation intractable, since Clifford circuits admit polynomial-time stabiliser simulation even when they generate entanglement. After the ideal circuits, noise extended this description to mixed states, represented by density matrices and evolved through quantum channels. However, storing a full density matrix requires $4^n$ entries, whereas stochastic trajectories estimate the noisy evolution from repeated pure-state simulations with less memory. The discussion of decoherence and gate errors also connected these models to the limitations of physical qubits. Together, these concepts provide the basis for the simulation methods and their computational costs reviewed in Chapter 3.

# 3

# State of the Art: Classical Quantum Circuit Simulation

## Contents


3.1 Classical Simulation Approaches . . . . . 20
3.2 Simulation Tools and Backends . . . . . 24
3.3 Noise Simulation Strategies . . . . . 25
3.4 The Cost of State-Vector Simulation . . . . . 26
3.5 Architectural Optimisations . . . . . 28
3.6 Prior Studies and the Remaining Gap . . . . . 33
3.7 Summary . . . . . 34

Classical simulation of quantum circuits depends on the state representation and on how the simulator uses the hardware. A dense state vector stores $2^n$ complex amplitudes, so memory capacity limits the number of qubits (Table 3.2), and memory bandwidth limits how fast each gate updates the state. Other methods exploit the structure of the circuit to reduce these costs. This chapter reviews those methods, the software that implements them and the architectural optimisations that raise their performance.

Section 3.1 presents the main simulation paradigms, Section 3.2 the landscape of tools and Section 3.3 the two strategies for simulating noise. Section 3.4 then develops a cost model for dense state-vector simulation, separating the capacity limit on circuit size from the bandwidth limit on throughput, and Section 3.5 examines the optimisations that address each limit. Section 3.6 reviews prior comparative and architectural studies to identify the gap this dissertation addresses. The frameworks benchmarked here and their configurations are described in Chapter 4.

## 3.1 Classical Simulation Approaches

Simulating a quantum circuit classically requires a representation of the quantum state and a classical operation for each gate, and the cost of both depends on the representation chosen. Four main paradigms are in use, namely state-vector simulation, path summation and its hybrids with the state vector, tensor-network simulation and stabiliser simulation. The last three avoid storing all $2^n$ amplitudes by exploiting a property of the circuit, respectively a small number of non-trivial paths, limited entanglement and Clifford structure, so each offers a different trade-off among memory, runtime and the class of circuits it handles efficiently, as Table 3.1 summarises. The four subsections below take them in that order, and the comparison of Section 5.7 tests the stabiliser and tensor-network paradigms against the state vector on the benchmark circuits.

### 3.1.1 State-Vector Simulation

The Schrödinger method stores the complete state vector described in Chapter 2 and updates its amplitudes after each gate operation. The simulator never forms the $2^n \times 2^n$ matrix of a gate; it applies the gate's own $2 \times 2$ or $4 \times 4$ matrix directly to the amplitudes the gate acts on [28]. This approach constitutes a strong simulation, as defined in Section 2.2.5, because it provides access to exact amplitudes and measurement probabilities at any stage of the circuit [14].

The $2^n$ amplitudes from Equation (2.3) are stored as a single array, ordered by the bit strings $x$ of their basis states $|x\rangle$, so the index of each amplitude follows directly from its subscript, as Figure 3.1 shows for three qubits. A single-qubit gate updates this array in place by pairing each amplitude with the one whose subscript differs only in the bit of the target qubit, such as $c_{000}$ with $c_{001}$ for a gate on qubit 3, and applying its matrix to every pair. The target qubit also sets the distance in memory between the two amplitudes of a pair, the stride marked in the figure, while the number of pairs stays the same. For $n$ qubits, a single-qubit gate therefore updates $2^{n-1}$ pairs, and a two-qubit gate updates $2^{n-2}$ groups of four amplitudes. As a result, a

gate on a fixed number of qubits requires $\mathcal{O}(2^n)$ operations, and the total runtime for a circuit with $m$ gates scales as $\mathcal{O}(m \cdot 2^n)$ [28].

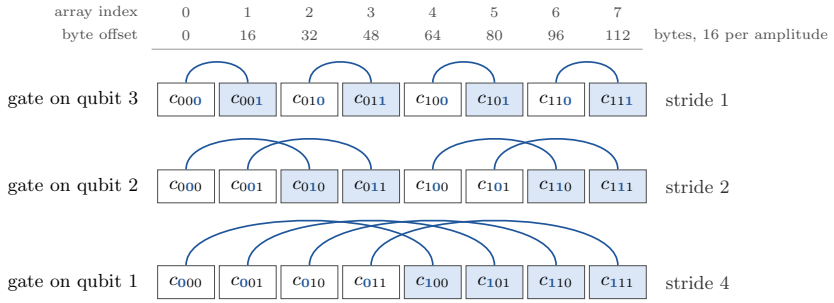



**Figure 3.1:** Amplitude pairs updated by a single-qubit gate on a three-qubit state vector. The array index and byte offset of each amplitude are shown above the cells, with 16 bytes per amplitude in double precision. Each row corresponds to a different target qubit, whose subscript bit is highlighted, and the amplitudes in which that bit is 1 are shaded. Each arc joins a pair that the gate's $2 \times 2$ matrix updates. Every row contains four pairs, while the stride, the distance between the two amplitudes of a pair, equals $2^{n-k}$ amplitudes for a gate on qubit $k$ of $n$ qubits and grows from 1 amplitude for qubit 3 to 4 amplitudes, or 64 bytes, for qubit 1.

Memory is the primary limiting factor for state-vector simulation. Each amplitude occupies 16 bytes in double-precision complex arithmetic, as indicated by the byte offsets in Figure 3.1, so the full state vector requires $16 \times 2^n$ bytes [10]. Each additional qubit therefore doubles the memory requirement, and Table 3.2 presents the corresponding sizes. The practical upper limit on current hardware is typically between 30 and 40 qubits, depending on available resources. Section 3.4 discusses the mechanisms underlying this limitation, and Chapter 5 details the observed limits on the platforms used in this dissertation.

Optimisation and parallelism shorten each pass over the state vector; however, each additional qubit still doubles both the stored state and the number of amplitudes each gate processes. Multi-CPU systems accelerate gate application by updating amplitudes in parallel [28], as do GPUs [29]. In the benchmark of 24 state-vector simulators by Gangapuram et al. [30], the runtime grows by a similar factor per added qubit in most simulators, while the runtime at a given size differs widely between them, as Figure 5.1 of Section 5.1 also shows for the frameworks benchmarked in this dissertation. State-vector simulation therefore provides exact results for arbitrary circuits, but its reach is limited by memory capacity. The subsequent alternatives avoid storing the full state vector, with their benefits depending on circuit structure and the required output.

### 3.1.2 Path-Summation and Hybrid Methods

The Feynman path-summation method computes output amplitudes without storing the state vector. For a circuit $U = U_m \cdots U_1$, inserting the computational basis between gates yields

$$\langle y|U|x\rangle = \sum_{x_1,\ldots,x_{m-1}} \langle y|U_m|x_{m-1}\rangle \cdots \langle x_1|U_1|x\rangle, \tag{3.1}$$

a sum over all sequences of intermediate basis states $x_1, \ldots, x_{m-1}$. Each such sequence is a path whose contribution is the product of the matrix elements along it, and the amplitude is the sum over all paths [31].

The simulator evaluates only the paths with non-zero contributions and branches when a gate maps a basis state to a superposition, such as the Hadamard gate, so a gate on one or two qubits generates at most two or four branches, respectively. Tracking a single path at a time requires $\mathcal{O}(n+m)$ space. However, the number of paths increases exponentially with the number of branching points. As a result, this method trades the exponential memory of state-vector simulation for a runtime that is exponential in the number of branching gates rather than in the qubit count.

The Schrödinger–Feynman hybrid combines the two methods by dividing the qubits into two blocks, each represented by a smaller state vector (Figure 3.2) [31]. Each gate across the cut is decomposed into a sum of products of operators acting separately on the two blocks, so a controlled gate such as CNOT or CZ requires two terms. Choosing one term per cut gate then defines a path along which the blocks evolve independently; summing the path contributions recovers the amplitude. For $g$ controlled gates across an equal partition, this gives $2^g$ paths, each involving two state vectors of about $2^{n/2}$ amplitudes. The memory reduction therefore comes at the cost of a number of paths that grows exponentially with the number of cut gates. The hybrid can be particularly useful when the full state vector exceeds available memory; at smaller sizes, direct state-vector simulation avoids that additional work [31].

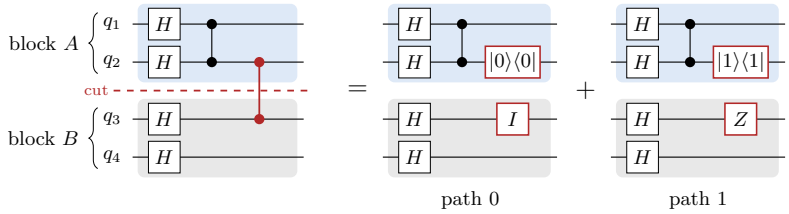



**Figure 3.2:** Schrödinger–Feynman hybrid on four qubits. The qubits are split into block $A$ ($q_1$, $q_2$) and block $B$ ($q_3$, $q_4$), and the controlled-$Z$ on $q_2$ and $q_3$ is the only gate across the cut. Writing it as $|0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes Z$, as in Equation (2.10), turns the circuit into the sum of two paths in which no gate crosses the cut, so each path is the product of two independent block simulations. With $g$ such controlled gates across the cut, the sum has $2^g$ paths.

### 3.1.3 Tensor-Network Simulation

Tensor-network simulators exploit the entanglement structure of the circuit. The quantum state or the circuit's action is represented as a connected network of smaller tensors, which reduces memory and computational requirements [32] when the entanglement of the state is limited [33]. The Matrix Product State (MPS) representation suits circuits that generate predominantly one-dimensional, nearest-neighbour entanglement, where the required bond dimension $\chi$ can remain bounded even as the number of qubits $n$ increases (Figure 3.3).

Under these conditions, qubit counts that are inaccessible to dense simulation methods become feasible. For instance, Pasquale et al. demonstrate a 400-qubit variational circuit on the tensor-network backend of Qibo, with a runtime that grows slowly across the measured range (Figure 3.4) [32]. This comparison is revisited in Figure 5.12 of Section 5.7 using benchmark circuits. Additionally, a large-scale benchmark by Leonteva et al. of seven emulators, based on matrix product states, tensor networks, decision diagrams

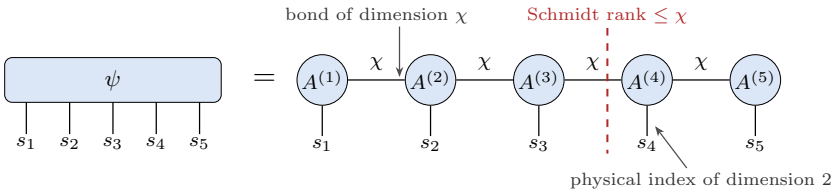



**Figure 3.3:** Matrix product state of five qubits. The state, drawn as a single tensor $\psi$ with one leg per qubit and $2^5$ entries, is factorised into one tensor $A^{(i)}$ per qubit, each with a physical index $s_i$ of dimension 2 and bonds of dimension $\chi$ to its neighbours. The chain stores $\mathcal{O}(n\chi^2)$ numbers instead of $2^n$. The Schmidt rank across any cut of the chain, such as the dashed one, is at most $\chi$, so the factorisation is exact only for states whose entanglement stays within that bound.

and factorised kets, across thirteen circuit families demonstrates that circuits of at least 100 qubits can be simulated within five minutes for twelve of the families, whereas the random circuit scales exponentially with every emulator [34].

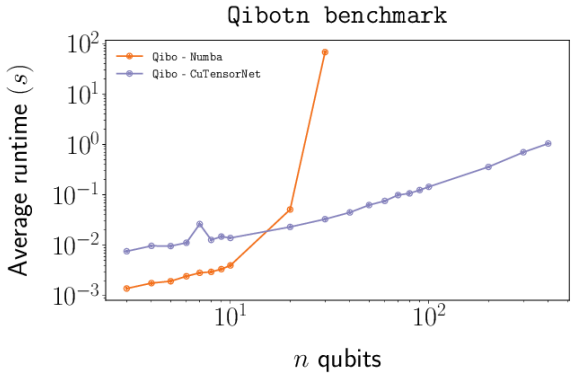



**Figure 3.4:** Total simulation time of a shallow variational circuit, with alternating layers of parameterised $R_Y$ rotations and entangling CZ gates, in Qibo against the number of qubits $n$, on the state-vector backend `qibojit` with Numba on an AMD EPYC 7713 processor and on the tensor-network backend `qibotn` with `cuTensorNet` on an NVIDIA A100 GPU. The state-vector curve is shown up to 30 qubits and grows exponentially, whereas the tensor-network measurements extend to 400 qubits with much slower runtime growth. Adapted from Pasquale et al. [32].

These performance gains depend on both the circuit structure and the chosen tensor-network representation. For an MPS, maintaining small bond dimensions ensures that storage and updates remain efficient, whereas for a general tensor network, the contraction order also determines the size of intermediate tensors. Truncating small singular values or capping $\chi$ further reduces computational cost, but introduces approximation error. In contrast, highly entangling circuits such as deep Random Quantum Circuits (RQCs) may require exponentially increasing bond dimensions, making exact MPS simulation computationally expensive. To reduce the runtime of large contractions, libraries such as NVIDIA's `cuTensorNet` utilise GPUs and distribute computation across nodes using MPI [29]. One strategy is to divide the contraction into smaller tasks by fixing selected internal indices to specific values. These tasks execute independently, and their results are summed to reconstruct the full contraction.

### 3.1.4 Stabiliser Simulation

Clifford circuits can be simulated in polynomial time via the Gottesman–Knill theorem (Section 2.3.3). A stabiliser simulator does not store amplitudes; instead, it represents the quantum state using its Pauli generators, as illustrated in Figure 2.4 for the Bell-state circuit, within a binary tableau of $\mathcal{O}(n^2)$ bits [18]. Each Clifford gate updates only the columns corresponding to the qubits it acts upon, requiring $\mathcal{O}(n)$ bit operations. The tableau size remains independent of circuit depth and entanglement, enabling efficient simulation of Clifford circuits with thousands of qubits. Figure 3.5 demonstrates these updates for the Bell-state circuit of Figure 2.2. Stim implements the tableau method efficiently for quantum-error-correction studies [35], and its performance is compared with the Clifford backends of Qiskit Aer and Qibo in Section 5.7.

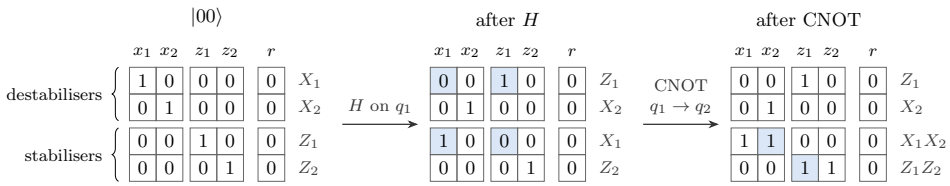



**Figure 3.5:** Stabiliser tableau of the Bell-state circuit of Figure 2.4. Each row encodes one Pauli operator on $n = 2$ qubits with two bits per qubit ($10$, $01$ and $11$ for $X$, $Z$ and $Y$) and a sign bit $r$, and the operator is written beside the row. The lower $n$ rows hold the stabiliser generators and the upper $n$ rows the destabilisers used during measurement. The Hadamard gate on $q_1$ swaps columns $x_1$ and $z_1$, the CNOT adds column $x_1$ to $x_2$ and column $z_2$ to $z_1$, and the changed bits are shaded. The final stabilisers are $X_1X_2$ and $Z_1Z_2$, and the tableau holds $2n(2n+1)$ bits, whatever the circuit.

Most circuits of practical interest break the stabiliser structure. Non-Clifford gates such as $T$ take the state outside the stabiliser representation, and extended stabiliser methods handle them by writing the state as a superposition of stabiliser states, whose number grows exponentially with the non-Clifford gate count, $t$, rather than with the total number of qubits, $n$. Low-rank decompositions have simulated circuits of 40 to 50 qubits with more than 60 non-Clifford gates [36], and related stabiliser methods have been accelerated on GPUs [37]. Whether the method outperforms dense simulation depends on both the number of qubits and the number of non-Clifford gates, and Table 3.1 compares its cost with that of the other three paradigms.

## 3.2 Simulation Tools and Backends

Many software frameworks implement these paradigms. Five of them, Qiskit Aer, CUDA-Q, Qulacs, Qibo and QuEST, are benchmarked in this dissertation and described in Section 4.3.1, and the broader ecosystem also comprises tools that were not benchmarked.

Intel-QS targets Intel architectures with compiler-assisted Advanced Vector Extensions 512 (AVX-512) vectorisation and MPI distribution, but does not support GPU execution. Google's qsim, the simulation backend of `Cirq`, employs hand-written Advanced Vector Extensions 2 (AVX2) intrinsics and cache blocking, while its hybrid variant qsimh implements the Schrödinger–Feynman decomposition. Similarly, PennyLane

**Table 3.1:** Comparison of classical quantum circuit simulation paradigms. *Strong* simulation computes amplitudes or probabilities exactly or to a specified precision; *weak* simulation only samples from the output distribution (cf. Section 2.2.5). Here $n$ is the number of qubits, $m$ the number of gates, $g$ the number of controlled gates across the cut of the hybrid method, $\chi$ the bond dimension of a matrix product state, $t$ the number of $T$ gates and $\alpha \leq 0.3963$ the exponent of the stabiliser-rank bound $2^{\alpha t}$ [38].

| Method | Time | Space | Type | Best suited for |
|---|---|---|---|---|
| State-vector | $\mathcal{O}(m \cdot 2^n)$ | $\mathcal{O}(2^n)$ | Strong | Exact results, $n \leq 30$–$40$ |
| Feynman (path-sum) | $\mathcal{O}(4^m)$[b] | $\mathcal{O}(n+m)$ | Both | Few branching gates |
| Hybrid (SF) | $2^g \cdot \mathcal{O}(m\, 2^{n/2})$ | $\mathcal{O}(2^{n/2})$ | Strong | Memory-limited, moderate depth |
| Tensor network | Depends on $\chi$, treewidth | $\mathcal{O}(n \cdot \chi^2)$[a] | Both | Low-entanglement, large $n$ |
| Extended stabiliser | poly$(n, m)\, 2^{\alpha t}$[c] | Poly$(n)$ per stab. state | Strong[c] | High $n$, low $T$-count |

[a] For MPS with bond dimension $\chi$; general tensor-network contractions depend on the network treewidth.
[b] Worst-case branching factor of four per two-qubit gate; the general statement is $2^{\mathcal{O}(m)}$.
[c] Stabiliser-rank bound for estimating a probability to a given relative error [38], not the runtime of every implementation; stabiliser-rank methods are also deployed for weak simulation by sampling from the decomposition.

LIGHTNING provides AVX2 and AVX-512 intrinsics for pure-state simulation, whereas MQT DDSIM replaces explicit state vectors with decision diagrams [39] and QUIMB offers tensor-network simulation in Python, including contraction on GPUs [40]. Finally, QRACK combines dense propagation with stabiliser, Schmidt-decomposition and decision-diagram methods that engage adaptively, and such adaptive combinations fall outside the comparison of dense state-vector backends in this dissertation.

Table 4.6, at the end of Chapter 4, compares the capabilities of the benchmarked frameworks and of selected tools from the broader ecosystem. Fewer than half of the listed simulators support all four execution modes, namely CPU, GPU, multithreading and distributed execution. However, these capabilities indicate only where each tool can run and not how its optimisations affect performance, which Sections 3.4 and 3.5 examine.

## 3.3 Noise Simulation Strategies

The noise channels described in Section 2.4.3 are incorporated into classical simulations using either density-matrix propagation or pure-state trajectory sampling. The former approach requires greater memory resources, while the latter demands increased computational effort.

**Density-matrix propagation.** In this approach, the simulator applies the Kraus map at each noisy location directly to the full $2^n \times 2^n$ matrix $\rho$, yielding ensemble averages without sampling error. Storing $\rho$ requires $\mathcal{O}(4^n)$ memory [12], which corresponds to $64$ GiB for $16$ qubits and $256$ GiB for $17$ qubits in double precision, excluding temporary allocations. Consequently, the maximum number of qubits that can be simulated is determined by the available node memory and is typically limited to approximately $16$ qubits on nodes with several hundred GiB. Tensor-network representations can reduce memory requirements when the mixed state has small bond dimensions [41]. For general circuits, however, the memory footprint remains prohibitive.

**Stochastic trajectories.** The Monte Carlo wave-function method instead evolves a pure state, maintaining memory usage at $\mathcal{O}(2^n)$. At each noisy location, a Kraus operator $E_k$ is selected with probability $p_k = \langle\psi|E_k^\dagger E_k|\psi\rangle$, and the state is updated to $E_k|\psi\rangle / \sqrt{p_k}$ [42]. Averaging over $N_{\text{traj}}$ independent trajectories recovers the density-matrix result, with statistical uncertainty scaling as $1/\sqrt{N_{\text{traj}}}$. Each trajectory incurs computational cost comparable to a noiseless run. Thus, the reduction in memory requirements is offset by the need for repeated evolution. However, trajectories can be executed in parallel.

The choice between these strategies also depends on the methods available in each framework. Qiskit Aer supports both approaches and constructs noise models from device calibration data. Qulacs and Qibo offer trajectory sampling via built-in noise gates, as well as density-matrix simulators suitable for exact simulations at small system sizes. CUDA-Q introduced GPU-accelerated trajectory simulation in version 0.10 and provides density-matrix propagation only on a CPU target [43], without device-calibrated models. However, the published comparisons in Section 3.6 do not address noisy simulation, and Section 5.6 examines the cost and accuracy of density-matrix simulation in these frameworks on the platforms used in this dissertation.

## 3.4 The Cost of State-Vector Simulation

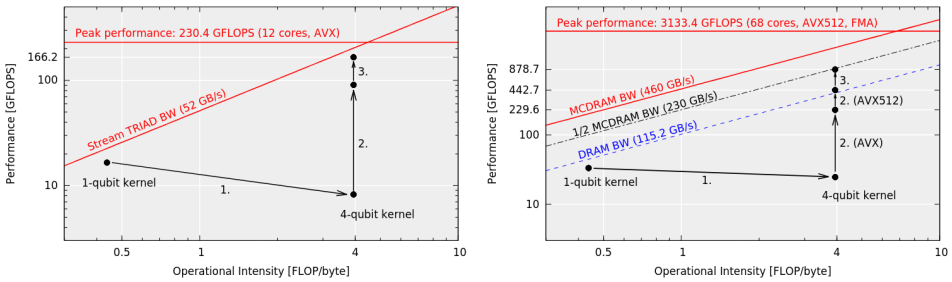
State-vector simulation is limited by two principal resources, memory capacity and memory bandwidth. Memory capacity determines the maximum register size, as an $n$-qubit state vector requires $16 \times 2^n$ bytes in double precision, which makes capacity a hard wall. Memory bandwidth generally constrains the rate at which the state can be updated. Greater memory enables larger registers, while higher bandwidth or fewer passes over the state reduce execution time. Table 3.2 summarises the capacity requirements and corresponding hardware examples.

**Table 3.2:** Memory required for a state vector in double-precision (64-bit) floating-point format, with two 64-bit components per complex amplitude (16 bytes), as a function of qubit count, and examples of hardware that can accommodate it. Values exclude temporary allocations and simulator overhead.

| Qubits ($n$) | Memory | Example hardware |
|---|---|---|
| 20 | 16 MiB | Laptop / desktop |
| 25 | 512 MiB | Single CPU socket |
| 30 | 16 GiB | Single GPU (an A100 holds $n = 31$ with 40 GB and $n = 32$ with 80 GB) |
| 33 | 128 GiB | Single compute node |
| 35 | 512 GiB | Dual-socket node |
| 40 | 16 TiB | Multi-node cluster |
| 45 | 512 TiB | Leadership-class supercomputer |
| 50 | 16 PiB | Beyond current systems; reached with two bytes per amplitude [44] |

The practical memory limit is lower than the installed memory suggests, as simulators require additional temporary storage. For systems with fewer than approximately 30 qubits, a single GPU or workstation is sufficient. Up to about 34 qubits, a node with 512 GiB can accommodate the state, which is the range where

the Non-Uniform Memory Access (NUMA)-aware optimisations discussed in Section 3.5.3 are relevant. When the memory requirement exceeds that of a single node, distributed execution using MPI increases capacity but introduces inter-node communication, which may dominate the runtime [10]. For simulations involving 45 qubits or more, only leadership-class supercomputers have performed full state-vector simulation, and even then, a single circuit evaluation may require several hours [45]. Employing reduced storage precision is an alternative; for example, the JUQCS-50 simulation of 50 qubits on the JUPITER exascale system stores each amplitude in two bytes instead of sixteen, reducing the storage requirement by a factor of eight [44].




(a) Roofline plot for one Edison socket.

(b) Roofline plot for one KNL node of Cori II.

**Figure 3.6:** Roofline plots for a single gate applied to the $2^n$-element state vector of $n$ qubits. (a) On an Edison socket (Intel Xeon E5-2695 v2, AVX), an unfused single-qubit kernel sits in the memory-bound region near $0.5$ FLOP/byte; gate fusion moves the kernel rightward to a four-qubit kernel, and explicit AVX vectorisation with register blocking then raises attained performance at that fixed intensity, together reaching $\sim$72% of peak. (b) On a Cori II KNL node (Xeon Phi 7250, AVX-512 with Fused Multiply-Add (FMA)), the MCDRAM bandwidth ceiling lifts the memory-bound roof, and the fused AVX-512 kernel nearly reaches the compute peak. Unfused single-qubit gates remain bandwidth-limited throughout. Adapted from Häner and Steiger [10], Fig. 2.

While memory capacity determines whether the state fits, the cost of updating it depends on the arithmetic intensity of the gate kernels, the number of floating-point operations per byte moved between memory and processor. For a general single-qubit gate, this ratio is small, since applying its $2 \times 2$ complex matrix to a pair of amplitudes takes four complex multiplications and two complex additions, 28 real floating-point operations in all, while reading and writing the two double-precision amplitudes once moves 64 bytes [9], which gives $0.4375$ FLOP/byte. This estimate assumes that each amplitude passes through main memory once per gate; the operation count does not depend on the instruction set, whereas the actual traffic depends on cache reuse and on the implementation. Gates with less arithmetic lie lower still, since a diagonal gate needs 12 operations per pair and a CNOT none, so circuits dominated by such gates have lower intensities (Section 5.5). Measurements confirm the estimate. Häner and Steiger obtained approximately $0.5$ FLOP/byte for the unfused single-qubit kernel of their simulator in the roofline analysis on Intel Xeon and Xeon Phi nodes reproduced in Figure 3.6 [10], and the analytical model of Pratipat gives $0.38$ FLOP/byte or less for every gate used in a study of eleven backends on a unified-memory system [46]. All of these values lie well below the ridge points of current CPUs and GPUs, which sit at several FLOP/byte, so unfused gate application is memory-bound.

Vectorisation and fusion address different aspects of the computational model. Using wider vector instructions increases arithmetic throughput without changing arithmetic intensity, but provides limited benefit when memory bandwidth is the main constraint. In contrast, fusion combines multiple gates into a single update, increasing arithmetic per traversal and shifting the operating point to the right. Of these optimisations, only fusion affects the horizontal position in Figure 3.6. Section 3.5 examines both effects, and Chapter 5 presents the capacity, bandwidth and computational limitations of the platforms evaluated in this dissertation.

## 3.5 Architectural Optimisations

For registers that fit in memory, dense state-vector simulators employ a common state representation; thus, runtime differences arise from the organisation of gate updates and hardware utilisation in each implementation. Systematic architecture-aware tuning achieves several-fold reductions in wall-clock time at a fixed circuit size. As computational work doubles with each additional qubit, a sixfold performance improvement extends the register size that can be simulated within a fixed time budget by between two and three qubits. Contributions include hardware-aware optimisations in the HPQC [47] and PAS [48] simulators, the low-level and NUMA-aware extension of QUEST [49], optimisation layers within production frameworks and a review of acceleration techniques encompassing CPUs, GPUs and Field Programmable Gate Arrays (FPGAs) [50]. The recurring optimisation axes identified in this literature organise the remainder of this section, and Table 3.3 summarises which surveyed tools exploit each axis.

### 3.5.1 Vectorisation

Gate application is inherently data-parallel because the same $2 \times 2$ or $4 \times 4$ matrix-vector product applies independently to multiple amplitude groups. This structure is well suited to Single Instruction, Multiple Data (SIMD) execution, where a single instruction operates on several values simultaneously. On x86 processors, AVX2 provides 256-bit vectors that hold four double-precision values, while AVX-512 extends this width to eight double-precision values. However, register width alone does not determine the overall benefit. The masking and shifting required to identify amplitude pairs can hinder compiler auto-vectorisation [49]. To address this limitation, hand-written kernels use intrinsics, compiler-provided functions that allow programmers to specify vector operations explicitly.

Implementations with hand-written intrinsics or assembly instructions report larger gains than compiler vectorisation. QSIM, PENNYLANE LIGHTNING, PROJECTQ and QULACS employ hand-written vector kernels, as does QISKIT AER for its dense and diagonal matrix kernels [51], whereas mainline QUEST relies on compiler vectorisation. Rezaei et al. quantified the gain of each successive optimisation on a Hadamard-gate benchmark spanning 23 to 35 qubits on a dual-socket Intel Xeon Gold 6230R [49]. Compared to the compiler-vectorised

AVX2 baseline, an array-of-structures layout achieves a speedup of approximately $1.2$. Incorporating hand-written AVX2 intrinsics increases the cumulative speedup to about $5$, while AVX-512, fused multiply-add, loop unrolling and software prefetching collectively raise it to about $6$. Therefore, most of the gain results from replacing compiler vectorisation with hand-written kernels rather than from the wider registers.

These gains remain bounded by the cost model of Section 3.4, since wider registers raise the operating point vertically without moving it towards the ridge. Consequently, hand-written 512-bit kernels outperform AVX2 kernels only when the kernel is not already memory-bound, as Section 5.5 shows for AVX2 versus scalar code.

### 3.5.2 Gate Fusion

Gate fusion merges consecutive gates into a single unitary, formed by multiplying their matrices, before any of them is applied to the state vector. Thus, a sequence of $k$ single-qubit gates acting on one qubit reduces to a single $2 \times 2$ matrix, while a sequence spanning $k$ distinct qubits reduces to a $2^k \times 2^k$ matrix applied to $2^{n-k}$ amplitude groups. Although the final result remains unchanged, the arithmetic count differs, since gates on the same qubit reduce the number of operations, whereas a dense $2^k \times 2^k$ matrix over distinct qubits can require more operations than the original gates.

The gain comes from the reduction in memory traffic. Under the one-pass-per-gate model described in Section 3.4, an unfused sequence traverses the state vector once for each gate, whereas a fused update traverses it once for the entire block (Figure 3.7). Consequently, the number of bytes moved decreases in proportion to the number of gates fused. As the fused matrix spans more qubits, the arithmetic per traversal increases, raising the arithmetic intensity and shifting the operating point rightwards towards the compute ceiling.

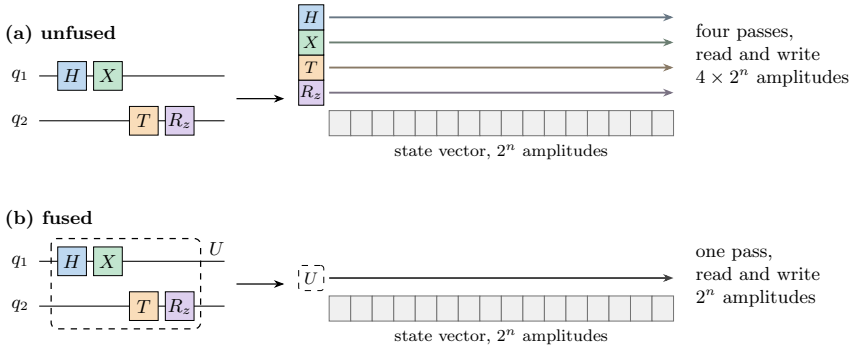



**Figure 3.7:** Gate fusion reduces the passes over the state vector, shown for two gates on each of two qubits. (a) Applied one at a time, each gate reads all $2^n$ amplitudes of the state vector of $n$ qubits, updates them and writes them back, so the four gates make four passes over the array. (b) Fused into a single $4 \times 4$ unitary $U$ before any of them is applied, the same gates need one pass. With each gate counted as a general $2 \times 2$ matrix, the bytes moved fall by a factor of four, while the floating-point count falls from 56 to 30 operations per amplitude, so the arithmetic intensity roughly doubles; Figure 3.6 shows the resulting rightward shift of the operating point.

Published studies demonstrate these benefits across implementations. In the Queen simulator, Wang et al. obtained $11.5$ times the speed of Qiskit Aer and `cuQuantum` run without transpilation on an optimisation circuit once fusion was enabled [52], although the contribution of fusion was not isolated. With fusion as the only optimisation under study, the QMin strategy of Xu et al. achieved a mean speedup exceeding $2.03$ times in total execution time over previous fusion methods [53]. Consistent with these measurements, the analytical model of Shi et al. for their vector-length-agnostic ARM implementation estimates that a fusion width of three increases the arithmetic intensity from $0.43$ to $1.93$ FLOP/byte [54].

However, increasing fusion block size does not always improve performance, because a dense block spanning $k$ qubits contains $4^k$ matrix entries, so its storage and arithmetic requirements grow rapidly with $k$. Beyond a certain width, therefore, the additional operations and the increased pressure on registers and cache can outweigh the benefits of reduced memory traffic, and the optimal fusion width depends on both the gates combined and the underlying hardware. The benchmarked frameworks differ in whether and how they apply it. Qiskit Aer and CUDA-Q apply fusion by default, Qulacs and Qibo offer it as an optional circuit pass, and mainline QuEST provides no fusion pass, although its general multi-qubit gate accepts a product matrix that the user forms beforehand.

### 3.5.3 Thread Placement and Memory Affinity

Thread-level parallelism is typically achieved using OpenMP, which parallelises the innermost loops of gate application. Within a node, distributing the updates across cores reduces runtime until memory bandwidth limits further gains [45]. On multi-socket systems, however, the gain also depends on where the threads run relative to the data, because memory is physically distributed across NUMA domains, one or more per socket, and an access to memory attached to another socket must traverse the socket interconnect. Rezaei et al. report a latency of approximately 100 ns for local reads and about 150 ns, with reduced bandwidth, for remote reads [49].

Figure 3.8 illustrates the implication for state-vector simulation, in which the amplitude array is divided into two partitions, each residing in one domain, so that the threads on a socket mainly access their local partition. Most simulators enforce neither side of this arrangement, however, since the operating system's default first-touch policy decides where the pages land and thread pinning is left to the user through environment variables [49], as the thread-pinning column of Table 3.3 shows.

Two complementary techniques close this gap. NUMA-aware allocation employs `mmap` with `mbind` to restrict the state vector to a single domain or to distribute it evenly across both, and locality-sensitive scheduling pins half the threads to each socket, so that a state which fits within one domain, as for up to 34 qubits on the referenced system, is accessed locally throughout. Combined with the kernel optimisations of Section 3.5.1, these techniques yielded speedups of approximately $4$ times for RQCs and $1.8$ times for the QFT on complete circuits [49].

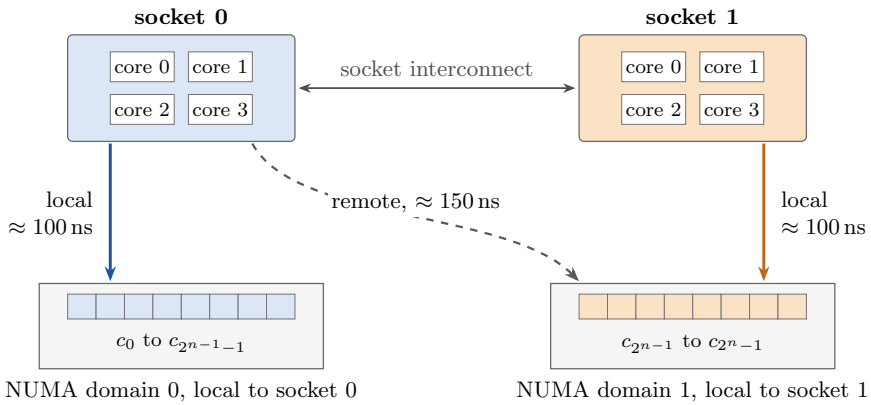



**Figure 3.8:** Memory affinity on a dual-socket node. The $2^n$ amplitudes of the state vector of $n$ qubits are split into two halves, each resident in the NUMA domain attached to one socket, so that the threads on a socket read mainly their own half along a local path at roughly $100$ ns. Without explicit placement, the Linux first-touch policy may leave a thread reading the other half across the socket interconnect, at roughly $150$ ns and lower bandwidth; that link is termed UPI on recent Intel platforms and Infinity Fabric on AMD. Latencies as reported by Rezaei et al. [49].

### 3.5.4 GPU Execution

GPUs offer both high memory bandwidth, approximately 1.6 TB/s on an NVIDIA A100 with 40 GB and 2 TB/s on the 80 GB model [55], and thread-level parallelism across thousands of concurrent threads [29]. In this execution model, the state vector is mapped to global memory, and each gate is applied as a kernel updating $2^{n-k}$ amplitude groups in parallel for a $k$-qubit gate. Since each kernel reads and writes the whole array while performing only a small matrix-vector product per amplitude group, gate application is memory-bound on the GPU for the same reason as on the CPU (Section 3.4). Measurements on the `cuStateVec` library confirm this, with gate application sustaining 78 to 83% of the peak memory bandwidth of an A100 and remaining memory-bound for gates of up to five target qubits in double precision [29]. All five benchmarked frameworks run on GPUs, either through `cuStateVec` or through their own Compute Unified Device Architecture (CUDA) kernels[1], as Section 4.3.1 details for each.

Memory capacity nevertheless remains a limiting factor that bandwidth cannot overcome, as a single A100 with 80 GB holds 32 qubits in double precision and an eight-GPU node 35 qubits. Beyond this size, the state vector must be distributed across nodes, and the resulting communication cost depends on the interconnect [56]. On newer systems where the NVLink domain spans several nodes, for instance, multi-node executions of CUDA-Q achieved $2.8$ to $4.1$ times higher performance than over InfiniBand [56], and Section 5.4.2 measures the same contrast between NVLink and InfiniBand on Deucalion.

[1] CUDA is NVIDIA's platform for programming its GPUs, and a CUDA kernel is a function that the GPU runs on many threads in parallel, each thread updating its own part of the state vector.

### 3.5.5 Distributed Execution

When the state vector exceeds the memory of a single node, it is divided among $P$ ranks, with $P$ a power of two and typically one rank per device. Each rank then stores a contiguous slice of $2^n/P$ amplitudes. Because the slices are contiguous, the $\log_2 P$ high-order bits of an amplitude's index identify its slice, and hence its rank, while the remaining bits give its position within that slice (Figure 3.9). Accordingly, qubits corresponding to the low-order bits are termed local, because a gate acting on them combines only amplitudes within one slice, whereas those corresponding to the high-order bits are termed global [10].

This distinction determines which gates require communication. A gate on a local qubit combines only amplitudes stored on the same rank, so each rank applies it to its own slice independently. A gate on a global qubit, by contrast, combines amplitudes that lie on different ranks, so the ranks must first exchange parts of their slices over the interconnect, at a cost set by its bandwidth. The only exceptions are gates that do not mix amplitudes across slices, such as a diagonal gate, which rescales each amplitude in place, or a controlled gate whose global qubit is only the control, which decides on which slices the gate acts; these need no communication even on global qubits. Reducing the exchanges caused by the remaining gates is the aim of the two techniques that follow.

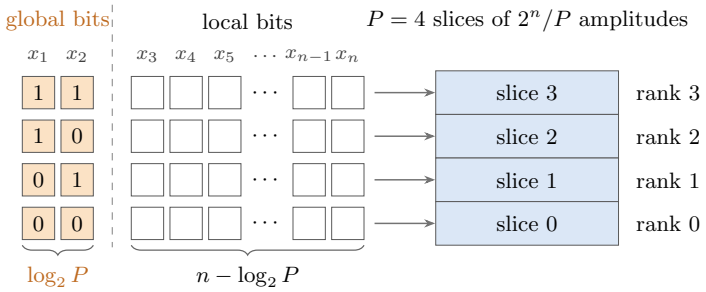



**Figure 3.9:** State-vector distribution across $P = 4$ ranks. The $\log_2 P$ high-order bits of the $n$-bit amplitude index, here $x_1x_2$, are the *global* bits, which identify the slice and therefore the rank storing it, while the remaining $n - \log_2 P$ *local* bits identify the position within that slice. Bit $x_k$ corresponds to qubit $k$, as in Figure 3.1. A gate on a local qubit combines amplitudes within individual slices; a gate that mixes amplitudes of different slices, such as a non-diagonal gate targeting a global qubit, requires data exchange between ranks. Adapted from Bayraktar et al. [29], Fig. 3.

Qubit reordering reduces repeated communication by altering the distribution of amplitudes. An index-bit swap exchanges a global bit with a local bit, consolidating the amplitude pairs required for a gate onto the same rank. This enables the gate, as well as subsequent gates with local targets, to execute locally. However, this rearrangement incurs communication overhead, as each rank must exchange half of its slice with a partner. Libraries such as `cuStateVec` provide dedicated routines that perform these index-bit swaps on a single GPU, across several GPUs and across nodes [29]. The advantage arises from executing several gates after each rearrangement, thereby distributing the communication cost across multiple updates.

Cache blocking systematises this amortisation. Instead of reordering whenever a gate targets a global qubit, the simulator restructures the circuit into blocks of gates that operate exclusively on local qubits. Consequently, index-bit swaps position the qubits of the next block locally, allowing all gates within the block

to execute without communication [57]. The term derives from the analogous technique of loop blocking for cache reuse. Adamski et al. applied the approach at scale with QuEST, which does not implement cache blocking, by rearranging the gates of the QFT by hand [58]. As a result, communication volume increases with the number of blocks rather than the number of gates. The benefit is maximised when many gates are grouped within each block, which is determined by the circuit's connectivity and gate ordering.

All five benchmarked frameworks distribute the state vector in this manner, except Qibo, which divides it only among the GPUs of a single machine (Section 4.3.1). The exchange scheme used by each framework is described in Section 4.3.2, and Table 3.3 summarises these capabilities alongside the other optimisation axes discussed in this section.

**Table 3.3:** CPU-side hardware-optimisation support in state-vector simulators. Rows in **bold** are the simulators benchmarked in this dissertation. Data partially adapted from Rezaei et al. [49]; the CUDA-Q, Qibo and `qibojit` rows are sourced independently from the respective documentation, and the Qiskit Aer vectorisation, cache-blocking and MPI entries and the Qulacs MPI entry are corrected against that source on the basis of the documentation and source code of the two frameworks.

| Simulator | Vectorisation | Manual unroll | Cache blocking | OpenMP | Thread pinning | NUMA | MPI |
|---|---|---|---|---|---|---|---|
| **Qiskit Aer** | AVX2 (intrin.) [51] | ✗ | ✓ | ✓ | – | ✗ | ✓ |
| **CUDA-Q**[a] | – | – | – | ✓ | – | ✗ | ✓ |
| **Qulacs** | AVX2 (intrin.) | ✗ | ✗ | ✓ | – | ✗ | ✓[b] |
| **Qibo**[c] | NumPy/BLAS | ✗ | ✗ | ✗ | – | ✗ | ✗ |
| **Qibo-qibojit**[c] | JIT | ✗ | ✗ | ✓ | – | ✗ | ✗ |
| PennyLane Lightning | AVX2/512 (intrin.) | ✓ | ✗ | ✓ | *likely* | ✗ | ✓ |
| ProjectQ | AVX (intrin.) | ✗ | ✓ | ✓ | – | ✗ | *fork* |
| qsim | AVX2/FMA (intrin.) | ✗ | ✓ | ✓ | – | ✗ | ✗ |
| HpQC / PAS[d] | AVX2 (intrin.) | ✗ | ✗ | ✓ | – | ✗ | ✗ |
| **QuEST** (baseline)[e] | AVX2 (comp.) | ✗ | ✗ | ✓ | – | ✗ | ✓ |
| QuEST (optimised)[f] | AVX-512 (intrin.) | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Intel-QS | AVX-512 (comp.) | ✗ | ✗ | ✓ | *likely* | ✗ | ✓ |
| NWQ-Sim | AVX2 (comp.) | ✗ | ✗ | ✓ | – | ✗ | ✓ |

[a] GPU-centric; on the CPU, vectorisation and threading are left to its runtime backend, and MPI distribution applies only to the multi-GPU target.
[b] Optional MPI build since version 0.6.0 [59], mpiQulacs [60].
[c] Threading from NumPy or, for `qibojit`, the `Numba` JIT; no MPI distribution of the state vector.
[d] Not openly available; entries inferred from the publications [47, 48].
[e] Characterised at v3.7, the base of the optimised extension; v4.2.0, used for all runs, adds NUMA-aware allocation [61].
[f] Extension by Rezaei et al. [49]; cache blocking applied to the QFT; v4.2.0 adopts NUMA-aware allocation (note e).
*comp.* = compiler auto-vectorisation; *intrin.* = hand-written intrinsics; *likely* = affinity only through runtime or OpenMP settings; "–" = not applicable or no explicit support; *fork* = third-party fork only.

## 3.6 Prior Studies and the Remaining Gap

The studies reviewed in this chapter fall into two overlapping strands. The first consists of comparative benchmarks, which evaluate many simulators under given conditions. Some of the most comprehensive are the 24-simulator benchmark by Gangapuram et al. (Section 3.1.1) [30] and the emulator benchmark by Leonteva et al. (Section 3.1.3) [34], complemented by a comparison of state-vector and tensor-network methods on GPUs across eight subroutines [62] and by a benchmark of seven state-vector simulators on a

multicore node [63]. However, these studies generally leave open the hardware mechanisms responsible for the rankings they report.

The second strand, architectural studies, explains those mechanisms for individual simulators or specific machines, as the roofline, vectorisation, NUMA and multi-GPU analyses of Sections 3.4 and 3.5 have shown. The same approach has inspired processing-in-memory designs [64] and clarified the GPU acceleration of Qiskit Aer [65]. On GPUs, moreover, Vallero et al. attributed a decline in `cuStateVec` performance beyond 22 qubits to the cost of applying each gate, which then outweighs the advantage of its asynchronous memory transfers [62], and Kumaresan et al. integrated runtime selection among CuPy, PyTorch-CUDA and NumPy-CPU execution paths with gate fusion and adaptive precision [66].

Neither strand, however, employs a unified set of frameworks that spans single-thread CPU execution, full-node, multi-node and multi-GPU operation on a single HPC system while relating the measured differences to the hardware. This dissertation addresses that gap through coordinated benchmarks on Deucalion, interpreted with a parallel-efficiency metric and supplemented by roofline analysis and top-down profiling on the i9-13900KS platform (Section 4.3).

## 3.7 Summary

This chapter reviewed classical circuit simulation and the costs that limit it. The choice of method depends on the circuit structure and the required output, since dense state-vector simulation accommodates arbitrary circuits, whereas path-summation, tensor-network and stabiliser methods reduce the cost only when the circuit has the structure they assume. Noise then adds another trade-off between density-matrix storage, which grows as $4^n$, and trajectory sampling, which repeats pure-state runs to reduce the statistical uncertainty. For dense state vectors, memory capacity limits the register size, while memory bandwidth usually limits the update speed. Accordingly, vectorisation, fusion, memory placement and distributed execution each address a different part of these costs, and the extent to which a simulator applies them helps explain why implementations of the same method perform differently. The literature combines benchmarking and architectural analysis to varying degrees; however, it still lacks coordinated comparisons across execution regimes. Chapter 4 presents the frameworks, circuits, platforms and methodology used to address that gap.

# 4

# Benchmarking Framework and Methodology

## Contents


4.1 Performance Analysis Framework . . . . . 36
4.2 Benchmark Circuits . . . . . 38
4.3 Experimental Setup . . . . . 41
4.4 Measurement and Analysis . . . . . 47
4.5 Summary . . . . . 51

This chapter describes how the six state-vector backends are compared in single-thread, full-node, multi-node, single-GPU and multi-GPU runs, and how the noisy and non-state-vector simulations of Chapter 5 are set up. Performance models, hardware counters and estimates of GPU activity are then used to examine how the circuit, the implementation of the backend and the resources of the run contribute to the differences in runtime.

Section 4.1 states the performance models, and Section 4.2 defines the benchmark circuits. Section 4.3 then describes the platforms, the simulators and how their runs spend their time. On this basis, Section 4.4 defines what each timed call contains, the measures that tie the times to the hardware and the accuracy metrics.

## 4.1 Performance Analysis Framework

To interpret backend performance in relation to the available parallelism and memory hierarchy, four models are used. The three scaling models describe how execution time changes with the resources allocated, using speedup $S(p) = T(1)/T(p)$ and parallel efficiency $\eta(p) = S(p)/p$, where $T(p)$ is the execution time on $p$ threads, nodes or GPUs, depending on the experiment. Completing this analysis, the Cache-aware Roofline Model (CARM) relates attained floating-point performance to arithmetic intensity and the hardware's compute and bandwidth limits.

To locate where added resources stop shortening a run, each scaling series is also read step by step. When the resources double from one point of the series to the next, the incremental speedup $r(p) = T(p)/T(2p) = S(2p)/S(p)$ isolates each step, with a value of two when the time halves and below one when the run becomes slower.

**Amdahl's law.** For a fixed problem whose serial fraction is $f$, Amdahl's law bounds the speedup on $p$ processing units [67],

$$S(p) \;\le\; \frac{1}{f + (1-f)/p} \;<\; \frac{1}{f}. \tag{4.1}$$

In this dissertation, the strong-scaling experiments keep the circuit and the number of qubits fixed while increasing the number of threads, nodes or GPUs. In a state-vector kernel, however, almost every operation updates all amplitudes in parallel, so once the passes over the state outweigh the fixed costs of a run (Section 4.3.2), the serial fraction is close to zero. The speedup is then limited instead by a shared resource such as memory bandwidth or the network (Section 3.4). The law is therefore used as an upper bound on the speedup, while the Karp–Flatt metric described below gives the serial fraction implied by each measured speedup.

**Gustafson's law.** When the problem grows with the resources, so that the parallel work per processing unit stays fixed, Gustafson's law gives the scaled speedup [68]

$$S_G(p) \;=\; p - f\,(p-1). \tag{4.2}$$

The weak-scaling series of Sections 5.3 and 5.4 follow this regime, since they keep the state held by each node or GPU fixed while the number of qubits grows with the resources. A series that starts from $n_0$ qubits on one node or GPU therefore runs $n_p = n_0 + \log_2 p$ qubits on $p$ of them, as in the CPU series of Section 5.3, which grows from $n_0 = 28$ on one node to $n_{64} = 34$ on $64$ nodes. Since the gate count $G(n)$ of the circuit grows with $n$ (Section 4.2), the work per node also grows along the series, so the weak-scaling efficiency $\eta_{\mathsf{W}}(p) = T(1)/T(p)$ falls below one even without any cost of distribution, and its ideal value is

$$\eta_{\mathsf{W}}^{\mathsf{ideal}}(p) \;=\; \frac{G(n_0)}{G(n_p)} \;<\; 1, \tag{4.3}$$

which assumes a constant time per gate at a fixed local state size, so that the run on $p$ units takes longer than the run on one unit only because the circuit has more gates. This value is the reference against which the measured efficiencies are read.

**Karp–Flatt metric.** Whereas Amdahl's law bounds the speedup for a known serial fraction, the Karp–Flatt metric solves the same law in reverse, giving the serial fraction from the measured speedup [69],

$$e(p) \;=\; \frac{1/S(p) - 1/p}{1 - 1/p}. \tag{4.4}$$

A roughly constant $e(p)$ is consistent with a fixed serial fraction. If $e(p)$ rises as more resources are added, it suggests increasing overhead or a shared bottleneck, whereas a falling $e(p)$ means that the losses grow more slowly than a fixed serial fraction would produce, even while the efficiency $S(p)/p$ decreases, and by itself it does not reveal whether the time spent on exchanges rises or falls. Synchronisation, communication and limited memory bandwidth can all produce a rise, so the metric alone cannot distinguish between them. Sections 5.2 to 5.4 therefore interpret $e(p)$ alongside the hardware resources added at each scaling step (Section 4.3.2).

**Cache-aware roofline model.** In the Cache-aware Roofline Model (CARM), the floating-point performance of a kernel is bounded by [70, 71]

$$P \;\le\; \min\!\big\{P_{\mathsf{max}},\; B \cdot I\big\}. \tag{4.5}$$

In this bound, $P$ is the attained performance in floating-point operations per second, and $P_{\text{max}}$ is the peak compute throughput. The bandwidth $B$ is that of the memory level that serves the data, while the arithmetic intensity $I$ gives the number of floating-point operations per byte transferred between the core and the memory hierarchy.

As Figure 4.1 shows, each memory level from L1 to Dynamic Random-Access Memory (DRAM) contributes a sloped bandwidth roof, while peak compute throughput forms a horizontal roof. Their intersections, called ridge points, mark the transition between bandwidth and compute limits. The L1 and DRAM ridge points delimit three regions [72]. To the left of the L1 ridge point lies the memory-bound region, where bandwidth is the tighter bound even when the L1 cache serves the data. To the right of the DRAM ridge point lies the compute-bound region, where compute throughput is the tighter bound even at DRAM bandwidth. Between them lies the mixed region, where the applicable bound depends on which memory level serves the data. The region of a kernel suggests whether its memory accesses or its computation should be optimised first, although its position alone does not establish the actual bottleneck.

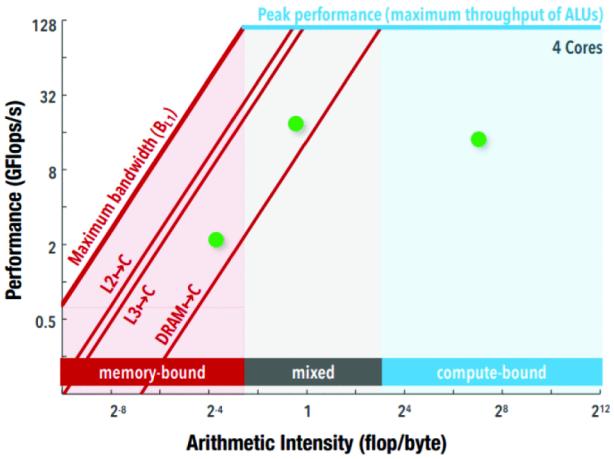



**Figure 4.1:** Cache-aware roofline model of a four-core processor, with example kernels shown in green. Sloped roofs represent the bandwidth limits from L1 cache to DRAM, and the horizontal roof represents peak floating-point throughput. The lower band identifies the memory-bound, mixed and compute-bound regions. Reproduced from Morgado et al. [72].

Changes to the code can move the operating point rightwards by increasing the operations performed per byte, or upwards by increasing the operations performed per second. Section 5.5 interprets these movements alongside runtime measurements to separate the effects of vectorisation and gate fusion in Qulacs.

## 4.2 Benchmark Circuits

Three circuit families are used throughout Chapter 5, each with a different gate structure. The QFT couples every pair of qubits through controlled phases and is a standard workload in benchmark suites for quantum simulators [73]. The HEA, by contrast, alternates single-qubit rotations with entangling gates between

neighbouring qubits, following the structure of variational ansätze[1] [74]. The Clifford circuit admits efficient stabiliser simulation, which makes it the reference for the stabiliser methods of Section 5.7. Table 4.1 summarises the three circuits at a common width of $n = 30$.

**Table 4.1:** Benchmark circuits at $n = 30$ qubits, where $d$ is the number of layers of the Clifford circuit. Gate counts and depths refer to the decomposed circuits as executed, before backend-internal gate fusion. Each gate has unit depth, and gates on disjoint qubits may share a layer. For the QFT, values in parentheses include the final SWAP gates, each decomposed into three CNOTs.

| Name | Notation | Structure | Gates | Depth |
|---|---|---|---|---|
| Quantum Fourier Transform | `qft` | Controlled phases between all qubit pairs | 1830 (1875) | 206 (209) |
| Hardware-Efficient Ansatz (8 ring layers) | `hea` | Rotation layers between nearest-neighbour CNOT rings | 990 | 265 |
| Clifford ($d = 10$) | `clifford` | Clifford gates only, stabiliser reference | 590 | 30 |

### 4.2.1 The Quantum Fourier Transform

The QFT applies the normalised Discrete Fourier Transform (DFT) to the amplitudes of a quantum state. It is relevant for quantum computing, given that it is a subroutine of quantum phase estimation, Shor's factoring algorithm [1] and algorithms for the Abelian Hidden Subgroup Problem (HSP) [12]. For an $n$-qubit register with $N = 2^n$ computational-basis states, its action is

$$|j\rangle \xrightarrow{\text{QFT}} \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i jk/N} |k\rangle , \qquad 0 \le j < N. \tag{4.6}$$

The standard circuit implements this transformation with Hadamard gates and controlled phase rotations (Figure 4.2). Each controlled-$R_k$ gate applies $R_k = \text{diag}(1, e^{2\pi i/2^k})$ to its target when the control qubit is in $|1\rangle$, and the rotation angle decreases as the separation between control and target increases. Together, these gates produce the QFT output in reversed qubit order, so $\lfloor n/2 \rfloor$ final SWAP gates restore the natural ordering; the benchmark circuit omits them, since its output is symmetric under that reordering.

All benchmark runs have $|0\rangle^{\otimes n}$ as input, for which every phase factor in (4.6) equals one, and the output is the uniform superposition

$$\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle = |+\rangle^{\otimes n} . \tag{4.7}$$

The output is therefore a product state, with probability $2^{-n}$ for each computational-basis outcome.

[1] An ansatz is a parameterised circuit whose rotation angles are tuned by a classical optimiser. Variational algorithms such as the Variational Quantum Eigensolver (VQE) run the ansatz, measure a cost function on its output state, such as the energy of a molecular Hamiltonian, and update the angles until the cost stops decreasing [74].

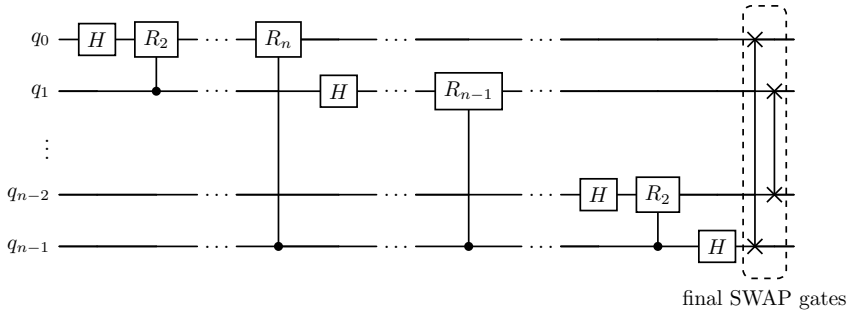



**Figure 4.2:** Standard $n$-qubit QFT circuit, composed of Hadamard gates and controlled-$R_k$ rotations with angles $2\pi/2^k$. The final SWAP gates (dashed box) restore the natural output ordering. Adapted from Nielsen and Chuang [12].

### 4.2.2 The Hardware-Efficient Ansatz

The HEA follows the structure of the parameterised circuits used in the VQE [74], alternating layers of single-qubit rotations with layers of CNOT gates (Figure 4.3). In the benchmark, eight CNOT layers each link the qubits in a closed ring, and between successive rings every qubit receives the rotations $R_Z R_X R_Z$ about the $z$ and $x$ axes ($R_X(\theta) = e^{-i\theta X/2}$, $R_Z(\theta) = e^{-i\theta Z/2}$). Each qubit also has two rotations before the first ring and two after the last, shown as the $R$ gates at both ends of Figure 4.3.

The circuit thus has $33n$ gates, $8n$ of them CNOTs and $25n$ of them rotations, so it tests how efficiently a backend executes repeated single-qubit rotations interleaved with CNOT gates. The rotation angles are drawn once from a generator with a fixed seed, so the benchmark applies the same circuit in every repetition. Together with the CNOT rings, these rotations entangle the register, so the HEA provides the entangled workload that complements the QFT from the all-zero input. Its nearest-neighbour ring also complements the all-to-all couplings of the QFT, since the placement of these gates can change how much data the GPUs must exchange when the state is distributed.

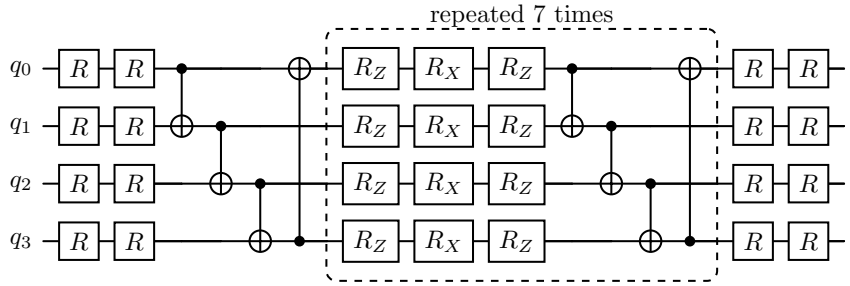



**Figure 4.3:** Structure of the HEA benchmark on four qubits. Each CNOT ring links every pair of neighbouring qubits and closes from the last qubit to the first, and between successive rings every qubit receives $R_Z R_X R_Z$. The dashed block repeats seven times, giving eight rings in total, and $R$ denotes the two rotations applied to each qubit before the first ring and after the last.

### 4.2.3 The Clifford Circuit

The Clifford benchmark uses only Clifford gates, which the Hadamard, phase and CNOT gates generate, so the Gottesman–Knill theorem makes it simulable in polynomial time (Section 2.3.3). The circuit is therefore the reference for the stabiliser methods, which Section 5.7 compares with the state-vector backends on the same circuit.

The circuit repeats the same layer structure $d = 10$ times, and Figure 4.4 shows one layer on four qubits. Each layer starts with one gate $G_{q,\ell}$ per qubit, chosen at random from $\{H, S, S^\dagger, X, Y, Z\}$, and since this choice is made once, when the circuit is built, every repetition applies the same circuit. Two CNOT stages then couple neighbouring qubits, the first acting on $(q_0, q_1)$ and $(q_2, q_3)$ and the second on $(q_1, q_2)$. For $n$ qubits, this pattern gives $n - 1$ CNOTs per layer, one for each neighbouring pair, so with the $n$ single-qubit gates, each layer has $2n - 1$ gates, and the $d$ layers have $d(2n - 1)$ gates with a depth of $3d$ for $n \geq 3$.

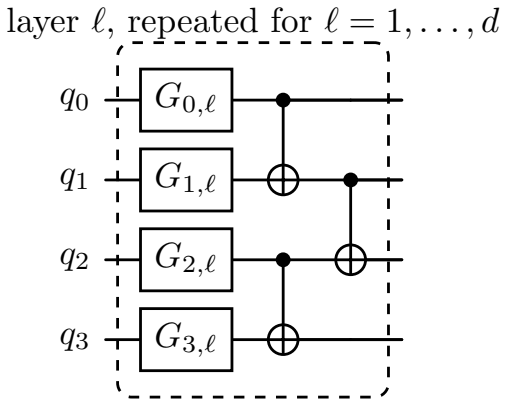



**Figure 4.4:** One layer $\ell$ of the Clifford benchmark on four qubits, which the benchmark repeats for $d = 10$ layers. Each $G_{q,\ell}$ is a gate chosen at random from $\{H, S, S^\dagger, X, Y, Z\}$, and the two CNOT stages couple the pairs $(q_0, q_1)$ and $(q_2, q_3)$ and then $(q_1, q_2)$.

## 4.3 Experimental Setup

The experiments are run on Deucalion, the EuroHPC supercomputer hosted in Portugal [75], and on a computing platform with an Intel Core i9-13900KS processor (Table 4.2). On Deucalion, the CPU experiments use the x86 partition, from one to $128$ threads on the two sockets of a node and on up to $64$ nodes in the distributed runs. Its nodes run with Simultaneous Multithreading (SMT) disabled, so the $128$ threads of a full node match its $128$ cores. The GPU experiments, in turn, use the A100-40 and A100-80 partitions, whose GPUs have $40$ and $80$ GB of memory. The CUDA-Q tensor-network runs and the single-GPU comparison of Section 5.1 use the A100-40 GPUs, whereas the multi-GPU experiments of Section 5.4 use the A100-80 GPUs.

The i9-13900KS platform hosts three parts of the results, the roofline ablation of Section 5.5, the top-down

analysis of Appendix A.3 and the density-matrix study of Section 5.6. The first two need hardware counters that this platform exposes. Since this platform differs from Deucalion in processor and memory, the times measured on it are compared only with each other.

**Table 4.2:** Platforms used for the results of Chapter 5, namely the x86 and A100 partitions of Deucalion and the i9-13900KS platform. Nominal bandwidths are the vendors' figures, except for the i9-13900KS platform, where the value follows from its two DDR4-3200 memory channels. InfiniBand rates are those of the system documentation [75]. Simulator versions and backend modes are listed in Table 4.3.

| | **Deucalion x86 partition** | **Deucalion A100 partitions** | **i9-13900KS platform** |
|---|---|---|---|
| Role | CPU runs | GPU runs | CPU runs |
| Nodes | 500 | 17 (A100-40), 16 (A100-80) | 1 |
| CPU | 2× AMD EPYC 7742 | 2× AMD EPYC 7742 | Intel Core i9-13900KS |
| L3 cache | 16 MiB per four-core complex, 256 MiB per socket, 512 MiB per node | 16 MiB per four-core complex, 512 MiB per node | 36 MiB |
| Cores / node | 128 (2 sockets × 64, SMT off), 8 NUMA domains of 16 cores | 128 | 8 performance and 16 efficiency cores (32 threads) |
| Vector ISA | AVX2 | AVX2 (host) | AVX2, no AVX-512 |
| GPUs / node | n/a | 4× NVIDIA A100-SXM4 | Integrated only, not used |
| GPU memory | n/a | 40 GB HBM2 (A100-40) or 80 GB HBM2e (A100-80) | n/a |
| Host memory / node | 251.7 GiB DDR4, eight channels per socket; 242 GiB per job | 504 GiB | 128 GiB DDR4-3200 (4× 32 GiB), 125 GiB usable |
| Nominal bandwidth | 409.6 GB/s per node [76] | 1.55 TB/s (A100-40) or 2.04 TB/s (A100-80) per GPU [55] | 51.2 GB/s (two DDR4-3200 channels) |
| Intra-node link | n/a | NVLink | n/a |
| Interconnect | InfiniBand, 200 Gb/s | InfiniBand, 2×200 Gb/s | n/a |
| Nodes per job | up to 64 | up to 4 (16 GPUs) | 1 |

### 4.3.1 Simulators and Builds

The benchmarks cover five quantum simulators, each built around a different approach to circuit execution. They are Qiskit Aer, CUDA-Q, Qulacs, Qibo and QuEST, and each is described below through the properties on which the results of Chapter 5 depend. The distributed-memory build of Qulacs, mpiQulacs, is evaluated as a separate backend, so the five simulators provide the six backends, and Table 4.6, at the end of this chapter, places them within the wider set reviewed in Chapter 3.

**Qiskit Aer.** Qiskit Aer is the simulator of the Qiskit software stack, implemented in C++ with a Python interface [51]. The experiments of Chapter 5 use two of its state-vector engines, which both run with gate fusion enabled but divide the work differently. In single-process CPU runs, the shared-memory engine can distribute the update of each gate among OpenMP threads, whereas the distributed engine divides the state vector itself into chunks held by different MPI processes, or ranks.

On CPUs, however, version 0.17.2 of the distributed engine applies the gates of each rank with a single thread [51]. The multi-node CPU runs of Qiskit Aer therefore launch 128 ranks per node, so that the gate

updates are distributed across the 128 cores of a node (Section 5.3). The source code of the engine shows this behaviour [51], and Appendix A.4 compares the layout with one rank per node.
The multi-GPU runs use the same engine with one rank per GPU. Each rank updates its chunks with the CUDA kernels of Qiskit Aer, since `cuStateVec` is not enabled, and exchanges state-vector data with the other ranks through MPI.

**CUDA-Q.** CUDA-Q is NVIDIA's framework for hybrid quantum-classical programs, implemented in C++ with a Python interface [77], and the benchmarks of this dissertation call it through that interface. The backend that executes a circuit is chosen by naming a target, a predefined configuration that names one simulator or one quantum processor [43]. For simulation, the target fixes the simulation method and the hardware that runs it, and its options, such as the numerical precision, refine that configuration.
The state-vector runs use the `nvidia` target on the GPU and the `qpp-cpu` target on the CPU. The `nvidia` target keeps the $2^n$ amplitudes in the global memory of the GPU and updates them through `cuStateVec` [43], the state-vector library of the NVIDIA `cuQuantum` Software Development Kit (SDK) [29]. With the `mgpu` option, it divides the state among several GPUs, with one MPI rank per GPU, as in the multi-GPU runs of Qiskit Aer. The `qpp-cpu` target instead stores the state in host memory and executes the same circuits on the CPU for the comparison of Section 5.1.
Two further targets support the comparison of simulation methods in Section 5.7. The `tensornet` target contracts the circuit's tensor network on the GPU through `cuTensorNet`, another library of `cuQuantum`. The `stim` target runs Clifford circuits on the CPU through the Stim stabiliser simulator [35].

**Qulacs.** Qulacs is designed for fast simulation on a single node, with a core in C and C++ behind a Python interface [78]. On the CPU, its gate kernels run on OpenMP threads and, in builds with SIMD support, use hand-written[2] AVX2 intrinsics, while on the GPU they are CUDA kernels. Gate fusion is a separate step, applied only when the circuit optimiser of Qulacs is called, which merges neighbouring gates into blocks of a chosen size. Because vectorisation and fusion are thus set independently, the roofline ablation of Section 5.5 varies each of them, whereas the other single-node runs of Qulacs apply the circuit without fusion. To control vectorisation in that ablation, the scalar configurations use a build without the AVX2 kernels and with the auto-vectorisation of the compiler switched off, so that their kernels execute scalar floating-point instructions only, while the AVX2 configurations use the hand-written kernels.
The MPI build of Qulacs is treated in this dissertation as a separate backend, mpiQulacs, the name under which its distributed execution was first developed [60]. It has been an optional build of the main distribution since version 0.6.0 [59]. Beyond state vectors, Qulacs also provides a density-matrix simulator, which the noise experiments of Section 5.6 use.

[2] Hand-written kernels call intrinsics, functions that the compiler translates into single processor instructions, such as the AVX2 vector instructions of Section 3.5.1. The programmer thus chooses each vector instruction, whereas in auto-vectorised code the compiler derives these instructions from ordinary loops.

**QIBO.** QIBO is a Python framework that provides a common interface between the user and interchangeable simulation backends, including GPU-accelerated ones [79], serving a similar role to the targets of CUDA-Q. The benchmarks use four of these backends, two for state-vector simulation, the default NumPy simulator and `qibojit`, and two for other methods, the Clifford backend and the tensor-network backend QIBOTN (Section 3.1.3).
Of the two state-vector backends, `qibojit` compiles its gate kernels just in time through `Numba` on CPUs and `CuPy` on GPUs [80]. Its CPU kernels are multithreaded, with vectorisation left to the compiler (Section 3.5.1), while its GPU kernels use CUDA. For execution on several GPUs, QIBO keeps the full state in host memory and processes parts of it on the GPUs of one machine [81]. Across nodes, it distributes tensor-network contractions through QIBOTN.

**QUEST.** QUEST, the Quantum Exact Simulation Toolkit, is a state-vector simulator designed for portability, with the same interface on CPUs, GPUs and distributed systems (Section 3.5.5) [82]. The benchmarks of this dissertation call version 4.2.0 [83] through its C interface for every run, with its CPU build within a single node and across several nodes and its CUDA build on one A100-40 and on up to sixteen A100-80 GPUs, where it divides the state with one MPI rank per GPU. The CUDA build runs the library's own kernels rather than `cuStateVec`, so that it does not measure the kernels of CUDA-Q a second time.
Within each process, OpenMP threads can share the work of applying each gate, while the library's native MPI support divides the state vector among processes for distributed execution. In its single-node build, QUEST can also allocate the state vector across the NUMA domains of the host, as in Figure 3.8, provided that it is compiled with the `libnuma` library [61].

**Versions and configurations.** Table 4.3 lists the simulator versions and backend modes used in each experiment. Several simulators run with more than one backend, since each experiment selects the one suited to its platform and simulation method. The state-vector comparisons use double-precision (64-bit) floating-point format, in which each complex amplitude occupies $16$ bytes, selected through the `fp64` option of the `nvidia` target of CUDA-Q and through the corresponding settings or builds of the other simulators. One format for all backends keeps the storage per amplitude the same across them, since the single-precision (32-bit) format would halve both that storage and the bytes moved in each pass over the state. Double precision also keeps the outputs accurate at the sizes benchmarked in this dissertation, because the rounding error of single precision grows with the gate count and the state size and lowers the fidelity measurably beyond about $20$ qubits and $50$ gates [66].
The simulators built from source for this dissertation, QULACS, QUEST and the MPI builds of QISKIT AER, are compiled at the optimisation level `-O3`, which is the release configuration of their build systems. The other backends run from existing builds, the cluster modules of MPIQULACS and QULACS 0.6.11 on Deucalion

and the prebuilt packages of CUDA-Q and the single-node Qiskit Aer, while Qibo compiles its `qibojit` kernels at run time.

**Table 4.3:** Versions and configurations of the simulators evaluated in this dissertation. The version column gives the version behind most results, and the notes below the table list the series that used another version.

| Simulator | Version | Configurations evaluated |
|---|---|---|
| Qiskit Aer | 0.17.2 | Shared-memory engine for single-node CPU and GPU runs; distributed cache-blocking engine with chunks of at most $2^{26}$ amplitudes, built with MPI (OpenMP backend on CPUs, CUDA backend on GPUs), for multi-node and multi-GPU runs. |
| CUDA-Q | 0.14.0 | `nvidia` target for single-GPU runs and its `mgpu` option for multi-GPU runs; `qpp-cpu` fallback as a CPU reference point; `stim` and `tensornet` targets for Section 5.7. |
| Qulacs | 0.6.13 | CPU state-vector backend, source build with AVX2, for the thread-scaling and most landscape runs; builds with and without SIMD kernels for the roofline ablation on the i9-13900KS platform. |
| mpiQulacs | 0.6.11 | MPI build of Qulacs with the FusedSWAP optimiser, for multi-node CPU scaling. |
| Qibo | 0.3.1 | Default NumPy backend; `qibojit` with Numba on CPUs and CuPy on GPUs; Clifford backend and cuTensorNet for Section 5.7. |
| QuEST | 4.2.0 | OpenMP and MPI build for the CPU runs and CUDA and MPI build for the GPU runs, with the library's own kernels and without `cuQuantum`, both version 4.2.0, in double-precision (64-bit) floating-point format. |

*Shared stack.* Deucalion runs Linux 4.18 (Enterprise Linux 8) with Python 3.10 to 3.12 across the series. GPU runs use the CUDA 12.1 toolkit, with the CUDA 12.9 runtime bundled in the Python packages, and the cuQuantum libraries that CUDA-Q 0.14.0 requires. Distributed runs use Open MPI 4.1.5 over Unified Communication X (UCX) 1.14.1, launched with `srun` on the x86 partition and with `mpirun` and one rank per GPU on the A100 partitions. The i9-13900KS platform runs Ubuntu 22.04 with Linux 5.15 and Python 3.13.

Qiskit Aer 0.15.1 ran the thread sweep of Section 5.2 and the 30-qubit QFT points that Figure 5.1 shares with it, and Qulacs 0.6.11, the cluster module, ran the HEA points and the GPU row of Section 5.1.

### 4.3.2 Execution Model

A run spends part of its time on fixed costs, such as preparing the circuit for execution and dispatching each gate to its kernel, which grow with the number of gates but not with the size of the state. The rest goes into updating the state, where a gate kernel typically reads and writes every amplitude once, a traversal that this dissertation calls a pass. Since the passes outweigh the fixed costs only above a size that differs between backends (Figure A.1), the backends are compared mainly at the largest sizes measured.

The fixed costs differ between backends. Qulacs passes each gate from Python straight to its native kernels, whereas Qiskit Aer takes each circuit through its Python and C++ layers before the first gate runs. CUDA-Q compiles the circuit, which is written as a Python function, just in time [84], so this compilation adds to every timed call, whereas `qibojit` compiles its kernels, or loads them from a cache, at their first call, which falls in the warm-up [80].

Gate fusion reduces the number of passes by merging several gates into one update (Section 3.5.2 and Figure 3.7). Qiskit Aer and CUDA-Q fuse neighbouring gates by default, which shortens a run only when the passes removed outweigh the higher arithmetic cost of each fused block. In contrast, QuEST, `qibojit` and Qulacs with fusion disabled apply one gate at a time, so their number of passes is taken as the gate

count of Table 4.1, with one pass per gate as the reference, although the traffic of each pass depends on the gate and its kernel.

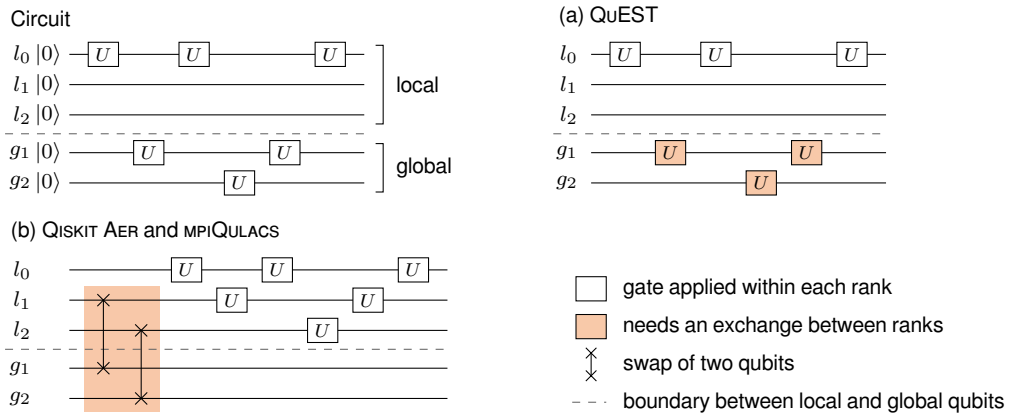



**Figure 4.5:** Schematic of how the distributed CPU backends handle gates on global qubits. The circuit applies gates $U$ to the local qubit $l_0$ and to the global qubits $g_1$ and $g_2$, while the local qubits $l_1$ and $l_2$ stay idle, and all qubits start in $|0\rangle$. (a) QuEST exchanges amplitudes between pairs of ranks before every non-diagonal gate on a global qubit [82]. (b) Qiskit Aer and mpiQulacs first swap $g_1$ and $g_2$ with the idle local qubits in one exchange, after which every gate acts on a local qubit. Swaps that restore the original qubit order at the end are omitted.

Distribution adds the cost of exchanging amplitudes between ranks or GPUs. As Section 3.5.5 explains, dividing the state among $2^k$ of them makes $k$ of the qubits global (Figure 3.9), and a non-diagonal gate on a global qubit needs amplitudes held elsewhere, unless the backend first swaps that qubit with a local one [10]. In the scaling experiments, each time the ranks or GPUs are doubled, one more qubit therefore becomes global, and the gates on that qubit then need exchanges.

The CPU backends arrange these exchanges differently (Figure 4.5). Before each non-diagonal gate on a global qubit, the ranks of QuEST exchange data in pairs, each rank with the one that holds the amplitudes the gate combines with its own [82], so its exchanges grow with the number of such gates (Figure 4.5a). Qiskit Aer and mpiQulacs instead apply the qubit reordering of Section 3.5.5, swapping global qubits with local ones so that the gates that follow need no communication (Figure 4.5b). The distributed engine of Qiskit Aer groups these gates into blocks that act only on the qubits within its chunks, a scheme called cache blocking [57], and applies several swaps in one all-to-all exchange [51]. mpiQulacs likewise combines the swaps of several global and local qubits into one FusedSWAP operation [60].

How long these exchanges take depends on the link they cross, since a run that spans several nodes sends part of its data over InfiniBand, which is slower than the links within a node, such as NVLink between GPUs. The three backends benchmarked on several GPUs reach these links differently. The `mgpu` option of CUDA-Q exchanges data directly between devices, using peer-to-peer copies within a node and CUDA-aware MPI between nodes [43] (Figure 4.6). Qiskit Aer, in turn, runs its cache-blocking engine [57] with one rank per GPU and exchanges chunks through MPI [51]; within a node, this transfer passes directly between GPUs only when the CUDA Inter-Process Communication (IPC) transport of UCX is enabled (Appendix A.6).

QuEST exchanges amplitudes in pairs before each gate on a global qubit, following the same approach as on CPUs (Figure 4.5a) [82], and uses CUDA-aware MPI for these transfers, sending from the GPU memory of one rank into an exchange buffer in the GPU memory of its partner [61]. That buffer is as large as the rank's share of the state, so in distributed runs each GPU holds at most $31$ qubits of the state and, from $n = 33$, the same size needs twice as many GPUs as for the other two backends.

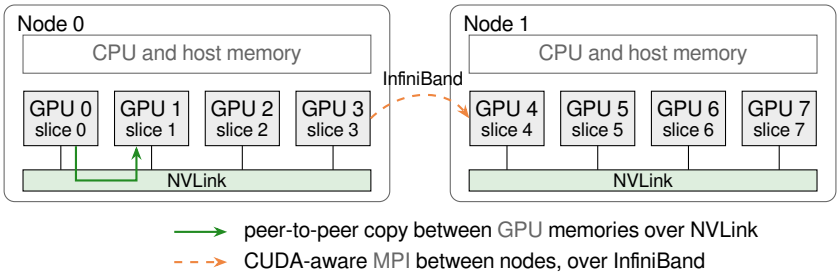



**Figure 4.6:** Data paths of the `mgpu` option of CUDA-Q on two A100 nodes. The state vector is divided into eight slices, one per GPU. Within a node, slices are exchanged by peer-to-peer copies between GPU memories over NVLink, without passing through host memory. Between nodes, the exchanges go through CUDA-aware MPI, which accepts GPU buffers directly, over the InfiniBand network [43]. One exchange of each kind is shown.

## 4.4 Measurement and Analysis

The setup of Section 4.3 fixes where each backend runs, but two measured times are comparable only when it is known how their runs differ and what work each timer covers. These conditions are defined for every experiment of Chapter 5, together with the quantities that relate the measured times and hardware counters to the platforms.

### 4.4.1 Comparisons and Timing

For each circuit and qubit count, the benchmark harness builds the circuit of Section 4.2 using the operations supported by each library. It uses the Python interfaces of Qiskit Aer, QuLacs and Qibo and the kernel builder of CUDA-Q. For QuEST, a C program reads the same sequence from an OpenQASM file and applies it through the library's C interface. The harness applies no circuit optimisation of its own, and Qiskit Aer transpiles at optimisation level zero. Explicit optimisation is used only in the roofline ablation of QuLacs (Section 4.3.1) and in the distributed runs of mpiQuLacs (Section 5.3), which use its optimiser with the same block size and fused swaps. The backends therefore receive the same sequence of operations and differ in how they prepare and execute it (Section 4.3.2).

Each repetition records two times. The *simulation time* covers the execution call that applies the circuit to the state, whereas the *total time* also includes the construction and transpilation of the circuit. The simulation time places the measurement as close to gate execution as each interface permits, although allocation,

compilation or a copy of the state may remain inside the call. In the single-GPU runs of Section 5.1, the calls of Qiskit Aer and CUDA-Q also copy the state to host memory, whereas the multi-GPU runs of Section 5.4 leave it on the GPUs. The sampled outputs of Section 5.7 draw $1024$ shots inside the timed call. Chapter 5 reports the simulation time, except in the roofline experiment, whose hardware counters cover the whole process (Section 4.4.2). The comparisons of that chapter therefore concern the execution calls that each backend exposes, read together with the scaling metrics and hardware measurements.

One untimed warm-up precedes the timed repetitions, at the same size or at a smaller one that already compiles the kernels and opens the MPI communication. Each timed repetition then applies the circuit to a freshly initialised zero state and, since every state-vector experiment is a strong simulation (Section 2.2.5), each repetition ends with the full final state in memory, on one device or divided among the ranks or GPUs. Where the state fits in host memory, the harness computes its fidelity and $L_2$ population error against the ideal output after the clock stops (Section 4.4.3). Each configuration runs in its own process, which records one row per repetition with the times, the resident memory, the platform and the versions of the library, of Python and of MPI. In the multi-GPU runs, `nvidia-smi` also samples the utilisation of every GPU every $0.5$ s (Appendix A.5).

The single-node state-vector, thread-sweep and stabiliser series use the fastest repetition, representing the best performance observed for each configuration, as in the benchmarks of Leonteva et al. [34] and Rezaei et al. [49]. For the multi-node CPU experiments, this minimum is taken over all repetitions across all the jobs that measured a point, with the spread between jobs given in Section 5.3. The multi-GPU experiments instead use the median, which describes a typical repetition when the cost of communication varies between repetitions [85]. The tensor-network experiments use it for a similar reason, since every repetition includes a new search for a contraction path, which can return a different contraction order from one repetition to the next. The density-matrix experiments also report the median, because at small sizes their times are set by fixed costs whose variation between repetitions is comparable to the differences between backends (Section 5.6). The roofline experiment reports means, since its hardware counters accumulate over the whole process of each repetition and are averaged together with the runtimes. Table 4.4 lists the layout, precision, repetitions and statistic of each series. Within one job, the slowest repetition of a point exceeds the fastest by less than $7\%$ in every group of Table A.1, so the fastest repetition, the median and the mean of each of those points differ by less than that.

### 4.4.2 Roofline Measurements

The Cache-aware Roofline Model (CARM) plot of Section 5.5 uses measurements of the $28$-qubit QFT collected on one performance core of the i9-13900KS platform (Section 4.3), given that this platform provides access to the required hardware counters. Four Qulacs configurations were tested, combining scalar or AVX2 code (Section 3.5.1) with gate fusion either disabled or enabled for blocks of up to two qubits (`bs2`).

**Table 4.4:** Layout, precision and repetitions of the experiments of Chapter 5, all in double precision (fp64), with the statistic that each reported point takes over its repetitions.

| Section | Layout | Precision | Repetitions and statistic |
|---|---|---|---|
| 5.1 | One x86 thread; one x86 node with 128 threads; one A100-40 | fp64 | Ten (one thread), five (128 threads) or six (A100-40; ten for QuEST) after one warm-up; fastest repetition |
| 5.2 | One x86 node, 1 to 128 threads, each count in a fresh process, threads placed by the OpenMP runtime | fp64 | Ten after one warm-up; fastest repetition |
| 5.3 | 1 to 64 x86 nodes; one rank of 128 threads per node, or 128 ranks of one thread for Qiskit Aer, threads pinned to cores | fp64 | Ten per job after one warm-up; fastest of all repetitions across all jobs |
| 5.4.1 | One, two and four A100-80 GPUs of one node, one rank per GPU | fp64 | Ten after one warm-up; median |
| 5.4.2 | 1 to 16 A100-80 GPUs on up to four nodes, one rank per GPU | fp64 | Ten after one warm-up; median |
| 5.5 | One performance core of the i9-13900KS platform | fp64 | Ten after one warm-up; mean |
| 5.6 | i9-13900KS platform, main thread pinned to one performance core | fp64 | Five after one warm-up; median |
| 5.7 | Stabiliser runs on the host processor of A100-40 nodes and on one x86 thread; tensor-network runs on A100-40 GPUs | fp64 | Stabiliser: five after one warm-up, fastest repetition. Tensor network: five after one warm-up, median of the completed repetitions, with a time limit per repetition of 3600 s (CUDA-Q) and 7200 s (Qibo) |

Each operating point is the mean of ten repetitions of its configuration, run one after another, so their spread leaves out the variation between sessions, which reaches about 6% in the runtime of the unfused builds. The CARM Tool [72] measured each configuration in two ways, through the Performance Monitoring Unit (PMU) of the core and through Dynamic Binary Instrumentation (DBI) with DynamoRIO. The operating points of Figure 5.8 come from the PMU, because it counts the floating-point operations and memory accesses during native execution, in the same repetitions that give the wall-clock times. DBI, in contrast, instruments the program and increases its run time [72], so it runs separately and is used to check the PMU counts. The two floating-point totals lie within 0.5% of each other, a nearly constant difference that is consistent with floating-point operations outside the instrumented region (Table 4.5).

**Table 4.5:** Double-precision (64-bit) floating-point operations, in units of $10^{12}$, for single-thread QuLACS runs of the 28-qubit QFT on the i9-13900KS platform. DBI totals cover the instrumented region; PMU totals cover the whole process and are means over ten repetitions. Relative differences are calculated from unrounded totals as $100\,|F_{\mathrm{DBI}} - F_{\mathrm{PMU}}|/F_{\mathrm{PMU}}$ and include differences in measurement scope.

| Configuration | DBI Region | PMU Whole process | Relative difference [%] |
|---|---|---|---|
| Scalar, no fusion | 1.247688 | 1.253236 | 0.443 |
| AVX2, no fusion | 1.248225 | 1.253773 | 0.442 |
| Scalar, `bs2` | 3.044058 | 3.049606 | 0.182 |
| AVX2, `bs2` | 3.072512 | 3.078061 | 0.180 |

The memory accesses need one further caveat. In every build except the fused AVX2 one, the memory instructions that DBI records move as many bytes as the width at which their accesses are counted. In the fused AVX2 build, most memory instructions are 8-byte broadcasts counted at 32 bytes, so its byte count overstates the data it moves. Its operating point would therefore lie further to the right at the same performance, and the plotted intensity is a lower bound, although how far it lies below the true intensity

would need a direct measurement of the memory traffic. The operating points are therefore read for the region of Figure 4.1 in which each configuration falls, memory-bound, mixed or compute-bound, while the wall-clock times quantify the runtime differences.

### 4.4.3 Accuracy Metrics

For the noisy runs of Section 5.6, three metrics describe the simulated state. Because the noise channels of Section 2.4.3 turn the pure output of the circuit into a mixed state, the simulated state is a density matrix. Two of the three metrics defined below measure its accuracy against the ideal output. The fidelity compares the two full quantum states, including their relative phases, whereas the $L_2$ error compares only the probabilities of the measurement outcomes in the computational basis, which the Born rule gives from each state (Section 2.2.4). The third metric, purity, needs no reference, since it is a property of the simulated state alone and measures how mixed that state has become (Section 2.4.1).

**State fidelity.** Fidelity is a standard measure of the similarity between two quantum states. For two density matrices $\rho$ and $\sigma$, this dissertation uses the square of the fidelity of Nielsen and Chuang [12],

$$F(\rho, \sigma) = \left( \mathrm{Tr} \sqrt{\sqrt{\rho}\, \sigma\, \sqrt{\rho}} \right)^2 . \tag{4.8}$$

Its value lies in the interval $[0, 1]$, where $F = 1$ indicates perfect agreement between the two states. When the ideal state is pure, $|\psi\rangle$, its fidelity with the simulated state $\rho$ simplifies to

$$F\big(|\psi\rangle\langle\psi|, \rho\big) = \langle\psi|\rho|\psi\rangle, \tag{4.9}$$

which is the overlap between the ideal state and the simulated state.

**Population error.** The $L_2$ error compares the output probabilities $p_x = \langle x|\rho|x\rangle$ of the simulated state with the probabilities $q_x$ of the ideal one,

$$\varepsilon_{L_2} = \Big( \sum_x \big(p_x - q_x\big)^2 \Big)^{1/2}. \tag{4.10}$$

It is zero when the probabilities are exact, even if the phases of the state are not. The single-node benchmarks also record it for the noiseless runs, where it checks their output probabilities against the ideal ones, while the fidelity checks their amplitudes, with their relative phases, against the ideal final state of the circuit, which for the QFT from the all-zero input is the uniform superposition (Section 4.2.1). Both are computed after the clock stops wherever the final state fits in host memory, and where the state is divided among ranks or GPUs the timed call returns one probability instead.

**Purity.** The purity of the simulated state is $\mathrm{Tr}(\rho^2)$, which satisfies

$$\frac{1}{2^n} \leq \mathrm{Tr}(\rho^2) \leq 1, \tag{4.11}$$

with the upper bound reached by a pure state and the lower bound by the maximally mixed state. The depolarising channel moves every state towards the maximally mixed one and lowers its purity. Amplitude damping, in contrast, moves population to the ground state, which is pure, so two channels that lower the fidelity by the same amount can leave states of different purity.

## 4.5 Summary

This chapter established the performance models and experimental procedures used to interpret the results of Chapter 5. Amdahl's and Gustafson's laws provide the references for strong and weak scaling, the Karp–Flatt metric expresses a measured speedup as an effective serial fraction, and the CARM relates the floating-point performance of a kernel to the compute and bandwidth limits of the hardware. These models are applied to the QFT, the HEA and the Clifford circuit, which differ in gate structure, run by the six backends of five simulators on the x86 and A100 partitions of Deucalion. The roofline, density-matrix and top-down measurements come instead from the i9-13900KS platform and are not compared directly with the Deucalion times. The execution model separates the fixed costs of preparing and dispatching gates from the passes over the state, which take most of the runtime at the sizes where the backends are compared, so performance there depends on the number of passes, which gate fusion reduces, and on the cost of each pass, to which distributed execution adds the exchanges of amplitudes. Table 4.4 fixes the layout, precision and repetitions of each series. The roofline ablation varies vectorisation and gate fusion independently in Qulacs, with PMU counts from native execution and separate DBI runs that check the floating-point work, and the noisy runs are assessed through the fidelity and population error against the ideal output and through the purity of the simulated state.

**Table 4.6:** Features of the simulators considered in this dissertation. The five benchmarked simulators appear above the mid-rule; additional tools referenced in the broader landscape appear below it. The features characterise each simulator without ranking it, since many of the tools listed are integrated into wider development kits, with varying concerns and capabilities, such as integrating with quantum hardware, being optimised for specific physical models or providing other middleware features, like compilation. Simulation methods are abbreviated as SV = state-vector (Schrödinger), DM = density matrix, SF = Schrödinger–Feynman, TN = tensor network, MPS = matrix product state, Stab = stabiliser, DD = decision diagram.

| Simulator | Simulation method(s) | Language | CPU | GPU | Multithreading | Multiprocessing | Other acceleration methods or features | Built-in noise simulation | Python interface | OpenQASM support |
|---|---|---|---|---|---|---|---|---|---|---|
| Qiskit Aer | SV, DM, TN, Stab, MPS | C++ | ✓ | ✓[1] | ✓ | ✓[2] | Gate fusion, shot branching, cache blocking, GPU batching | ✓✓[3] | ✓ | ✓ |
| CUDA-Q | SV, TN, MPS, DM, Stab | C++ / Python (MLIR/LLVM) | ✓[4] | ✓✓ | ✓ | ✓[2] | Gate fusion, JIT (Quake MLIR), stream concurrency | ✓[5] | ✓ | ✗[6] |
| Qulacs | SV, DM | C/C++ | ✓ | ✓[7] | ✓ | ✓[8] | AVX2 vectorisation, gate fusion, circuit rewriting | ✓[9] | ✓ | ✗[10] |
| Qibo | SV, DM, TN, Stab | Python[11] | ✓[11] | ✓[1] | ✓[11] | ✗[12] | JIT (Numba), gate fusion, in-place updates | ✓ | ✓ | ✓ |
| QuEST | SV, DM | C/C++ | ✓ | ✓[13] | ✓ | ✓[2] | Auto-vectorisation (AVX2), runtime deployment selection | ✓ | ✓[14] | ✗[10] |
| PennyLane Lightning | SV, TN | C++ | ✓ | ✓[1] | ✓ | ✓[2] | AVX2/512 vectorisation, adjoint diff., Kokkos | ✗[15] | ✓ | ✓[16] |
| Cirq (qsim) | SV | C++ | ✓ | ✓[17] | ✓ | ✗ | Gate fusion, AVX/FMA, cache blocking | ✓[9] | ✓ | ✓ |
| Cirq (qsimh) | SV (SF) | C++ | ✓ | ✗ | ✓ | ✗ | Same as above | ✓[9] | ✓ | ✓ |
| Intel Quantum Simulator | SV | C++ | ✓ | ✗ | ✓ | ✓[2] | AVX-512 (compiler), gate fusion | ✓[9] | ✓ | ✓ |
| MQT DDSIM | DD | C++ | ✓ | ✗ | ✓ | ✗ | Decision-diagram compression | ✓ | ✓ | ✓ |
| quimb | TN | Python[11] | ✓[11] | ✓[11] | ✓[11] | ✓[18] | Contraction-path optimisation (cotengra) | ✗ | ✓ | ✓[19] |
| ProjectQ[20] | SV | C++ / Python | ✓ | ✗ | ✓ | ✗ | AVX intrinsics, OpenMP | ✗ | ✓ | ✗ |
| qFlex[20] | TN | C++ | ✓ | ✗ | ✓ | ✓[2] | Vectorisation | ✗ | ✓ | ✗ |

✓ = yes
✗ = no
✓✓ = primary focus

[1] Own GPU kernels or the NVIDIA `cuQuantum` SDK (cuStateVec, cuTensorNet), depending on the backend chosen.
[2] MPI-based distributed execution.
[3] DM + MC trajectories; device-calibrated models.
[4] CPU supported, but GPU is the primary target.
[5] Trajectory-based (GPU-batched); DM on CPU only.
[6] Native kernel syntax; OpenQASM 2 export, import via third-party tooling.
[7] Requires the `qulacs-gpu` package or a CUDA source build.
[8] MPI build in the main distribution since v0.6.0.
[9] Stochastic (via noise channels/trajectories).
[10] Via third-party tools.
[11] Using high-performance packages in the Python ecosystem.
[12] No MPI distributed state-vector path in mainline; MPI for tensor networks via QIBOTN only.
[13] CUDA and HIP (AMD), optional cuQuantum; multi-GPU through MPI since v4.0.
[14] Via `pyQuEST` (Cython) and `qoqo-quest` (Rust).
[15] Pure-state only; noise via PennyLane `default.mixed`.
[16] Via the PennyLane SDK.
[17] Native CUDA + cuStateVec; HIP/ROCm (AMD) since v0.18.0.
[18] Via `cotengra` contraction slicing and PETSc, not a native distributed state.
[19] OpenQASM 2 only.
[20] Dormant; ProjectQ last released v0.8.0 in October 2022, qFlex unmaintained.

# 5

# Results

## Contents


5.1 Single-Node Performance Landscape . . . . . 54
5.2 QFT Thread Scaling on One Node . . . . . 58
5.3 Multi-Node CPU Scaling . . . . . 60
5.4 GPU Acceleration . . . . . 64
5.5 Vectorisation and Gate Fusion . . . . . 69
5.6 Noise-Aware Density-Matrix Simulation . . . . . 70
5.7 Beyond State-Vector Simulation . . . . . 73
5.8 Simulator-Selection Framework . . . . . 75
5.9 Summary . . . . . 77

This chapter compares the performance of the simulation backends using the methodology of Section 4.4.1. It examines how circuit structure, hardware resources and implementation choices affect runtime and the size of the simulations that each backend can execute.

Sections 5.1 and 5.2 establish the single-node baselines, followed by Sections 5.3 and 5.4, which distribute the state vector over CPU nodes and GPUs. Section 5.5 isolates the effects of vectorisation and gate fusion on one core. The analysis then extends to noisy density-matrix simulation in Section 5.6 and to stabiliser and tensor-network methods in Section 5.7. Finally, Section 5.8 brings these findings together into a framework for choosing a simulation method and backend.

## 5.1 Single-Node Performance Landscape

The QFT, HEA and Clifford circuits of Section 4.2 run on one CPU thread, on the full dual-socket EPYC node with one thread per core ($128$ threads) and on one A100-40 GPU. The single-thread runs provide a reference for each backend's fixed costs (Section 4.3.2) and native kernel performance. The full-node and GPU runs then show the gains from shared-memory parallelism and GPU acceleration.

Figure 5.1 plots simulation time against qubit count, with one row per execution regime and one column per circuit. Each point represents the fastest repetition at that size. Most curves pass from a region dominated by fixed costs, where runtime grows slowly relative to the state size, to one dominated by state updates, where it roughly doubles with each added qubit (Section 4.3.2). The transition is taken as the first qubit count from which the average time per gate increases by at least a factor of $1.8$ for two consecutive additions of one qubit. This average is the simulation time divided by the circuit's gate count.

**Single-thread CPU regime.** In the single-thread measurements (Figure 5.1a–c), the average time per gate begins to double with each added qubit between $n = 11$ and $18$, depending on the backend and circuit. Below this transition, execution time is largely determined by fixed costs, which favour Qulacs and, on the QFT, QuEST. The timed calls of both backends cover gate execution, and that of Qulacs also allocates and initialises the state. At $n = 9$, the average time per gate is $0.6$ to $16\,\mu$s for Qulacs and $3$ to $12\,\mu$s for QuEST, against $26$ to $40\,\mu$s for Qiskit Aer. Up to $n = 12$, almost all sampled time in Qiskit Aer falls outside the numerical kernels (Appendix A.2) and in the Python and C++ layers, which prepare the circuit before execution, including any gate fusion. The CPU fallback of the GPU-oriented CUDA-Q framework (Section 4.3.1) is slower still, taking two to three orders of magnitude longer than Qulacs from $n = 12$ onwards. However, high fixed costs and efficient kernels coexist in Qiskit Aer, so its ranking at small sizes does not predict its ranking at larger ones.

Above the transition, kernel performance determines the ranking. Each added qubit doubles the state, so the time per gate doubles if the processing rate remains unchanged. Between $n = 20$ and $21$, however, the

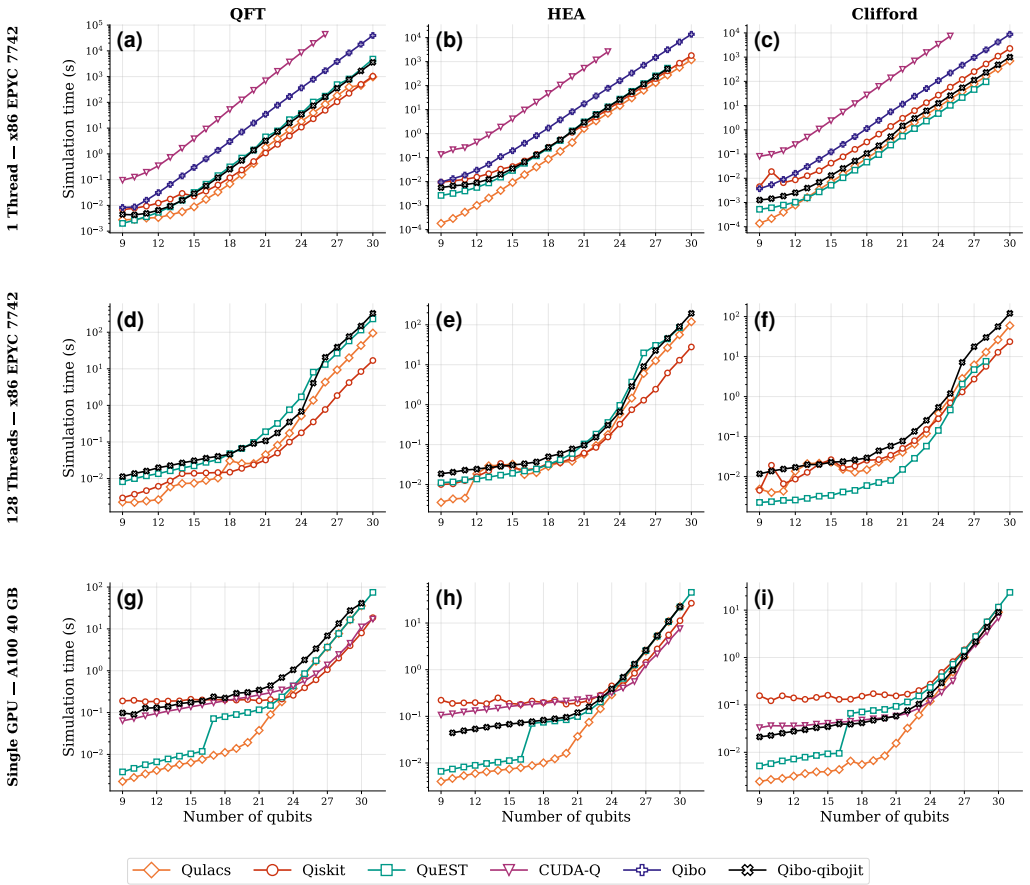



**Figure 5.1:** Simulation time as a function of qubit count $n$ for state-vector simulation of the QFT, HEA and Clifford circuits in three execution regimes. Each backend evolves or returns the full $2^n$-amplitude state vector, without tensor-network, stabiliser or approximate methods. Panels (a)–(c) show single-thread execution on one core of an AMD EPYC 7742, (d)–(f) full-node execution with 128 threads on the same node and (g)–(i) single-GPU execution on an NVIDIA A100-40, and the columns correspond to the QFT, the HEA and the Clifford circuit. All runs use double precision. In panel (d), the Qulacs QFT points with 9 to 12 qubits ran confined to one core, and in panel (e) the 13-qubit Qiskit Aer HEA point ran on a login node, so these points are not full-node times. The $y$-axis is logarithmic in every panel.

average time per gate of Qulacs rises almost fourfold on the QFT and HEA, and threefold on the Clifford circuit (Figure 5.1a–c). At this step, the state not only doubles in size, from $16$ to $32$ MiB, but also exceeds the $16$ MiB L3 cache available to one EPYC 7742 core (Table 4.2). The extra slowdown is consistent with the kernels fetching more data from main memory as they lose cache reuse. The jump is smaller on the Clifford circuit because its time per gate starts higher. Up to $n = 20$, the average time per gate of Qulacs on the Clifford circuit is $1.3$ times that on the HEA, and from $n = 21$ the two nearly coincide. In contrast, the average time per gate of Qiskit Aer approximately doubles on all three circuits, with no comparable jump at the cache boundary. Its fused blocks perform more arithmetic per pass (Section 3.5.2), which may explain why the loss of cache reuse affects its runtime less.

As a result, Qiskit Aer overtakes Qulacs at $n = 21$ on the QFT (Figure 5.1a), although it remains slower on the HEA and Clifford circuit (Figure 5.1b and c). On the QFT, Qiskit Aer is $1.6$ to $1.8$ times faster than Qulacs up to $n = 28$, and their times nearly coincide at $n = 29$ and $30$.

At $n = 26$, the default fusion of Qiskit Aer reduces the $1378$ gates of the QFT to about $106$ blocks, each taking as long as about eight gates of Qulacs, so the reduction in operations outweighs the higher cost per block (Section 3.5.2). On the HEA, it reduces $858$ gates to about $105$ blocks, each taking as long as about twelve gates of Qulacs, so the reduction does not offset the cost per block, and Qiskit Aer remains about $1.5$ times slower than Qulacs up to $n = 30$ (Figure 5.1b). On the Clifford circuit, the lead of Qulacs over Qiskit Aer is wider, at about $3$ times (Figure 5.1c).

Among the backends that apply only one gate at a time, Qulacs remains fastest on the QFT and HEA. From $n = 21$, QuEST and `qibojit` take two to three times as long per gate. Both rely on compiler vectorisation, whereas Qulacs uses hand-written AVX2 kernels (Section 4.3.1). Section 5.5 examines the contribution of this vectorisation in Qulacs on the i9-13900KS platform, using runtime measurements and hardware counters. On the Clifford circuit, however, QuEST leads Qulacs from $n = 13$ to $28$, running $1.4$ to $1.9$ times faster from $n = 16$ (Figure 5.1c).

For all three circuits, starting at $n = 17$, Qibo's default backend is six to thirteen times slower than `qibojit`, with the largest gap on the QFT. Both backends run the same circuits through the same interface and differ only in how they apply the gates. The default backend uses generic NumPy array operations that can require copies of the state, adding memory writes that `qibojit` avoids with kernels compiled just in time (Sections 4.3.1 and 4.3.2). The top-down diagnostics in Appendix A.3, collected for the QFT at $n = 26$ and $28$ on the i9-13900KS platform, show a corresponding difference in processor activity. With the default backend, store stalls account for up to half of the pipeline slots, while fewer than $20$% are classified as retiring. With `qibojit`, no store stalls are recorded, and about two thirds of the slots retire. Since each backend runs the same execution path on both platforms, these measurements support additional memory writes as a contributor to the QFT gap measured on Deucalion (Figure 5.1a).

**Multi-thread CPU regime.** With $128$ threads on the full node (Figure 5.1d–f), runtime begins to double with each added qubit between $n = 21$ and $24$ qubits, because the threads shorten the state updates but not the fixed costs, so the updates dominate only at larger sizes. Up to $n = 13$, multithreading brings no speedup except for Qiskit Aer on the QFT, which runs about twice as fast. On the Clifford circuit, Qulacs even runs up to thirty-five times slower than on one thread, as the overhead of dividing these small state updates among threads outweighs the work saved. As the state grows, every backend gains from parallel execution, but Qiskit Aer benefits most and leads all three circuits at the largest sizes.

Qiskit Aer leads the QFT from $n = 18$ (Figure 5.1d), the HEA from $n = 22$ (Figure 5.1e) and the Clifford circuit from $n = 26$ (Figure 5.1f); on the Clifford circuit, QuEST remains fastest up to $n = 25$. At $n = 30$, the speedup of Qiskit Aer over its own single-thread time (Figure 5.1a–c) reaches about $60$ on the QFT, $65$ on the HEA and almost $100$ on the Clifford circuit. On the HEA and the Clifford circuit, the speedup of Qulacs at the same size is $10$ and $12$, respectively, and for those same circuits, QuEST and `qibojit` have

a speedup of at most about $13$ at $n = 28$. At this size, the $4$ GiB state is eight times the $512$ MiB of L3 cache on the node (Table 4.2), so each pass over the state streams data from main memory. The fewer passes enabled by fusion therefore help explain the lead of Qiskit Aer. On one thread, the extra arithmetic of each fused block costs more time than the passes it removes on the HEA and the Clifford circuit. On $128$ threads, however, that arithmetic is divided among all the cores, while the bandwidth to main memory is shared by them, so the number of passes sets the time, which is consistent with the larger speedups of Qiskit Aer given above. At $n = 26$, fusion reduces the operation count thirteenfold on the QFT and eightfold on the HEA, and the lead of Qiskit Aer is correspondingly wider on the QFT than on the HEA, and narrowest on the Clifford circuit. At the other end of the ranking, `qibojit` is the slowest of the four backends on all circuits at the largest sizes.

Qulacs is second on the QFT and the HEA at the largest sizes, whereas on the Clifford circuit QuEST is second at $n = 28$. For the three circuits, between $n = 23$ and $26$ the curves of Qulacs climb more steeply than those of Qiskit Aer (Figure 5.1d–f), so that by $n = 26$ Qulacs takes two to six times as long, depending on the circuit. For the $30$-qubit QFT, Section 5.2 relates the scaling limit of Qulacs, reached at $16$ threads, to memory bandwidth.

**Single-GPU regime.** On one NVIDIA A100-40 (Figure 5.1g–i), with Qibo using its `qibojit`/`CuPy` backend, the average time per gate begins to double with each added qubit between $n = 20$ and $21$ for Qulacs, $24$ to $27$ for QuEST, depending on the circuit, $25$ and $26$ for `qibojit` and $27$ and $28$ for Qiskit Aer, and CUDA-Q reaches this rate only around $n = 28$. These transitions occur later than on a single CPU thread because the GPU shortens the state updates much more than the fixed costs, which the nearly flat curves up to $n = 20$ mainly show (Section 4.3.2). In this range the state occupies at most $16$ MiB, which one full read and write would move in about $22\,\mu$s at the nominal $1.55$ TB/s of the A100-40 (Table 4.2), whereas between $n = 12$ and $18$ the fixed costs amount to $13$ to $20\,\mu$s per gate for Qulacs and $0.1$ to $0.4$ ms per gate for `qibojit`, whose GPU kernels run through `CuPy` (Section 4.3.1), so the state updates outweigh the fixed costs only from about $n = 20$ for Qulacs and later still for the backends with higher fixed costs. Qiskit Aer and CUDA-Q, by contrast, process the entire circuit in a single call and pay their fixed costs, which include fusion and, for CUDA-Q, just-in-time compilation, only once per circuit.

QuEST alone shows a step in Figure 5.1g–i, between $n = 16$ and $17$ on all three circuits, where its time rises six- to sevenfold while every other curve stays flat. Its fixed cost is $22$ to $30\,\mu$s per gate up to $n = 16$, depending on the circuit, and five to seven times higher from $n = 17$. Before each non-diagonal gate it allocates a temporary device array for the qubits of the gate and frees it afterwards, and traces show these two calls rising from a few microseconds to about $0.1$ ms each at $n = 17$, while the gate kernels themselves take $3$ to $5\,\mu$s at both sizes, and the diagonal $R_Z$ gate, which allocates nothing, shows no step. This step is consistent with the state reaching $2$ MiB, the granularity of CUDA device allocations, so that the temporaries no longer share its allocation.

QULACS has the lowest fixed costs, since each gate passes straight to a native CUDA kernel and its timed call covers only the gates and the allocation of the state (Section 4.3.2), so up to $n = 20$ its whole simulation time is shorter than the per-circuit fixed cost of QISKIT AER or CUDA-Q alone. It therefore leads on every circuit up to $n = 23$, running at least six times faster than QISKIT AER, CUDA-Q and `qibojit` up to $n = 20$. QUEST, whose fixed cost is the next lowest, is the second fastest up to $n = 16$.

As the state vector grows, runtime depends increasingly on how many passes each backend makes over it and how long each pass takes. On the HEA at $n = 30$, QULACS and `qibojit` both take $23$ ms per gate and QUEST $22$ ms, close to the $22$ ms estimated for one read and write of the $16$ GiB state at the nominal bandwidth of the A100-40 (Table 4.2), so each gate corresponds to roughly one full pass over the state. QISKIT AER and CUDA-Q, however, fuse gates into fewer operations (Section 4.3.2), so the original gate count alone does not predict their runtime. At $n = 30$, QISKIT AER is $4.4$ times faster than QULACS on the QFT, while CUDA-Q is about $3$ times faster on the HEA and $1.3$ times faster on the Clifford circuit. These times include a copy of the state to host memory, a cost that grows with the state and that the timed calls of QULACS, `qibojit` and QUEST do not include, so in these cases the time that fusion saves outweighs the cost of the copy.

The ranking of the two fusing backends depends on the circuit. QISKIT AER leads the QFT from $n = 24$ to $30$, running $1.1$ to $1.7$ times faster than CUDA-Q (Figure 5.1g). This advantage is consistent with its fusion of controlled rotations into diagonal operations, which reduced the operation count thirteenfold on one CPU thread at $n = 26$. On the HEA and Clifford circuit, which contain no controlled rotations, CUDA-Q is fastest from $n = 25$ (Figure 5.1h–i). For the Clifford circuit, QULACS and `qibojit` also finish ahead of QISKIT AER, which is the slowest of the five at every size up to $n = 30$, although from $n = 27$ QUEST takes almost as long. On the QFT and the HEA, the QUEST curve joins that of QULACS from about $n = 25$, and it takes $35$ s for the QFT at $n = 30$. The single-GPU series extends to $n = 31$, where the double-precision state occupies $32$ GiB, the largest state vector that fits an A100-40 (Table 3.2).

## 5.2 QFT Thread Scaling on One Node

Figure 5.2 follows the QFT at $n = 30$ from one thread to all $128$ cores of a Deucalion x86 node. At this size, the state vector occupies $16$ GiB, far more than any cache, so every pass over it goes to DRAM. The sweep measures strong scaling in the layout of Table 4.4. Figure 5.1 shares some of its $30$-qubit QFT points with this sweep and takes the others from separate runs, so its $128$-thread times for QUEST and QULACS may slightly differ from the fastest times of the sweep (by a maximum of 4%).

The three speedup curves separate beyond eight threads. Up to that count they roughly follow the ideal line, and QISKIT AER stays close to it up to $64$ threads, with a small further gain from the second socket (Table 5.1). QUEST and QULACS obtain most of their speedup by $16$ threads; QUEST gains less than one

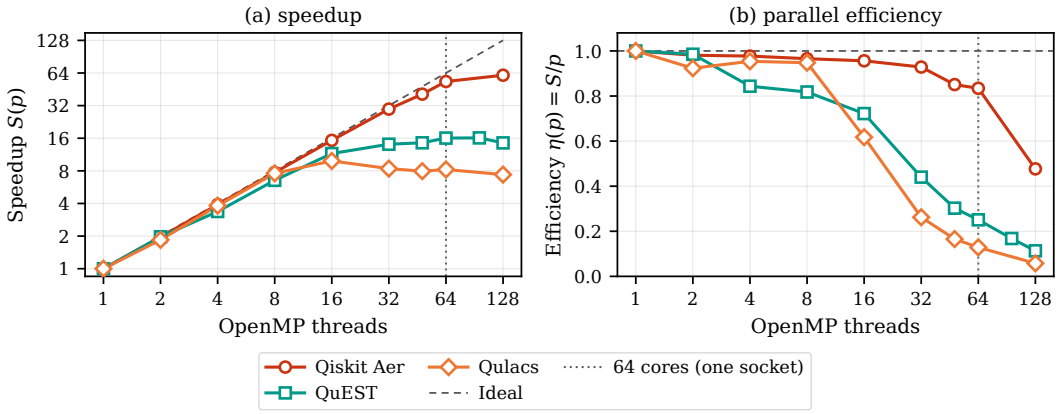



**Figure 5.2:** Thread scaling of the QFT at $n = 30$ (16 GiB state vector) on one Deucalion x86 node with two AMD EPYC 7742 sockets (128 cores, SMT off); each point is the fastest of ten repetitions, each thread count running in a fresh process after one untimed warm-up. (a) Speedup $S(p)$ over each backend's single-thread time; the dashed line is ideal linear speedup. (b) Parallel efficiency $\eta(p) = S(p)/p$. The dotted line marks 64 threads, the core count of one socket.

**Table 5.1:** Summary of QFT thread scaling at $n = 30$ (Figure 5.2). $S_2$ is the speedup at two threads, $S_{\max}$ the best measured speedup, $S_{64}$ the speedup with as many threads as one socket has cores and $S_{128}$ the speedup on both sockets. $\eta(p) = S(p)/p$ is the parallel efficiency (Section 4.1), and Table 5.2 gives the Karp–Flatt fraction.

| Backend | $S_2$ | $S_{\max}$ | $S_{64}$ | $S_{128}$ | $\eta_{64}$ | $\eta_{128}$ | Scaling signature |
|---|---|---|---|---|---|---|---|
| Qiskit Aer | 1.96× | 61.03× @ 128t | 53.40× | 61.03× | 0.834 | 0.477 | Near-linear to 64t, 1.14× more at 128t |
| QuEST | 1.97× | 16.12× @ 96t | 16.05× | 14.50× | 0.251 | 0.113 | Plateaus at 64t, then declines |
| Qulacs | 1.85× | 9.88× @ 16t | 8.23× | 7.39× | 0.129 | 0.058 | Peaks at 16t, then declines |

per cent beyond 64 threads, and Qulacs peaks at 16 and loses speedup with more threads.

This pattern is consistent with a memory-bandwidth limit, the memory-bound regime of the roofline model (Section 4.1). On most systems, applying a gate to a state vector of this size is memory-bandwidth bound [10]. Adding threads raises the compute ceiling, but once the running threads saturate their available bandwidth, more threads cannot shorten a pass, and speedup stops growing. The less arithmetic each pass carries, the fewer threads it takes to reach this point. QuEST and Qulacs apply the circuit one gate at a time, so each pass carries a single gate, and they stop gaining at 64 and 16 threads, respectively. The two limits differ because each thread of Qulacs demands more bandwidth. Up to eight threads, where the curves still roughly follow the ideal line, QuEST takes 3.5 to 4.3 times as long as Qulacs, so a thread of Qulacs moves data about four times faster, and about a quarter as many threads fill the same channels. Qiskit Aer, in contrast, fuses several gates into each pass, so each of its threads needs less bandwidth and the limit arrives later. From 64 to 128 threads, however, every speedup changes little (Table 5.1). At this step the run spans both sockets, and a thread on the second socket adds bandwidth only when the amplitudes it updates are held in the memory attached to that socket; otherwise it reads them across the socket interconnect, at higher latency and lower bandwidth than a local read (Section 3.5.3). Since the OpenMP runtime placed the threads (Table 4.4), the share of the state that each socket holds locally is not

known, so the step is consistent with reaching the bandwidth limit of the full node or with the cost of this remote traffic.

The Karp–Flatt fraction $e(p)$ of Table 5.2 (Section 4.1) separates Qiskit Aer from the other two backends in the same way. For Qiskit Aer, it stays below one per cent from four threads up to the full node and rises only at the last step, where its speedup flattens. For QuEST and Qulacs, it grows at every step from $16$ threads, and on the full node it is $2.4$ times its value at $16$ threads for QuEST and $3.1$ times for Qulacs, which points to overhead or a shared bottleneck that grows with parallelism. The fraction itself does not identify the resource, so it cannot confirm the bandwidth limit of the previous paragraph.

**Table 5.2:** Karp–Flatt fraction $e(p)$ (Section 4.1) of the thread sweep of Figure 5.2, computed from the speedup over each backend's single-thread time.

| Backend | 2t | 4t | 8t | 16t | 32t | 64t | 128t |
|---|---|---|---|---|---|---|---|
| Qiskit Aer | 0.019 | 0.008 | 0.005 | 0.003 | 0.002 | 0.003 | 0.009 |
| QuEST | 0.015 | 0.062 | 0.032 | 0.026 | 0.041 | 0.047 | 0.062 |
| Qulacs | 0.083 | 0.016 | 0.008 | 0.041 | 0.091 | 0.108 | 0.129 |

In absolute time, the sweep divides into two regions for the QFT at $n = 30$. Up to eight threads, the times of Qulacs and Qiskit Aer differ by at most $6$%, with QuEST behind both. From $16$ threads Qiskit Aer is the fastest of the three backends, and because it alone keeps gaining up to the full node, its lead grows with the thread count, so that on $128$ threads it completes the circuit about eight times sooner than Qulacs and fifteen times sooner than QuEST.

## 5.3 Multi-Node CPU Scaling

Distributing a simulation over nodes serves two purposes. It can shorten a run that fits on one node, and it can hold a state vector that exceeds the memory of one node. Which of the two applies depends on capacity, because a Deucalion x86 node holds a double-precision state vector up to $n = 33$, where it occupies $128$ GiB of the node's memory (Table 4.2), whereas at $n = 34$ the $256$ GiB state must be spread over at least two nodes. The strong-scaling curves of Figure 5.3 therefore address the first purpose, keeping the QFT fixed at $n = 30$, a $16$ GiB state that fits one node with room to spare, on up to $64$ nodes, and the weak-scaling series of Figure 5.4 the second, growing the state with the node count until it crosses the capacity of one node. Both series use the QFT, so the comparison holds for this circuit.

All multi-node runs reserve whole nodes of $128$ cores, and each backend runs in the layout that uses all of them. mpiQulacs and QuEST run one MPI rank per node whose $128$ OpenMP threads cover all the cores, the layout they are built for and the one the authors of QuEST used for their own distributed benchmarks [82], and within a node their threads share one copy of the node's part of the state, so gates on it need no messages. Qiskit Aer, in contrast, runs $128$ MPI ranks per node, one on each core, the layout that Section 4.3.1 explains. With one rank per node, the engine would keep only one core of each node busy, and that layout

is shown as a dashed line for comparison. Each point is the fastest of all repetitions across all jobs, which lie within 5% of one another (Appendix A.1).

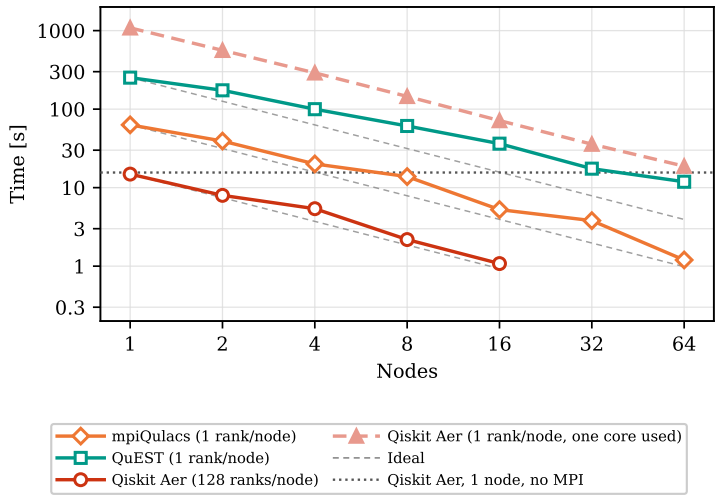



**Figure 5.3:** Strong scaling of the QFT at $n = 30$ on Deucalion x86, each backend in its best layout, with mpiQulacs and QuEST at one MPI rank per node and 128 threads and Qiskit Aer at 128 MPI ranks per node, one per core (solid, dark red), measured up to 16 nodes. The dashed light-red line is Qiskit Aer with one rank per node (Section 4.3.1); its one-node point is the distributed engine forced onto one rank. Each point is the fastest of all repetitions across all jobs; grey dashed lines show ideal scaling from each backend's one-node time, and the dotted line marks Qiskit Aer on one node without MPI (15.58 s).

**Table 5.3:** Strong scaling of the QFT at $n = 30$ on Deucalion x86, each backend in its best layout, with Qiskit Aer at one rank per node for comparison. Times in seconds on 1 to 64 nodes; each is the fastest of all repetitions across all jobs, and italics mark a point measured in a single job. $S_{64} = T(1\,\text{node})/T(64\,\text{nodes})$ uses each row's own one-node time. The last row is each node's share of the 16 GiB state vector. *Distributed engine forced onto one rank. †Computed from the time of Qiskit Aer on one node without MPI (15.58 s). ‡At 16 nodes, the highest node count measured with 128 ranks per node; dashes mark node counts beyond the measured range.

| Backend | Layout | 1 | 2 | 4 | 8 | 16 | 32 | 64 | $S_{64}$ |
|---|---|---|---|---|---|---|---|---|---|
| mpiQulacs | 1 rank/node | 63.04 | 39.20 | 20.04 | 13.83 | 5.25 | 3.80 | *1.20* | 52.6× |
| QuEST | 1 rank/node | 251.89 | *173.71* | 99.85 | *61.02* | 36.33 | *17.44* | *11.89* | 21.2× |
| Qiskit Aer | 128 ranks/node | 14.87 | 7.97 | 5.39 | 2.18 | 1.08 | – | – | 13.8×‡ |
| Qiskit Aer | 1 rank/node | 1084.07* | 560.76 | 289.54 | 145.23 | 71.48 | 35.78 | 18.76 | 0.83×† |
| State per node | | 16 GiB | 8 GiB | 4 GiB | 2 GiB | 1 GiB | 512 MiB | 256 MiB | |

With one rank per node for all three backends, mpiQulacs is the fastest at every node count, QuEST follows, and Qiskit Aer is the slowest, about an order of magnitude behind mpiQulacs, because of the single busy core per node noted above. On one node, where no data cross the network, mpiQulacs already runs about four times faster than QuEST, since it applies the gates that its optimiser has fused, whereas QuEST applies the QFT one gate at a time with compiler-vectorised kernels (Section 5.1). With 128 ranks per node, Qiskit Aer is instead the fastest backend at every node count measured, from one to 16 nodes, running between 3.7 and 6.3 times faster than mpiQulacs at equal node counts.

Most of that lead, however, already exists on one node, where Qiskit Aer runs about four times faster than mpiQulacs. Part of it comes from the advantage that Qiskit Aer shows on a full node in Section 5.1, where

gate fusion lets it apply the QFT in fewer passes over the state. Another part may come from mpiQulacs itself, since the thread sweep of Section 5.2 showed Qulacs reaching its best speedup at $16$ threads on one node and losing part of it with more, whereas here each rank runs $128$ threads. The gap is nonetheless half of the one in that sweep, because mpiQulacs runs its optimiser and fuses gates before the clock starts. Distribution then keeps that gap roughly constant, as both backends become about $12$ to $14$ times faster from one to $16$ nodes. On $64$ nodes, mpiQulacs and QuEST run about $53$ and $21$ times faster than on one node (Table 5.3). The lower speedup of QuEST is consistent with its exchange scheme, since it exchanges data in pairs before every non-diagonal gate on a global qubit, whereas mpiQulacs moves several global qubits in one FusedSWAP (Figure 4.5). The last step of mpiQulacs, from $32$ to $64$ nodes, shortens the run by more than the ideal factor of two, at the node count where each node's share of the state, $256$ MiB, first falls below the $512$ MiB of L3 cache of the node, having equalled it on $32$ nodes (Table 4.2), which is consistent with the passes over the state no longer streaming from main memory, although the step from $8$ to $16$ nodes, far above the cache size, also shortens the run by more than a factor of two.

Table 5.4 gives the Karp–Flatt fraction of these speedups (Section 4.1). For every backend, the fraction is lower at the final node count than at two nodes, so the losses grow more slowly than a fixed serial fraction would produce, although the efficiency of QuEST and Qiskit Aer still falls and the fraction alone does not show whether the time spent on exchanges increases or decreases.

For mpiQulacs and QuEST, the highest fraction occurs with two nodes, when data are first exchanged over the network. If the two-node values were fixed serial fractions, Amdahl's law would cap the speedup at about $4.1$ times for mpiQulacs and $2.6$ times for QuEST. Both backends exceed these caps by far on $64$ nodes, so the loss at the first step marks the start of the exchanges and is not a fixed serial fraction. Qiskit Aer with one rank per node keeps the lowest fraction at every node count, since each node computes on a single core and exchanges add at most a few per cent to the total time. The fractions of mpiQulacs and of Qiskit Aer with $128$ ranks per node rise at some steps, by more than the spread between repetitions would explain (Appendix A.1), but their cause is not identified, so only the overall downward trend is read from the table.

**Table 5.4:** Karp–Flatt fraction $e(p)$ (Section 4.1) of the strong scaling of Table 5.3 on $p$ nodes, computed from the speedup over each row's own one-node time. Dashes mark node counts beyond the 128-ranks-per-node layout.

| Backend | Layout | 2 | 4 | 8 | 16 | 32 | 64 |
|---|---|---|---|---|---|---|---|
| mpiQulacs | 1 rank/node | 0.244 | 0.091 | 0.108 | 0.022 | 0.030 | 0.003 |
| QuEST | 1 rank/node | 0.379 | 0.195 | 0.134 | 0.087 | 0.039 | 0.032 |
| Qiskit Aer | 128 ranks/node | 0.072 | 0.150 | 0.025 | 0.011 | – | – |
| Qiskit Aer | 1 rank/node | 0.035 | 0.023 | 0.010 | 0.004 | 0.002 | 0.002 |

Splitting each node of mpiQulacs into $128$ one-thread ranks, mirroring the layout used by Qiskit Aer, slows it down (Appendix A.4). With $28$ qubits per node, as in the weak-scaling series of Figure 5.4, mpiQulacs in this layout ran about $1.4$ times slower on one node and $1.5$ times slower on two nodes than with one rank

per node, and about five times slower than Qiskit Aer with $128$ ranks per node under the same conditions. Part of this drop comes from each rank holding only $1/128$ of its node's share of the state, so gates on seven more of the qubits require MPI exchanges between ranks on the same node, and with the same settings the circuit optimiser of mpiQulacs also produced about $1.6$ times as many gates in this layout, $618$ instead of $392$ at $n = 28$. The $128$-ranks-per-node layout therefore helps Qiskit Aer and slows mpiQulacs, so each of the two is compared in its faster layout.

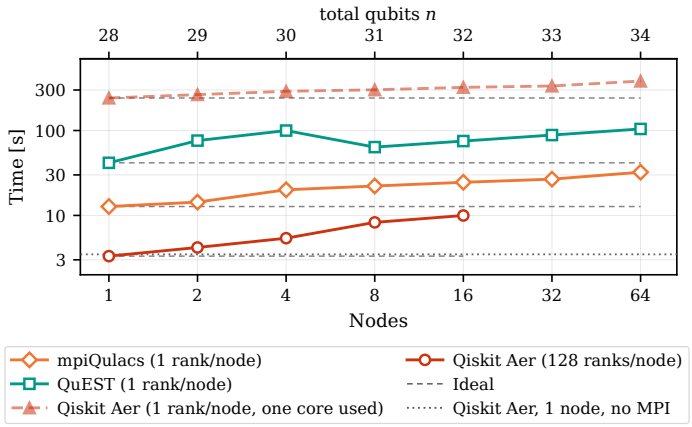



**Figure 5.4:** Weak scaling of the QFT on Deucalion x86 with $28$ qubits per node, from $n = 28$ on one node to $n = 34$ on $64$ nodes. mpiQulacs, QuEST and Qiskit Aer run one MPI rank per node with $128$ threads, the layout of Figure 5.3, whose one-node point for Qiskit Aer is its distributed engine forced onto one rank; Qiskit Aer is also measured with $128$ ranks per node up to $16$ nodes and $n = 32$. Each point is the fastest of all repetitions across all jobs; grey dashed lines mark each backend's one-node time, and the dotted line is Qiskit Aer on one node without MPI ($3.48$ s). The gate count grows with $n$ along each curve.

In the weak-scaling series of Figure 5.4, every node holds $4$ GiB of state, well below its capacity, so the series measures the cost of distribution at a fixed state per node, the regime of Gustafson's law (Section 4.1). With one rank per node, the three backends keep the ranking they have in strong scaling at every step. The QuEST series does not grow monotonically, since its time falls from four to eight nodes before rising again, a change larger than the spread between its repetitions (Appendix A.1). From one to $64$ nodes, where all three backends reach $n = 34$, the times of mpiQulacs and QuEST rise to about $2.5$ times their one-node values while the state grows $64$ times. A factor of about $1.5$ in that rise comes from the circuit itself, whose gate count grows with the number of qubits (Section 4.1). The rest comes from distribution, since each added qubit doubles the node count, and that qubit's gates move data across the network. Qiskit Aer with $128$ ranks per node is the fastest backend at every step measured, as in strong scaling, but its time grows faster along the series than that of mpiQulacs, so its lead narrows as nodes are added.

Across both series, the backends' ranking for the QFT on Deucalion x86 follows from capacity, which determines when nodes must be added, and from layout, which determines how fast each backend runs on them. Up to $n = 33$, where one node holds the entire state, adding nodes remains optional, and Qiskit Aer with $128$ ranks per node is the fastest backend at every node count measured, up to $16$ nodes, with a $16$-node time even shorter than that of mpiQulacs on $64$ nodes. On $32$ and $64$ nodes, and from $n = 34$, where distribution becomes necessary, mpiQulacs is the fastest backend measured.

## 5.4 GPU Acceleration

The GPU measurements move from one device to several devices, dividing the state vector among up to sixteen GPUs. Unlike the single-device comparison of Figure 5.1, whose Qiskit Aer and CUDA-Q times include copying the whole state to host memory, these runs leave the state on the GPUs, since at $n = 35$ the state reaches $512$ GiB, more than the host memory of a node. These runs also use A100-80 GPUs and distributed layouts, so the times of this section measure different work on different hardware from those of Figure 5.1, and the order of the backends can change from one to the other. The multi-GPU runs use CUDA-Q, Qiskit Aer and QuEST with one MPI rank per GPU, exchanging data by the paths of Section 4.3.2, and run Qiskit Aer with the CUDA IPC transport of UCX[1] enabled, without which its exchanges within a node pass through host memory (Appendix A.6).

### 5.4.1 Within One Node

The effect of the circuit and of the state size on multi-GPU execution is measured with CUDA-Q, which runs the QFT and the HEA of Table 4.1 from $n = 28$ to $34$ on one, two and four A100-80 GPUs (Figure 5.5, Table 5.5).

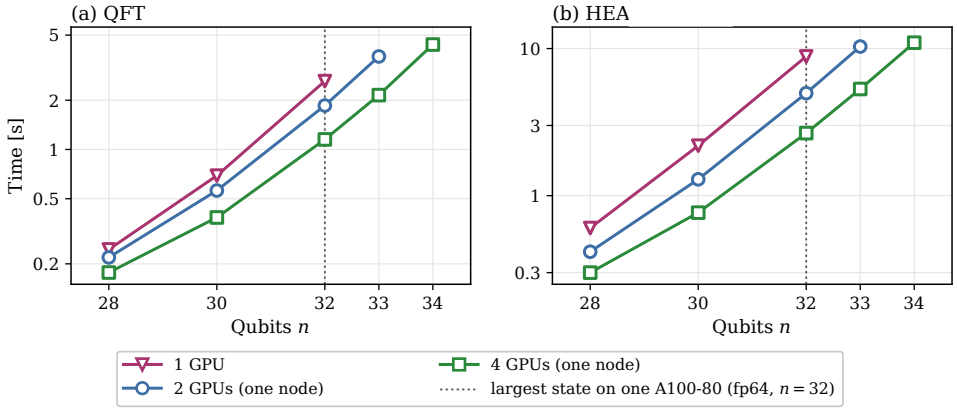



**Figure 5.5:** CUDA-Q time against the number of qubits on one, two and four A100-80 GPUs of one node, in double precision (64-bit), for (a) the QFT and (b) the HEA of Table 4.1. Each point is the median of ten repetitions, and each line comes from a single job. The dotted line marks $n = 32$, the largest state that one A100-80 holds in double precision ($64$ GiB).

Memory sets how many GPUs each size needs, since in double precision one A100-80 holds the state up to $n = 32$, which occupies $64$ GiB, whereas $n = 33$ needs two GPUs, and $n = 34$ needs all four GPUs of a node. The next size, $n = 35$, needs two nodes (Section 5.4.2).

When the problem size fits on a single GPU, dividing the state across multiple GPUs speeds up execution, more at larger sizes and for circuits that require more computation per pass. At $n = 28$, the three QFT points in Figure 5.5 nearly overlap, as updating a $4$ GiB state takes only milliseconds, making inter-GPU exchange costs comparable to the computational savings. At $n = 32$, the lines diverge, as four GPUs reduce the QFT

[1] UCX is the communication library under Open MPI; its CUDA IPC transport copies data directly between GPUs of one node.

**Table 5.5:** CUDA-Q time in *seconds* on one, two and four A100-80 GPUs of one node, in double precision (64-bit). Each value is the median of ten repetitions, and each row comes from a single job. Dashes mark the sizes whose state does not fit on that number of GPUs. Figure 5.5 plots these values.

| | **GPUs** | $n = 28$ | $n = 30$ | $n = 32$ | $n = 33$ | $n = 34$ |
|---|---|---|---|---|---|---|
| QFT | 1 | 0.245 | 0.692 | 2.617 | – | – |
| | 2 | 0.219 | 0.561 | 1.856 | 3.702 | – |
| | 4 | 0.177 | 0.384 | 1.152 | 2.149 | 4.379 |
| HEA | 1 | 0.603 | 2.187 | 8.842 | – | – |
| | 2 | 0.416 | 1.292 | 4.975 | 10.318 | – |
| | 4 | 0.300 | 0.767 | 2.663 | 5.311 | 10.962 |

runtime by more than half. The HEA, whose gates require more computation per pass, achieves the greater speedup at every size, and on four GPUs at $n = 32$ its runtime falls by over two-thirds.

Read along the diagonal of Table 5.5, where each GPU holds $64$ GiB, these runs form a weak-scaling ladder in the regime of Gustafson's law (Section 4.1). On two and four GPUs, the QFT keeps a weak efficiency of $0.71$ and $0.60$, against ideal values of $0.94$ and $0.89$ once its gate count grows (Section 4.1), and the HEA of $0.86$ and $0.81$, against $0.97$ and $0.94$. Both circuits reach $n = 34$, the largest state that one node holds, in about eleven seconds or less.

Table 5.6 gives the Karp–Flatt fraction of the same runs (Section 4.1). It falls from two to four GPUs for both circuits at every size, so the losses grow more slowly than a fixed serial fraction would produce, although the efficiency $S(p)/p$ still falls from two to four GPUs and the fraction alone does not establish whether the communication time itself increases or decreases. It also falls as the number of qubits grows, and on four GPUs from $n = 28$ to $n = 32$ it drops to less than half for the QFT and to about a fifth for the HEA, which is consistent with part of the cost of dividing the state, such as synchronising the GPUs and setting up the exchanges, being amortised as the state grows. Read as serial fractions, the four-GPU values at $n = 32$ would let Amdahl's law cap the speedup at about four times for the QFT and about fourteen for the HEA, however many GPUs were added. In contrast, across nodes the time rises (Section 5.4.2).

**Table 5.6:** Speedup $S(p) = T(1)/T(p)$ and Karp–Flatt fraction $e(p)$ (Section 4.1) of CUDA-Q on two and four A100-80 GPUs of one node, computed from the times of Table 5.5.

| | | **Speedup** | | **Karp–Flatt** | |
|---|---|---|---|---|---|
| **Circuit** | $n$ | $S(2)$ | $S(4)$ | $e(2)$ | $e(4)$ |
| QFT | 28 | $1.12\times$ | $1.38\times$ | $0.79$ | $0.63$ |
| | 30 | $1.23\times$ | $1.80\times$ | $0.62$ | $0.41$ |
| | 32 | $1.41\times$ | $2.27\times$ | $0.42$ | $0.25$ |
| HEA | 28 | $1.45\times$ | $2.01\times$ | $0.38$ | $0.33$ |
| | 30 | $1.69\times$ | $2.85\times$ | $0.18$ | $0.13$ |
| | 32 | $1.78\times$ | $3.32\times$ | $0.13$ | $0.07$ |

### 5.4.2 Across Nodes

The cross-node runs use $n = 30$, a 16 GiB state that fits on one GPU, and $n = 32$ to $n = 35$, from the largest state one GPU can hold to one that exceeds a four-GPU node. At $n = 30$, all three backends speed up to four GPUs and slow down when the run moves to two nodes with eight GPUs (Figure 5.6). With the state left on the device, CUDA-Q is about four times faster than Qiskit Aer on one GPU and three to four times faster within the node, but on eight GPUs the two take the same time, since CUDA-Q slows down by about six times relative to its four-GPU time and Qiskit Aer by about two. QuEST follows the same shape at a higher level, twice the time of Qiskit Aer and eight times that of CUDA-Q on one GPU, and it gains the most within the node, 3.4 times on four GPUs against 2.7 for Qiskit Aer and 1.9 for CUDA-Q, which is consistent with its applying one unfused gate per pass, so that each GPU's own work dominates the run and shrinks with every GPU added. At the node boundary it slows by 2.5 times, close to Qiskit Aer, and from eight to sixteen GPUs it gains nothing.

With 30 qubits, the simulation completes in about a second or less on one node, so adding GPUs saves at most a fraction of a second, and even less when exchanges between GPUs must cross InfiniBand. On sixteen GPUs, CUDA-Q and Qiskit Aer partly recover this loss without reaching their single-node time, and Qiskit Aer recovers more and becomes faster than CUDA-Q. The same pattern has been reported elsewhere. The authors of the cache-blocking engine of Qiskit Aer found a 30-qubit simulation becoming slower as nodes were added [57], and CUDA-Q on a system of GB200 GPUs, with the nodes connected over InfiniBand, slowed from four to eight GPUs, the step at which the run first crossed nodes, and improved again as further nodes were added [56].

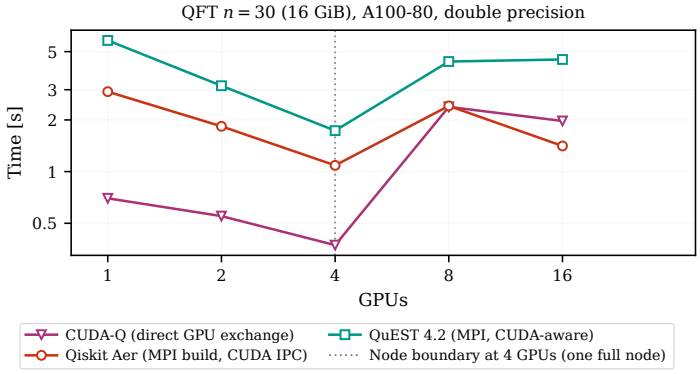



**Figure 5.6:** Multi-GPU QFT time at $n = 30$ qubits (16 GiB state vector) on Deucalion A100-80 nodes with four GPUs each. The dotted line at four GPUs marks one full node, so runs up to four GPUs exchange data over NVLink, and the eight- and sixteen-GPU runs also exchange data between nodes over InfiniBand. All runs use double precision and one MPI rank per GPU, and each point is the median of ten repetitions from one job. Inside a node CUDA-Q exchanges data through its peer-to-peer path, Qiskit Aer through the CUDA IPC transport of UCX (Appendix A.6) and QuEST through CUDA-aware MPI.

From $n = 32$, all three backends follow the pattern of $n = 30$ at every size (Figure 5.7). Within one node, going from two to four GPUs reduces the runtime by about 40 to 50% for each of them.

With four GPUs on a single node, CUDA-Q is the fastest backend, followed by Qiskit Aer and then QuEST, and CUDA-Q leads Qiskit Aer by a factor of $3.6$ to $4.2$. However, this advantage mostly disappears when the computation extends across more than one node. From four to eight GPUs, the runtime of CUDA-Q increases by a factor of about eight at every size from $n = 32$ to $34$, while that of Qiskit Aer increases only about two and a half times, so CUDA-Q becomes only about $1.3$ times faster than Qiskit Aer on eight GPUs. Increasing the problem size does not reduce this penalty, since on eight GPUs each additional qubit roughly doubles the runtime of all three backends, consistent with the estimated time GPUs wait for exchanges growing with the data each GPU holds (Appendix A.5). The same ordering therefore remains at $n = 35$, the first size that requires two nodes for both CUDA-Q and Qiskit Aer. On sixteen GPUs, however, Qiskit Aer benefits more from the extra GPUs and overtakes CUDA-Q, becoming $1.1$ to $1.3$ times faster. QuEST remains the slowest backend at every GPU count, and since its exchange buffer doubles the memory required per GPU, it also needs twice as many GPUs to hold the same state, reaching $n = 33$, $34$ and $35$ on four, eight and sixteen GPUs, respectively. The step from four to eight GPUs increases its runtime by about two times at $n = 32$ and $33$, and at $n = 35$ on sixteen GPUs it takes $128$ s, about $2.5$ times as long as CUDA-Q and Qiskit Aer.

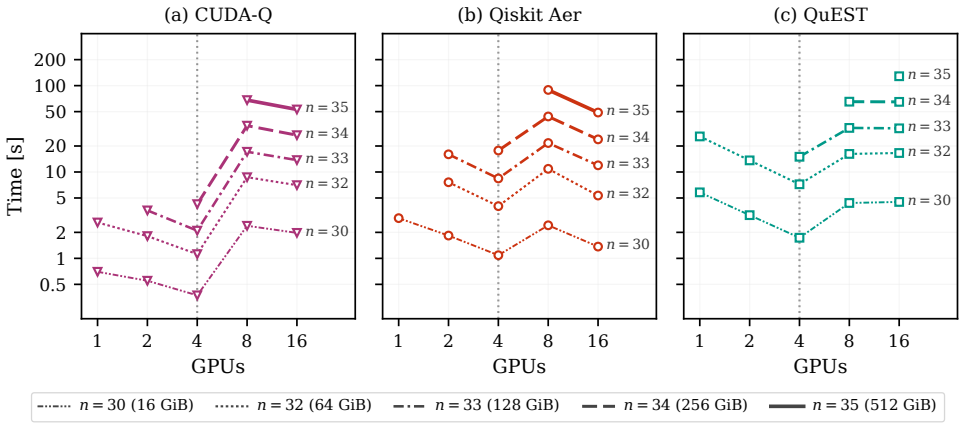



**Figure 5.7:** Multi-GPU QFT time from $n = 30$ to $n = 35$ on Deucalion A100-80 nodes with four GPUs each, for (a) CUDA-Q, (b) Qiskit Aer and (c) QuEST, in double precision with one MPI rank per GPU. QuEST holds at most $31$ qubits of the state per GPU, so from $n = 33$ each of its series starts one GPU count later than in (a) and (b), and its $n = 35$ point exists only on sixteen GPUs. The dotted line at four GPUs marks one full node, beyond which the GPUs also exchange data over InfiniBand. The line style gives the number of qubits $n$ and the size of the state vector, from $16$ to $512$ GiB, the first state that no node holds. Each point is the median of ten repetitions. Inside a node Qiskit Aer uses the CUDA IPC transport, with each repetition in its own process from $n = 32$ (Appendix A.6).

Read one step at a time (Table 5.7), the same runs place the whole loss at the node boundary, since every step within a node shortens the run and the step from four to eight GPUs lengthens it, for all three backends at every size. From eight to sixteen GPUs, with the exchanges already crossing the network, Qiskit Aer approximately halves its runtime, CUDA-Q gains about twenty per cent and QuEST nothing. Considering weak scaling instead, where each GPU holds $64$ GiB of state and Gustafson's law applies (Section 4.1),

**Table 5.7:** Incremental speedup $r(p) = T(p)/T(2p)$ (Section 4.1) of the QFT runs of Figure 5.7 on Deucalion A100-80 nodes with four GPUs each. A ratio of two is ideal scaling and a ratio below one a slower run, and the step from four to eight GPUs crosses the node boundary. Dashes mark steps whose run on fewer GPUs cannot hold the state. These runs are separate from those of Table 5.5, so the within-node ratios of CUDA-Q differ slightly from the speedups of Table 5.6.

| | | Within one node | | Node boundary | Across nodes |
|---|---|---|---|---|---|
| **Backend** | $n$ | **1 to 2** | **2 to 4** | **4 to 8** | **8 to 16** |
| CUDA-Q | 30 | 1.27 | 1.48 | 0.16 | 1.21 |
| | 32 | 1.44 | 1.62 | 0.13 | 1.24 |
| | 33 | – | 1.72 | 0.12 | 1.25 |
| | 34 | – | – | 0.12 | 1.29 |
| | 35 | – | – | – | 1.28 |
| Qiskit Aer | 30 | 1.59 | 1.69 | 0.45 | 1.76 |
| | 32 | – | 1.89 | 0.37 | 2.04 |
| | 33 | – | 1.90 | 0.39 | 1.81 |
| | 34 | – | – | 0.40 | 1.84 |
| | 35 | – | – | – | 1.83 |
| QuEST | 30 | 1.84 | 1.83 | 0.39 | 0.97 |
| | 32 | 1.90 | 1.89 | 0.45 | 0.98 |
| | 33 | – | – | 0.46 | 1.01 |
| | 34 | – | – | – | 1.00 |

the step from $n = 33$ on two GPUs to $n = 34$ on four GPUs increases runtime by a factor of $1.11$ for Qiskit Aer and $1.18$ for CUDA-Q, compared to $1.06$ expected from circuit growth alone[2]. The subsequent step, to $n = 35$ on eight GPUs across two nodes, multiplies the time by five for Qiskit Aer and by sixteen for CUDA-Q. For QuEST, whose ladder runs at $32$ GiB per GPU, the step within the node from $n = 32$ on two to $n = 33$ on four GPUs multiplies the time by $1.10$, the step across the boundary to $n = 34$ on eight GPUs by $4.3$ and the step between nodes to $n = 35$ on sixteen GPUs by $2.0$.

An estimate of the time in which the GPUs are active and a bandwidth test between GPU buffers are consistent with this pattern (Appendix A.5). Within one node, all three backends exchange data directly over NVLink, at about $89$ GB/s per GPU and direction when two GPUs swap $1$ GiB, and the estimates place their GPUs active, executing kernels, for most of the run, with the longer runs of Qiskit Aer and QuEST matched by longer estimated active time, so the gap between the backends inside the node appears to lie in their own execution on the GPUs rather than in the exchanges. Across nodes, the same exchanges cross InfiniBand, where under the default UCX settings of Deucalion each GPU swaps data at only about $0.9$ GB/s when all four GPUs of a node exchange at once, two orders of magnitude below NVLink, and the estimates place the GPUs of all three backends inactive for most of the run, for a time that grows with the data each GPU holds, as waiting for exchanges would. These estimates come from utilisation samples that also cover the correctness checks and the gaps between repetitions (Appendix A.5), so they indicate where the time goes without separating computation from communication inside the timed call. Read with that limitation, the similar inactive time of CUDA-Q and Qiskit Aer is consistent with their runtimes falling within $1.3$ times of

[2]The controlled phases of the QFT couple every pair of qubits, so its gate count grows with $n^2$, and the circuit at $n = 34$ has $1.06$ times as many gates as at $n = 33$. With the same state on every GPU, this is the whole increase that ideal weak scaling allows.

each other on eight GPUs, and with CUDA-Q slowing down more at the node boundary, since its run within the node is shorter, so the added time is a larger multiple of it.

The node boundary therefore determines the relative performance of the backends for the QFT distributed across GPUs. CUDA-Q is the fastest while the state fits within the four GPUs of one node, up to $n = 34$; once the run crosses to a second node, which $n = 35$ requires, its lead over Qiskit Aer narrows to about $1.3$ times on eight and reverses on sixteen GPUs, under the default UCX settings. QuEST trails both at every GPU count, $1.5$ to $2$ times behind Qiskit Aer up to eight GPUs and $2.6$ to $3.3$ times on sixteen GPUs.

## 5.5 Vectorisation and Gate Fusion

The roofline of Häner and Steiger (Figure 3.6) positions an unfused state-vector kernel in the memory-bound region. This placement is the reference for examining how vectorisation and gate fusion affect the QFT on the i9-13900KS platform. Their effect was measured with Qulacs, which sets the two optimisations independently, in the four configurations of Section 4.4.2 (scalar or AVX2 code, with fusion disabled or enabled through the `bs2` pass) on the $28$-qubit QFT on one performance core (Figure 5.8, Table 5.8). Wall-clock times give the runtime gains, and the roofline positions show how each optimisation changes the execution.

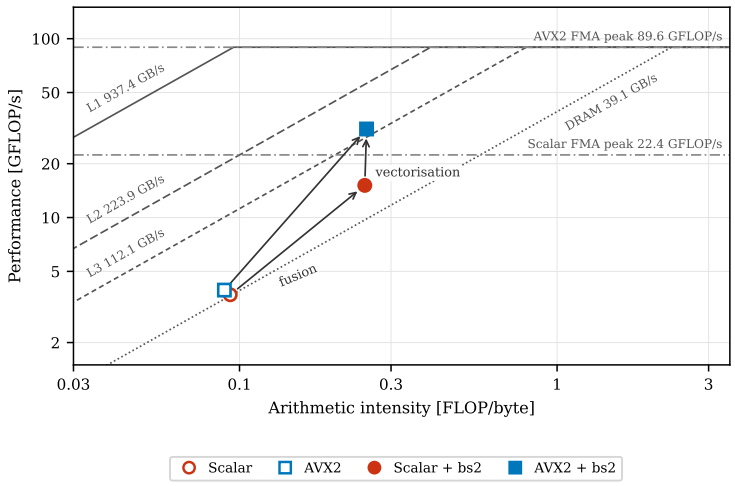



**Figure 5.8:** CARM PMU operating points for the $28$-qubit QFT on one performance core of the i9-13900KS platform at $5.6$ GHz. The four configurations combine the scalar and AVX2 builds of Qulacs (Section 4.4.2) with the circuit as decomposed or fused by the `bs2` pass into blocks of up to two qubits. Hollow markers are the unfused runs and filled markers the fused ones. Points use whole-process arithmetic-event counts and estimated core-side bytes, averaged over ten repetitions. Bandwidth ceilings are those that the CARM Tool reports for the i9-13900KS platform with AVX2 code, while compute ceilings are nominal double-precision peaks with fused multiply-add at the stated frequency. The intensity of the fused AVX2 build is a lower bound (Section 4.4.2). Arrows identify the changes associated with fusion and vectorisation.

Vectorisation yields greater runtime reduction when fusion is enabled. With `bs2`, the AVX2 build is approximately $2.0$ times faster than the scalar fused build. In contrast, without fusion, the AVX2 build is only about

$6\%$ faster, a difference that falls within the session-to-session variation (Section 4.4.2).

The roofline estimates in Figure 5.8 help interpret these differences. Fusion combines gates into dense operations on up to two qubits, reducing the number of passes over the state (Section 3.5.2). The computational work more than doubles while the estimated data movement decreases, raising the arithmetic intensity. Both fused points move rightwards and, as the larger workload also completes in less time, upwards, with the scalar point above the DRAM roof and the AVX2 point near the L3 roof. The uncertainty in the byte estimate, however, prevents this proximity from identifying L3 bandwidth as the limiting resource (Section 4.4.2). Vectorisation has a much smaller effect without fusion, since the arithmetic intensity remains nearly unchanged and performance rises only slightly, leaving both unfused configurations on or just above the DRAM roof. This behaviour is consistent with memory traffic limiting the benefit of wider instructions. With fusion, the scalar point lies within a factor of $1.5$ of the scalar FMA peak, leaving less headroom below the scalar compute ceiling. AVX2 raises that ceiling and roughly doubles the attained performance, showing the greater benefit of vectorisation when each pass performs more arithmetic.

In this single-core Qulacs experiment, fusion reduces runtime more than vectorisation, regardless of the other optimisation configuration. This finding aligns with the greater performance of the fusing backends observed at the upper end of the sweeps on the full node and on a single GPU (Sections 5.1 and 5.2). However, the ablation study isolates the effect of fusion within a single backend on one core, whereas the broader comparisons also reflect differences in backend implementations and parallel execution.

## 5.6 Noise-Aware Density-Matrix Simulation

To model NISQ-era noise exactly, the noisy runs here explicitly represent the noisy state by propagating a density matrix rather than a pure-state vector. Three density-matrix backends, Qiskit Aer, Qulacs and Qibo with its NumPy backend, are compared on the QFT with depolarising noise, and Qiskit Aer alone is run under the four channels of Section 2.4, namely depolarising, bit-flip, amplitude damping and phase damping, to compare their effects. Accuracy is measured by the state fidelity against the ideal output, the purity $\mathrm{Tr}(\rho^2)$ and the population error $\varepsilon_{L_2}$ (Section 4.4.3).

The time panel in Figure 5.9 shows two computational regimes. Up to $n = 7$, Qiskit Aer and Qulacs take a few hundredths of a second, almost independent of size, indicating that preparation and dispatch costs

**Table 5.8:** Qulacs configurations of Figure 5.8. Arithmetic intensity and performance are means of ten whole-process `perf` measurements, with the bytes estimated from the load and store counts as in Section 4.4.2. Wall-clock times are reported as mean $\pm$ sample standard deviation over the same ten repetitions.

| Configuration | Intensity [FLOP/byte] | Performance [GFLOP/s] | Wall-clock time [s] |
|---|---|---|---|
| Scalar | 0.094 | 3.70 | $338.33 \pm 0.19$ |
| AVX2 | 0.090 | 3.93 | $319.02 \pm 0.10$ |
| Scalar + `bs2` | 0.248 | 15.13 | $201.50 \pm 0.86$ |
| AVX2 + `bs2` | 0.251 | 31.27 | $98.42 \pm 0.05$ |

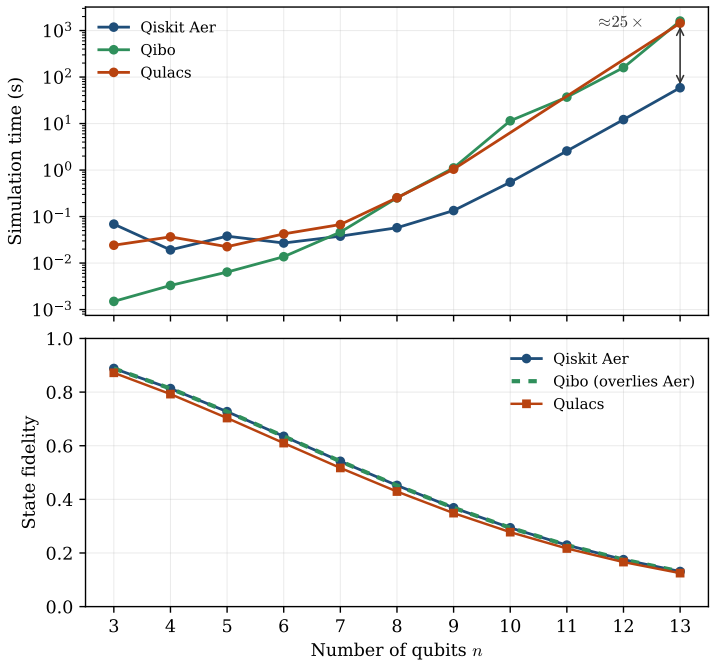



**Figure 5.9:** Density-matrix simulation of the QFT with depolarising noise of parameter $p = 0.01$ on the i9-13900KS platform, against the number of qubits $n$. Top: simulation time on a logarithmic scale; the arrow marks the gap between Qiskit Aer and the other two backends at $n = 13$, and the Qulacs curve has no points from $n = 10$ to 12. Bottom: state fidelity against the ideal output; the curve of Qibo overlies that of Qiskit Aer.

dominate. Qibo starts twenty to fifty times faster, but its runtime grows with the matrix from $n = 3$. From about $n = 8$, updating the density matrix dominates all three backends. Its $4^n$ elements increase fourfold with each added qubit, while the QFT also gains gates, so runtime grows faster than the matrix alone. Qiskit Aer stays closest to the fourfold growth rate, while the other two backends grow more steeply, leaving it about twenty-five times faster at $n = 13$. The ratio to Qulacs compares different noisy operations, however, since on each CNOT Qulacs applies a one-qubit channel to the target, whereas Qiskit Aer and Qibo apply a two-qubit channel. The sweep stops at $n = 13$, where a repetition of either slower backend already exceeds one hour, although a single density matrix would fit in the platform's memory up to $n = 16$.

The fidelity panel reflects the gate count. The QFT gains gates quadratically with $n$, so the curves bend downwards, and a loss of $p/2$ of the fidelity at each one-qubit channel and of $3p/4$ at each two-qubit channel reproduces them to within $0.02$ at every size. Qibo closely matches Qiskit Aer because both apply to the same gates the same noise channel, which, with probability $p$, replaces the state of the gate's qubits with the maximally mixed state [81, 86]. The noise model of Qulacs differs from theirs in two ways that act in opposite directions. Its parameter is the probability of a Pauli error, $X$, $Y$ or $Z$ each with probability $p/3$ [87], which gives a channel stronger by a factor of $4/3$ on a single qubit at the same $p$ (Section 2.4.3), so each one-qubit gate lowers its fidelity more than in the other two backends. On each CNOT gate, however, Qulacs applies this one-qubit channel only to the target qubit, whereas the other two backends apply a two-qubit channel, so each CNOT affects its fidelity less. These two effects partly cancel, and their net result leaves Qulacs just slightly below the other two backends.

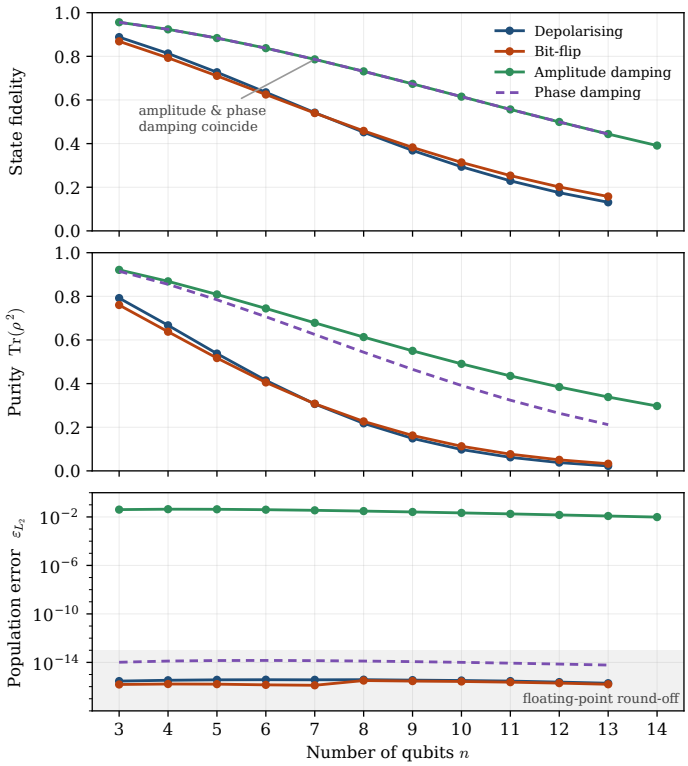



**Figure 5.10:** Accuracy of the QFT under four noise channels at $p = 0.01$ against the number of qubits $n$, from $n = 3$ to $13$ and to $14$ for amplitude damping, computed on Qiskit Aer alone so that every difference between the curves comes from the channel. The panels show the state fidelity (top), the purity $\mathrm{Tr}(\rho^2)$ of the same runs (middle) and the population error $\varepsilon_{L_2}$ on a logarithmic scale (bottom), where the shaded band marks errors below $10^{-13}$, at the level of floating-point round-off.

Comparing noisy simulators also requires knowing what each accuracy metric can detect. Figure 5.10 therefore compares the four noise channels on the same runs against the ideal output of the QFT for the all-zero input, $|+\rangle^{\otimes n}$, where each outcome has probability $2^{-n}$ (Section 4.2.1). The three metrics tell the channels apart to different degrees. Fidelity sorts them into two pairs, amplitude damping with phase damping, and bit-flip with depolarising noise, without separating the channels within a pair, since both damping channels shrink the coherence of a qubit by the same factor, and both Pauli channels also affect qubits remaining in $|0\rangle$ (Section 2.4.3).

Purity distinguishes between the damping channels, as amplitude damping drives each qubit towards the pure state $|0\rangle$, while phase damping only reduces coherence; thus, higher purity does not necessarily indicate a more accurate state. In these runs, only amplitude damping produces a population error above round-off, reflecting its bias towards $|0\rangle$. On the uniform ideal output of this experiment, the output probabilities of the other three channels match the ideal values to within round-off error even as the fidelity decreases, so their effect appears only in the coherences. In cost, Qiskit Aer is the fastest of the three backends above about $n = 8$, where the density matrix, rather than the fixed costs of Section 4.3.2, sets the time, with a margin that widens up to $n = 13$. This comparison is between the channels, as each backend defines them at

the same nominal $p$, which differ for QULACS, as its fidelity offset in Figure 5.9 shows.

## 5.7 Beyond State-Vector Simulation

All preceding results, except for the density-matrix runs in Section 5.6, employ dense state-vector simulation, which scales as $\mathcal{O}(2^n)$ in memory regardless of circuit structure. Stabiliser and tensor-network simulation are reductions of quantum dynamics, each applicable only when the circuit satisfies its structural assumption (Sections 3.1.3 and 3.1.4), and their runs differ in what they return, since the CUDA-Q runs sample $1024$ shots of the output, whereas the Clifford backend of QIBO builds the tableau without sampling and QIBOTN contracts the full state.

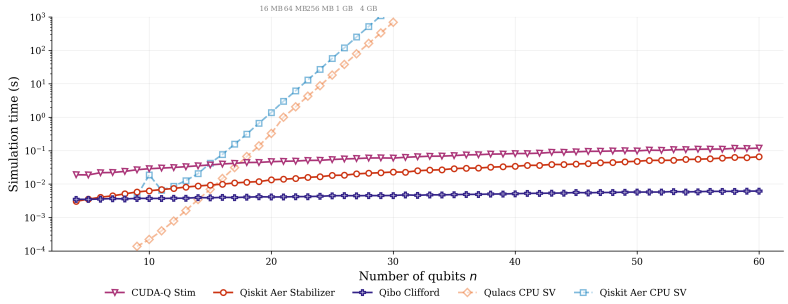



**Figure 5.11:** Simulation time for the Clifford circuit of Table 4.1, run by the stabiliser backends and by the state-vector references. QISKIT AER STABILIZER and QIBO Clifford stay below $0.1$ s up to $n = 60$, and CUDA-Q STIM exceeds it from $n = 51$; the single-thread state-vector references end at $n = 30$, where the single-thread sweeps of this dissertation stop. CUDA-Q STIM and QIBO Clifford ran on the host processors of Deucalion A100-40 nodes, and QISKIT AER STABILIZER and the state-vector references ran single-threaded on the Deucalion x86 partition. Each stabiliser point is the fastest of five repetitions.

**Stabiliser simulation.** Since the stabiliser tableau tracks $\mathcal{O}(n^2)$ bits in place of $2^n$ amplitudes (Section 3.1.4), the shape of Figure 5.11 follows from the cost of one gate on each representation. On a state vector, every gate is one pass over the $2^n$ amplitudes, so the reference times double with every qubit, and a state of $60$ qubits would need $2^{60} \times 16$ bytes, $16$ EiB. On a tableau, a gate touches $\mathcal{O}(n)$ bits, and a circuit of fixed depth has a number of gates proportional to $n$, so the whole circuit costs $\mathcal{O}(n^2)$ operations, which at $n = 60$ is still below the fixed cost of a run.

The stabiliser lines therefore stay almost flat where the references climb exponentially, and their ordering reflects the overhead of each framework around the tableau, since the tableau work itself is negligible at these sizes. CUDA-Q STIM, which runs the STIM library on the host processor, incurs the dispatch and just-in-time compilation of the CUDA-Q framework in every repetition, and that overhead places it above QISKIT AER STABILIZER and QIBO Clifford. The benefit is circuit-bound, since non-Clifford gates break the stabiliser structure and the cost then grows exponentially with their number (Section 3.1.4).

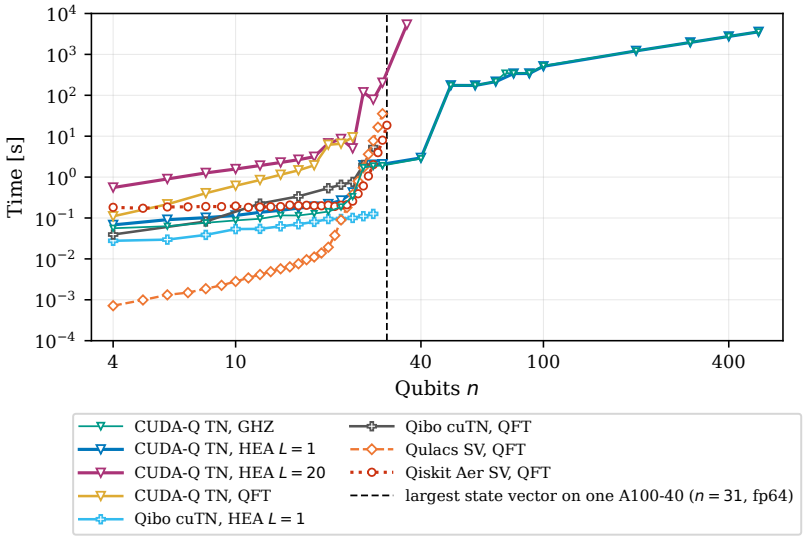



**Figure 5.12:** Tensor-network simulation (CUDA-Q and Qibo, both through cuTensorNet on Deucalion A100-40 GPUs) against the state-vector QFT on one A100-40 (Qulacs and Qiskit Aer, Section 5.1). Each point is the median of the completed repetitions at that size, and each series ends at the last size with a completed repetition. Repetitions that exceeded the time limit of Table 4.4 are not drawn; for Greenberger–Horne–Zeilinger (GHZ) and the HEA with $L = 1$ a repetition at $n = 75$ exceeded it while another completed, and every repetition from $n = 600$ on exceeded it; for the HEA with $L = 20$, repetitions at $n = 30$ and $36$ exceeded it while another completed at each size; for the QFT every repetition at $n = 26$ exceeded it. TN, cuTN and SV stand for tensor network, cuTensorNet and state vector, and $L$ for the number of HEA layers. The dashed vertical line marks the largest state vector that fits one A100-40 in double precision ($n = 31$).

**Tensor-network simulation.** The cost of tensor-network simulation depends on the entanglement generated by the circuit, the network structure and the contraction order (Section 3.1.3). Each CNOT crossing a cut, the only two-qubit gate in the GHZ and HEA circuits, can at most double the Schmidt rank[3]. Additional entangling layers can therefore increase the bond dimensions needed to represent the state exactly, raising the simulation cost.

Figure 5.12 includes GHZ and the HEA of Section 4.2.2 with $L$ CNOT rings in place of eight. The GHZ circuit prepares $(|0\cdots0\rangle + |1\cdots1\rangle)/\sqrt{2}$ with a Hadamard followed by CNOTs, giving Schmidt rank two across any non-trivial bipartition. A single HEA ring also keeps the rank bounded independently of $n$, at most four across each cut of the qubit chain. With CUDA-Q, both circuits completed runs at every tested size up to $n = 500$, where runtime reaches about an hour; every repetition at larger sizes exceeded the time limit (Table 4.4). Increasing the HEA depth reduces this reach. With twenty layers, $n = 36$ is the largest size with a completed repetition, and other repetitions at that size timed out.

For the QFT, tensor-network simulation is slower than the faster state-vector reference at every measured size. Although the all-zero input produces a product state (Section 4.2.1), this simple output does not guarantee a cheap contraction, since the all-to-all gate network can still produce large intermediate tensors,

[3]The Schmidt rank of a pure state across a bipartition is the number of non-zero terms in its Schmidt decomposition, $|\psi\rangle = \sum_i \lambda_i |a_i\rangle |b_i\rangle$, with orthonormal $\{|a_i\rangle\}$ and $\{|b_i\rangle\}$ [12]. It is one for a product state. In the decomposition of Vidal, on which matrix-product-state simulation builds, the bond dimension at a cut equals the Schmidt rank across it [33].

depending on the contraction order. Path search and, for CUDA-Q, sampling add to the timed cost. CUDA-Q completed runs only up to $n = 24$, with every repetition at $n = 26$ timing out, whereas a state vector fits on one A100-40 up to $n = 31$. QIBO's cuTensorNet path is faster than CUDA-Q's at all shared sizes on both the QFT and HEA, but its reach is limited by memory because it returns the full $2^n$-amplitude state. Its last completed runs are at $n = 28$; at $n = 30$, the intermediate tensors exceed GPU memory (Figure 5.12).
The stabiliser backends of QISKIT AER and QIBO run the Clifford circuit up to $60$ qubits, the largest size of Figure 5.11, in under a tenth of a second, and tensor-network simulation reaches $500$ qubits on the low-entanglement circuits with sampled output. Neither method improves on state-vector simulation for the QFT in the configurations tested, since the circuit is non-Clifford and its tensor-network contraction is slower at every measured size. Dense state-vector simulation therefore reaches the largest sizes for this circuit among the evaluated implementations, limited by device memory.

## 5.8 Simulator-Selection Framework

The findings of this chapter are synthesised into a two-level selection framework, applied sequentially. The first level determines the simulation method based on the model, circuit structure and required output (Table 5.9). Only the dense state vector proceeds to the second level, which selects the backend according to the execution regime, once the qubit count establishes which regimes can accommodate the state (Table 5.10). Figure 5.13, at the end of the chapter, presents the principal recommendations from both tables as a decision sequence.

**Table 5.9:** First level of the selection framework. Circuit structure or simulation objective determines the paradigm before any backend comparison becomes meaningful. ✓ recommended; ○ viable or conditionally useful; ✗ not recommended for this use case.

| **Circuit structure or objective** | Stabiliser | Tensor network | Dense state vector | Density matrix |
|---|---|---|---|---|
| Clifford-only | ✓ | ○ | ○ | ✗ |
| Low-bond-dimension circuits, e.g. GHZ and shallow HEA | ○ | ✓ | ○ | ✗ |
| Growing local entanglement, e.g. deeper HEA | ✗ | ○ | ○ | ✗ |
| Densely connected non-Clifford, e.g. QFT | ✗ | ✗* | ✓ | ✗ |
| Exact general noisy evolution | ✗ | ✗ | ✗ | ✓ |

*Exact contraction in the implementations tested, slower than the state vector at every size measured (Section 5.7).

The first level begins with the output that the task requires. Tasks that need exact amplitudes or probabilities call for strong simulation, whereas tasks that need only measurement samples use weak simulation (Section 2.2.5).

Table 5.9 assigns the simulation method based on the model and circuit, and the measurements in this chapter establish the limits of each method. Representing noisy evolution with a density matrix requires $4^n$ elements, compared with $2^n$ for a state vector, roughly halving the number of qubits that fit in the same memory. For Clifford circuits, the output type determines the appropriate stabiliser backend, since Qiskit Aer Stabilizer returns $1024$ samples, whereas Qibo Clifford provides the tableau. For low-entanglement circuits, the tensor network is suitable when the task does not need the full state, since the sampled output of CUDA-Q reached $n = 500$, whereas the contraction of qibotn into the full state hit the memory limit at $n = 30$, before the state vector does; the recommendation covers the two outputs evaluated, samples and the full state. The applicability of this method diminishes as additional layers increase entanglement, although its reach still surpasses the largest state that a single GPU node can hold, so it remains an option for deeper circuits (Section 5.7). For the QFT, however, exact contraction was slower than the state vector at all measured sizes, so the QFT proceeds to the dense state vector.

**Table 5.10:** Second level of the selection framework, for dense state-vector simulation on the evaluated platforms. ✓ recommended; ○ viable or competitive, or the fastest only for some circuits or sizes; ✗ not recommended or not available in the builds used.

| **Execution regime** | QuLacs | mpiQuLacs | QuEST | Qiskit Aer | Qibo | CUDA-Q |
|---|---|---|---|---|---|---|
| Single-thread CPU | ✓ | ✗ | ○ | ○ | ○ | ✗ |
| Full node (128 threads) | ○ | ✗ | ○ | ✓ | ✗ | ✗ |
| Multi-node CPU (QFT) | ✗ | ✓* | ✗ | ✓* | ✗ | ✗ |
| Single GPU† | ○ | ✗ | ○ | ✓ | ✗ | ✓ |
| Multi-GPU, one node (NVLink)‡ | ✗ | ✗ | ○ | ○ | ✗ | ✓ |
| Multi-GPU, across nodes (InfiniBand)§ | ✗ | ✗ | ○ | ✓ | ✗ | ✓ |

*Qiskit Aer with $128$ MPI ranks per node, the fastest at every node count measured, up to $16$ nodes; mpiQuLacs on $32$ and $64$ nodes and at $n = 34$ (Section 5.3). †The order depends on the size, the circuit and whether the state is copied to host memory; QuEST is the second fastest up to $n = 16$ and the only backend measured on the Clifford circuit at $n = 31$ (Sections 5.1 and 5.4.2). ‡Qiskit Aer with UCX's CUDA IPC transport; QuEST gains the most from the four GPUs of a node but remains the slowest (Section 5.4.2). §Under the default UCX settings of Deucalion, CUDA-Q and Qiskit Aer end within $1.3$ times of each other from $n = 32$, CUDA-Q ahead on eight and Qiskit Aer on sixteen GPUs, with QuEST behind both (Section 5.4.2). The Qibo column refers to its `qibojit` backend.

The second level depends on the number of qubits, since each regime's memory capacity limits the size of the state vector it can store. In double precision, one A100-40 GPU supports up to $n = 31$ qubits, one A100-80 up to $n = 32$, a Deucalion CPU node up to $n = 33$ and four A100-80 GPUs in a single node up to $n = 34$, or $33$ for QuEST, whose exchange buffer doubles the memory each GPU needs. Consequently, the state must be distributed across nodes from $n = 34$ on the CPU and A100-40 nodes, and on A100-80 nodes from $n = 35$ for Qiskit Aer and CUDA-Q, or from $n = 34$ for QuEST.

Within these memory capacities, the hardware determines the backend selection (Table 5.10), and the measurements add what the table cannot show. On CPUs, distributing the computation reduces runtime even when a single node can hold the state, provided each backend uses its faster layout, one rank per node for mpiQuLacs and QuEST and $128$ ranks per node for Qiskit Aer (Section 5.3). On GPUs, one node

gave the fastest time at every size it holds, with CUDA-Q about four times faster than Qiskit Aer on four GPUs, and crossing nodes slowed all three backends under the default UCX settings of Deucalion.
Figure 5.13 should be read from the top, with the questions of Table 5.9 forming the first level and those of Table 5.10 the second. The first question of the second level asks whether the run executes on GPUs, a choice left to the user, since at small sizes a backend on one CPU thread can finish before the same backend on one GPU, whose fixed cost per gate is higher (Section 5.1). The single-GPU boxes rest on runs that copy the state to host memory, as computing the fidelity against a reference requires (Section 4.4.3), and they stop at $n = 30$, since at $n = 31$ only QuEST provides results on all three circuits, Qiskit Aer on the QFT and the HEA, and CUDA-Q on the QFT. The one-GPU-node box rests on runs that retain the state on the GPUs, which suffices when only samples or a single probability are needed (Section 5.4.2). The multi-node and multi-GPU boxes pertain exclusively to the QFT and end at the largest sizes measured, $n = 34$ on CPU nodes and $n = 35$ on GPU nodes, which QuEST, holding one qubit less per GPU, reaches only on sixteen GPUs; larger sizes lie outside the measurements. These conclusions are specific to the circuits, builds and platforms evaluated, as other studies indicate that the performance ranking varies with the workload [30].

## 5.9 Summary

This chapter compared the state-vector backends of Table 4.3 on the QFT, the HEA and a Clifford circuit from one thread to a full node and one GPU, and on the QFT up to $64$ CPU nodes and sixteen GPUs, and no backend led in every regime. Fixed costs decided the order at small sizes, gate fusion and memory bandwidth at large sizes on one node or one GPU, and across nodes the layout of the distributed state and the link that the exchanges cross. Beyond the dense state vector, the density matrix halved the number of qubits that a given memory holds, and the stabiliser and tensor-network methods ran the circuits suited to them at $60$ and $500$ qubits, while the exact contraction of the QFT remained slower than the state vector. The selection framework of Section 5.8 arranges these findings so that the circuit and the required output determine the method, and the hardware and the state size the backend.

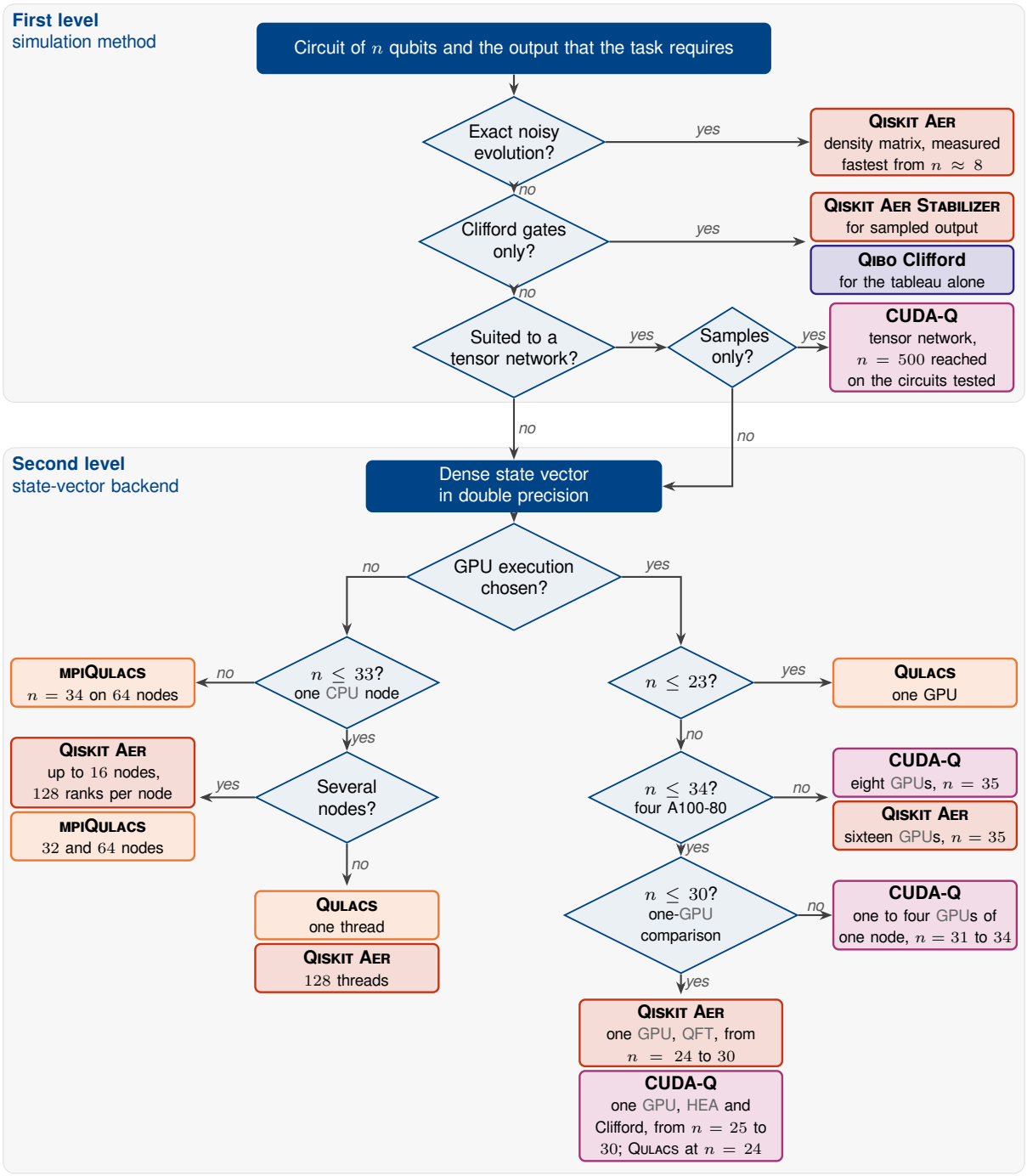



**Figure 5.13:** Two-level selection of a simulation method and a state-vector backend, which arranges the main recommendations of Tables 5.9 and 5.10 as a sequence of decisions, in a layout adapted from Kumaresan et al. [66]. Each box names the backend that was fastest in that regime, in its colour in this chapter, and stacked boxes divide one answer by output, circuit or resources. The single-thread, full-node and single-GPU boxes rest on runs up to $n = 30$ and the density-matrix box on runs up to $n = 13$. The multi-node and multi-GPU boxes rest on the QFT alone and on the largest sizes measured, $n = 34$ on CPU nodes and $n = 35$ on GPU nodes, which QuEST, holding one qubit less per GPU, reaches only on sixteen GPUs, and larger sizes lie outside the measurements.

# 6

# Conclusions and Future Work

## Contents


6.1 Conclusions . . . . . 80

6.2 Limitations . . . . . 81

6.3 Future Work . . . . . 82

This chapter summarises the findings of the dissertation and the results behind them, states their limitations and outlines future work.

## 6.1 Conclusions

This dissertation compared six state-vector backends from one CPU thread to $64$ nodes and sixteen GPUs, thereby addressing the gap identified in Section 3.6, where previous studies did not employ a consistent set of simulators across these regimes. The observed rankings are consistent with the performance models and with the cache-aware roofline and top-down profiles, which were collected on one core of the i9-13900KS platform. The results indicate that no single backend consistently outperforms the others across all circuits and execution regimes.

On a single thread, QULACS was at or near the front on all three circuits, combining low fixed costs with efficient native kernels. On the full $128$-thread node, QISKIT AER demonstrated superior performance at high qubit counts, achieving a speedup of $61$ over a single thread on the $30$-qubit QFT. Its fusion reduced the number of passes over the state, while the unfused backends stopped benefiting from additional cores, consistent with the memory-bandwidth limitation discussed in Section 5.2. Across CPU nodes, QISKIT AER with $128$ ranks per node was the fastest at $n = 30$ at every node count measured, up to $16$ nodes. MPIQULACS was the fastest of the backends measured on $32$ and $64$ nodes, running $53$ times faster on $64$ nodes than on one. It was also the fastest at $n = 34$, where the weak-scaling series exceeds the memory of one node. On one A100, QULACS led up to about $n = 23$ through its low fixed costs, and at larger sizes QISKIT AER led on the QFT and CUDA-Q on the HEA and Clifford circuit, in runs whose times for these two backends included a copy of the state to host memory. With the state left on the GPUs, CUDA-Q was the fastest of the three distributed backends within one node at every measured size, and QUEST the slowest, holding one qubit less per GPU because of its exchange buffer. Under the default UCX settings of Deucalion, crossing nodes slowed all three, so one node was fastest whenever it held the state, and beyond it, at $n = 35$, the lead of CUDA-Q over QISKIT AER narrowed on eight and reversed on sixteen GPUs.

The roofline ablation isolated vectorisation and gate fusion within QULACS on one core. Vectorisation raised attained performance at a fixed arithmetic intensity, while fusion increased intensity by merging gates. Fusion gave speedups of about $1.7$ with scalar code and $3.2$ with AVX2, exceeding the corresponding gains from vectorisation, about $1.06$ without fusion and $2.0$ with it. Each optimisation also made the other more effective.

Specialised simulation methods achieved larger problem sizes than dense state-vector simulation for circuits whose structure they could exploit. The stabiliser backends of QISKIT AER and QIBO executed the Clifford circuit in under a tenth of a second for up to $60$ qubits. Tensor-network simulation scaled to $500$ qubits for GHZ and a single-layer HEA with sampled output, whereas the contraction into the full state was limited by GPU memory at $n = 30$. Neither method provided an advantage for the QFT in the exact

form tested, since stabiliser simulation was not applicable and tensor-network contraction was slower at all measured sizes. Dense state-vector simulation therefore reached furthest on this circuit among the tested implementations and configurations.

Density-matrix storage increases fourfold per added qubit, halving the qubit capacity compared to a state vector at the same precision. Among the three backends evaluated, Qiskit Aer was the fastest from approximately $n = 8$, while fixed costs dominated at smaller sizes. The comparison also showed the need to match noise definitions, since Qulacs defines the depolarising channel differently at the same nominal rate. For the QFT under the four channels at the tested rate, fidelity alone did not distinguish all channels. Purity separated the two damping channels, while population error isolated amplitude damping, so the three metrics are needed together.

These findings support the two-level selection framework of Section 5.8. Circuit structure and the required output guide the choice of simulation method; state size and execution regime then guide the choice of backend. Within the tested configurations, the main performance differences reflect fixed execution costs, gate fusion, memory bandwidth and distributed-state layout.

## 6.2 Limitations

The results of this dissertation hold within the conditions under which they were measured, and three limitations constrain their generalisation. The first concerns the platforms and timing scopes. The main state-vector comparisons ran on Deucalion, while the roofline, density-matrix and top-down measurements ran on the i9-13900KS platform, where the required hardware counters were available. Therefore, times from the two platforms are not compared directly. The roofline analysis covers one backend on one core and supports the interpretation of the Deucalion rankings without quantifying the underlying mechanisms on Deucalion itself. Timed calls also differ in whether they include allocation, compilation or copies to host memory, so the comparisons concern the execution interfaces rather than isolated gate kernels. The thread sweep of Section 5.2, in turn, leaves thread placement to the OpenMP runtime (Table 4.4), so the thread counts at which its curves flatten cannot be attributed to a particular socket or NUMA domain.

The second concerns distributed execution. Under Deucalion's default inter-node UCX settings, exchanges between GPU nodes slowed all three backends. Transport tuning could reduce this cost and change their relative performance, but the conclusions apply to the configurations documented in Chapter 4. On CPU nodes, the lead of Qiskit Aer depended on running $128$ ranks per node, one per core, because its distributed engine applies each rank's gates with a single thread. This layout was measured up to $16$ nodes, so beyond that count the comparison includes only mpiQulacs, QuEST and Qiskit Aer with one rank per node.

The third concerns the workloads. The selection framework is based on three circuit families, while the four-channel noise comparison uses only Qiskit Aer, the QFT and one noise rate, up to $13$ qubits, or $14$ for amplitude damping. Other circuits, including those that combine the tested structures, may produce

different rankings. The conclusions about the accuracy metrics likewise remain specific to the circuit and noise rate measured.

## 6.3 Future Work

The limitations discussed above indicate several avenues for future research. Validated GPU hardware counters would enable a roofline analysis of the A100 runs, testing whether the fused kernels of Qiskit Aer and CUDA-Q are limited by memory bandwidth or by computation. This would complement the CPU analysis of Qulacs. Moreover, tuning the communication between GPU nodes would establish how much of the measured overhead can be reduced, while extending the Qiskit Aer layout to $32$ and $64$ CPU nodes would complete the comparison with mpiQulacs and QuEST.

The noise study offers the most immediate extensions. For instance, trajectory sampling, which stores a state vector for each trajectory, could be compared with density-matrix propagation at specified accuracy targets and tested beyond the memory limit of the density matrix. Furthermore, the channel comparison could cover more backends, a broader range of noise strengths and additional variational circuit types.

Broader workloads would also provide a way to test the selection framework beyond the three circuit families studied here. For instance, exploring ansätze with different connectivity and depth, as well as circuits like the quantum approximate optimisation algorithm or arithmetic subroutines from Shor's algorithm, would reveal whether the performance rankings hold for other gate structures. Running these tests on the A64FX partition of Deucalion and on consumer GPUs would further show whether these rankings are consistent across different hardware architectures.

In addition to expanding the workloads, including more simulators would enhance both the comparison and the optimisation study. For example, PennyLane Lightning distributes state vectors across GPUs using cuQuantum [88], enabling a direct comparison with CUDA-Q, since both use `cuStateVec`. Profiling their execution and exchanges could help distinguish the shared library's effects from backend scheduling. Similarly, qsim offers adjustable fusion blocks [89], which would extend the ablation study beyond two-qubit blocks.

Finally, an automated selector based on Section 5.8 could use the accumulated measurements to recommend a configuration for a given circuit and machine. Because the measured rankings change with the circuit, state size and hardware, such a tool could help researchers choose a simulator. A first prototype, encoding both levels of the framework and the results of Chapter 5 as rules, was developed alongside this dissertation [90].

# Bibliography


[1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134.

[2] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, Aug. 2018. [Online]. Available: https://doi.org/10.22331/q-2018-08-06-79

[3] Google Quantum AI and Collaborators, “Quantum error correction below the surface code threshold,” *Nature*, vol. 638, pp. 920–926, 2025.

[4] J. Preskill, “Beyond NISQ: The megaquop machine,” *ACM Transactions on Quantum Computing*, vol. 6, no. 3, pp. 1–7, 2025.

[5] EuroHPC Joint Undertaking, “Our quantum computers,” https://www.eurohpc-ju.europa.eu/eurohpc-quantum-computers/our-quantum-computers_en, 2026, accessed 2026-09-28.

[6] A. Holmes, S. Johri, G. G. Guerreschi, J. S. Clarke, and A. Y. Matsuura, “Impact of qubit connectivity on quantum algorithm performance,” *Quantum Science and Technology*, vol. 5, no. 2, p. 025009, 2020.

[7] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Physical Review A*, vol. 86, p. 032324, 2012.

[8] I. M. Georgescu, S. Ashhab, and F. Nori, “Quantum simulation,” *Reviews of Modern Physics*, vol. 86, no. 1, pp. 153–185, Mar. 2014. [Online]. Available: https://doi.org/10.1103/revmodphys.86.153

[9] B. Fang, M. Y. Özkaya, A. Li, Ü. V. Çatalyürek, and S. Krishnamoorthy, “Efficient hierarchical state vector simulation of quantum circuits via acyclic graph partitioning,” in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2022, pp. 289–300, arXiv:2205.06973.

[10] T. Häner and D. S. Steiger, “0.5 petabyte simulation of a 45-qubit quantum circuit,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: https://doi.org/10.1145/3126908.3126947

[11] N. Quetschlich, L. Burgholzer, and R. Wille, “MQT Bench: Benchmarking software and design automation tools for quantum computing,” *Quantum*, vol. 7, p. 1062, 2023.

[12] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[13] Smite-Meister, “Bloch sphere,” https://commons.wikimedia.org/wiki/File:Bloch_sphere.svg, 2009, CC BY-SA 3.0.

[14] M. Van den Nest, “Simulating quantum computers with probabilistic methods,” *Quantum Information and Computation*, vol. 11, no. 9–10, pp. 784–812, 2011, arXiv:0911.1624.

[15] S. Hillmich, I. L. Markov, and R. Wille, “Just like the real thing: Fast weak simulation of quantum computation,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.

[16] E. Rieffel and W. Polak, *Quantum Computing: A Gentle Introduction*, 1st ed. The MIT Press, 2011.

[17] D. Aharonov, “A simple proof that Toffoli and Hadamard are quantum universal,” Jan. 2003. [Online]. Available: https://arxiv.org/abs/quant-ph/0301040

[18] S. Aaronson and D. Gottesman, “Improved simulation of stabilizer circuits,” *Physical Review A*, vol. 70, no. 5, p. 052328, Nov. 2004. [Online]. Available: https://doi.org/10.1103/physreva.70.052328

[19] D. Gottesman, “The Heisenberg representation of quantum computers,” in *Group22: Proceedings of the XXII International Colloquium on Group Theoretical Methods in Physics*. International Press, 1999, pp. 32–43, arXiv:quant-ph/9807006.

[20] S. Bravyi and D. Gosset, “Improved classical simulation of quantum circuits dominated by Clifford gates,” *Physical Review Letters*, vol. 116, no. 25, p. 250501, 2016.

[21] M. Kjaergaard, M. E. Schwartz, J. Braumüller, P. Krantz, J. I.-J. Wang, S. Gustavsson, and W. D. Oliver, “Superconducting qubits: Current state of play,” *Annual Review of Condensed Matter Physics*, vol. 11, pp. 369–395, 2020.

[22] A. Ransford *et al.*, “A 98-qubit trapped-ion quantum computer with all-to-all connectivity,” *Nature*, vol. 655, pp. 81–86, 2026.

[23] D. Bluvstein *et al.*, “A quantum processor based on coherent transport of entangled atom arrays,” *Nature*, vol. 604, pp. 451–456, 2022.

[24] S. J. Evered, D. Bluvstein, M. Kalinowski, S. Ebadi, T. Manovitz, H. Zhou, S. H. Li, A. A. Geim, T. T. Wang, N. Maskara, H. Levine, G. Semeghini, M. Greiner, V. Vuletić, and M. D. Lukin, “High-fidelity parallel entangling gates on a neutral-atom quantum computer,” *Nature*, vol. 622, pp. 268–272, 2023.

[25] A. Noiri, K. Takeda, T. Nakajima, T. Kobayashi, A. Sammak, G. Scappucci, and S. Tarucha, “Fast universal quantum gate above the fault-tolerance threshold in silicon,” *Nature*, vol. 601, pp. 338–342, 2022.

[26] PsiQuantum team, “A manufacturable platform for photonic quantum computing,” *Nature*, vol. 641, pp. 876–883, 2025.

[27] M. B. Plenio and P. L. Knight, “The quantum-jump approach to dissipative dynamics in quantum optics,” *Reviews of Modern Physics*, vol. 70, no. 1, pp. 101–144, 1998.

[28] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik, “qHiPSTER: The quantum high performance software testing environment,” *CoRR*, vol. abs/1601.07195, 2016.

[29] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa, Y.-L. L. Fang, Y. Gao, J. Guan, J. Gunnels, A. Haidar, A. Hehn, M. Hohnerbach, M. Jones, T. Lubowe, D. Lyakh, S. Morino, P. Springer, S. Stanwyck, I. Terentyev, S. Varadhan, J. Wong, and T. Yamaguchi, “cuQuantum SDK: A high-performance library for accelerating quantum science,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1, 2023, pp. 1050–1061, arXiv:2308.01999.

[30] A. J. Gangapuram, A. M. Läuchli, and C. Hempel, “Benchmarking quantum computer simulation software packages: State vector simulators,” *SciPost Physics Core*, vol. 7, no. 4, p. 075, 2024.

[31] L. Burgholzer, H. Bauer, and R. Wille, “Hybrid Schrödinger-Feynman simulation of quantum circuits with decision diagrams,” in *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2021, pp. 199–206, arXiv:2105.07045.

[32] A. Pasquale, A. Papaluca, R. M. S. Farias, M. Robbiati, E. Pedicillo, and S. Carrazza, “Beyond full statevector simulation with Qibo,” 2024. [Online]. Available: https://arxiv.org/abs/2408.00384

[33] G. Vidal, “Efficient classical simulation of slightly entangled quantum computations,” *Physical Review Letters*, vol. 91, no. 14, p. 147902, Oct. 2003. [Online]. Available: http://dx.doi.org/10.1103/PhysRevLett.91.147902

[34] A. Leonteva, G. Masella, M. Outteryck, A. Piñeiro Orioli, and S. Whitlock, “Comparative benchmarking of utility-scale quantum emulators,” *ACM Transactions on Quantum Computing*, vol. 7, no. 2, pp. 1–29, 2026, arXiv:2504.14027.

[35] C. Gidney, “Stim: a fast stabilizer circuit simulator,” *Quantum*, vol. 5, p. 497, 2021.

[36] S. Bravyi, D. Browne, P. Calpin, E. Campbell, D. Gosset, and M. Howard, “Simulation of quantum circuits by low-rank stabilizer decompositions,” *Quantum*, vol. 3, p. 181, 2019.

[37] V. T. Hai, B. C. Doanh, L. V. T. Duong, P. H. Luan, and Y. Nakashima, “Qimax: Efficient quantum simulation via GPU-accelerated extended stabilizer formalism,” arXiv:2505.03307, 2025, version 2, January 2026.

[38] H. Qassim, H. Pashayan, and D. Gosset, “Improved upper bounds on the stabilizer rank of magic states,” *Quantum*, vol. 5, p. 606, Dec. 2021. [Online]. Available: http://dx.doi.org/10.22331/q-2021-12-20-606

[39] A. Zulehner and R. Wille, “Advanced simulation of quantum computations,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 5, pp. 848–859, 2019.

[40] J. Gray, “quimb: A Python package for quantum information and many-body calculations,” *Journal of Open Source Software*, vol. 3, no. 29, p. 819, 2018.

[41] A. H. Werner, D. Jaschke, P. Silvi, M. Kliesch, T. Calarco, J. Eisert, and S. Montangero, “Positive tensor network approach for simulating open quantum many-body systems,” *Physical Review Letters*, vol. 116, no. 23, p. 237201, 2016.

[42] S. V. Isakov, D. Kafri, O. Martin, C. Vollgraff Heidweiller, W. Mruczkiewicz, M. P. Harrigan, N. C. Rubin, R. Thomson, M. Broughton, K. Kissell, E. Peters, E. Gustafson, A. C. Y. Li, H. Lamm, G. Perdue, A. K. Ho, D. Strain, and S. Boixo, “Simulations of quantum circuits with approximate noise using qsim and Cirq,” arXiv:2111.02396, 2021.

[43] NVIDIA Corporation, “CUDA-Q documentation, version 0.14.0,” https://nvidia.github.io/cuda-quantum/0.14.0/, 2026, pages “Noisy Simulators”, “State Vector Simulators”, “CUDA-Q Python API” (`cudaq.Target`), “Installation from Source” (MPI) and “CUDA-Q Releases”. Accessed 2026-10-31.

[44] H. De Raedt, J. Kraus, A. Herten, V. Mehta, M. Bode, M. Hrywniak, K. Michielsen, and T. Lippert, “Universal quantum computer simulation of 50 qubits on Europe’s first exascale supercomputer harnessing its heterogeneous CPU–GPU architecture,” *Future Generation Computer Systems*, vol. 183, p. 108592, 2026.

[45] H. De Raedt, F. Jin, D. Willsch, M. Willsch, N. Yoshioka, N. Ito, S. Yuan, and K. Michielsen, “Massively parallel quantum computer simulator, eleven years later,” *Computer Physics Communications*, vol. 237, pp. 47–61, 2019.

[46] G. Pratipat, “A controlled study of memory hierarchy transitions in quantum circuit simulation on Apple M4 Pro unified memory architecture,” arXiv:2605.08792, 2026.

[47] H. Bian, J. Huang, R. Dong, Y. Guo, and X. Wang, “HpQC: A new efficient quantum computing simulator,” in *Algorithms and Architectures for Parallel Processing (ICA3PP 2020)*, ser. Lecture Notes in Computer Science, vol. 12453. Springer, 2020, pp. 111–125.

[48] H. Bian, J. Huang, J. Tang, R. Dong, L. Wu, and X. Wang, “PAS: A new powerful and simple quantum computing simulator,” *Software: Practice and Experience*, vol. 53, no. 1, pp. 142–159, 2023, first published 29 October 2021.

[49] A. Rezaei, L. Jaulmes, M. Bahna, O. T. Brown, and A. Barbalace, “Low-level and NUMA-aware optimization for high-performance quantum simulation,” 2025. [Online]. Available: https://arxiv.org/abs/2506.09198

[50] A. Cicero, M. A. Maleki, M. W. Azhar, A. F. Kockum, and P. Trancoso, “Simulation of quantum computers: Review and acceleration opportunities,” *ACM Transactions on Quantum Computing*, vol. 7, no. 1, pp. 3:1–3:35, 2026, arXiv:2410.12660.

[51] Qiskit Development Team, “Qiskit Aer, version 0.17.2, source code,” https://github.com/Qiskit/qiskit-aer, chunked (cache-blocking) executor, `src/simulators/parallel_state_executor.hpp` and `src/simulators/statevector/statevector_executor.hpp`; AVX2 kernels, `src/simulators/statevector/qv_avx2.cpp`. Accessed 2026-10-06.

[52] C.-C. Wang, Y.-C. Lin, Y.-J. Wang, C.-H. Tu, and S.-H. Hung, “Queen: A quick, scalable, and comprehensive quantum circuit simulation for supercomputing,” arXiv:2406.14084, 2024.

[53] L. Xu, E. H.-M. Sha, Y. Song, and Q. Zhuge, “QMin: Quantum circuit minimization via gate fusions for efficient state vector simulation,” *Quantum Information Processing*, vol. 25, no. 1, p. 6, 2026.

[54] R. Shi, G. Schieffer, P.-H. Lin, M. Gokhale, A. Herten, and I. Peng, “High-performance vector-length agnostic quantum circuit simulations on ARM processors,” in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2026, arXiv:2602.09604.

[55] NVIDIA, “NVIDIA A100 Tensor Core GPU datasheet,” https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf, 2021, accessed 2026-10-31.

[56] W. M. Brown, A. Ramesh, T. Lubinski, T. Nguyen, and D. E. Bernal Neira, “Multi-GPU quantum circuit simulation and the impact of network performance,” *Computer Physics Communications*, vol. 324, p. 110126, Jul. 2026. [Online]. Available: http://dx.doi.org/10.1016/j.cpc.2026.110126

[57] J. Doi and H. Horii, “Cache blocking technique to large scale quantum computing simulation on supercomputers,” in *2020 IEEE International Conference on Quantum Computing and Engineering*

*(QCE)*. IEEE, Oct. 2020, pp. 212–222. [Online]. Available: http://dx.doi.org/10.1109/QCE49297.2020.00035

[58] J. Adamski, J. P. Richings, and O. T. Brown, "Energy efficiency of quantum statevector simulation at scale," in *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. ACM, Nov. 2023, pp. 1871–1875. [Online]. Available: http://dx.doi.org/10.1145/3624062.3624270

[59] Qulacs developers, "Qulacs, version 0.6.11, source code," https://github.com/qulacs/qulacs, MPI build described in `README_MPI.md`, present from version 0.6.0. Accessed 2026-10-06.

[60] S. Imamura, M. Yamazaki, T. Honda, A. Kasagi, A. Tabuchi, H. Nakao, N. Fukumoto, and K. Nakashima, "mpiQulacs: A distributed quantum computer simulator for A64FX-based cluster systems," 2022. [Online]. Available: https://arxiv.org/abs/2203.16044

[61] QuEST-Kit, "QuEST: the Quantum Exact Simulation Toolkit, version 4.2.0, source code," https://github.com/QuEST-Kit/QuEST, state-vector allocation in `quest/src/api/qureg.cpp` and `quest/src/cpu/cpu_config.cpp` (`cpu_allocNumaArray`); GPU exchanges in `quest/src/comm/comm_routines.cpp`; release notes of version 4.2.0. Accessed 2026-10-06.

[62] M. Vallero, P. Rech, and F. Vella, "State of practice: Evaluating GPU performance of state vector and tensor network methods," *Future Generation Computer Systems*, vol. 174, p. 107927, 2026.

[63] J. Cerrudo-Herrera, D. Talaván-Vega, P. Rodríguez-Oliver, A. Ziabat-Ziabat, and J. A. Rico-Gallego, "Benchmarking quantum computing statevector simulators on high-performance computing," *SIMULATION*, vol. 102, no. 7, pp. 493–507, 2026.

[64] D. Lee, E. Jang, S. Choi, J. An, C. Kim, and W. W. Ro, "PIMutation: Exploring the potential of real PIM architecture for quantum circuit simulation," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASP-DAC)*. ACM, 2025, pp. 483–490.

[65] J. Faj, I. Peng, J. Wahlgren, and S. Markidis, "Quantum computer simulations at warp speed: Assessing the impact of GPU acceleration: A case study with IBM Qiskit Aer, Nvidia Thrust & cuQuantum," in *2023 IEEE 19th International Conference on e-Science (e-Science)*, 2023, pp. 1–10, arXiv:2307.14860.

[66] P. Kumaresan, P. Muruganantham, L. Rajendran, and S. Sivasubramani, "GPU-accelerated quantum simulation: Empirical backend selection, gate fusion, and adaptive precision," 2026, arXiv:2604.03816.

[67] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, ser. AFIPS ’67 (Spring). ACM, 1967, pp. 483–485.

[68] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[69] A. H. Karp and H. P. Flatt, “Measuring parallel processor performance,” *Communications of the ACM*, vol. 33, no. 5, pp. 539–543, 1990.

[70] A. Ilić, F. Pratas, and L. Sousa, “Cache-aware roofline model: Upgrading the loft,” *IEEE Computer Architecture Letters*, vol. 13, no. 1, pp. 21–24, 2014.

[71] D. Marques, A. Ilić, Z. A. Matveev, and L. Sousa, “Application-driven cache-aware roofline model,” *Future Generation Computer Systems*, vol. 107, pp. 257–273, 2020.

[72] J. Morgado, L. Sousa, and A. Ilić, “CARM Tool: Cache-aware roofline model automatic benchmarking and application analysis,” in *2024 IEEE International Symposium on Workload Characterization (IISWC)*, 2024, pp. 68–81.

[73] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, “QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation,” *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 10:1–10:26, 2023.

[74] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, “Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets,” *Nature*, vol. 549, pp. 242–246, 2017.

[75] Minho Advanced Computing Centre (MACC), “Deucalion user guide,” Online documentation, 2024, hybrid EuroHPC system: 1632 Arm A64FX nodes (2.0 GHz, 48 cores, 32 GB) and 500 x86 nodes with dual AMD EPYC 7742. [Online]. Available: https://docs.macc.fccn.pt/

[76] Advanced Micro Devices, Inc., “AMD EPYC™ 7742 processor specifications,” Product specification page, 2019, 64 cores; eight DDR4-3200 channels; 204.8 GB/s memory bandwidth per socket. [Online]. Available: https://www.amd.com/en/support/downloads/drivers.html/processors/epyc/epyc-7002-series/amd-epyc-7742.html

[77] J.-S. Kim, A. McCaskey, B. Heim, M. Modani, S. Stanwyck, and T. Costa, “CUDA Quantum: The platform for integrated quantum-classical computing,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–4.

[78] Y. Suzuki, Y. Kawase, Y. Masumura, Y. Hiraga, M. Nakadai, J. Chen, K. M. Nakanishi, K. Mitarai, R. Imai, S. Tamiya, T. Yamamoto, T. Yan, T. Kawakubo, Y. O. Nakagawa, Y. Ibe, Y. Zhang, H. Yamashita, H. Yoshimura, A. Hayashi, and K. Fujii, "Qulacs: A fast and versatile quantum circuit simulator for research purpose," *Quantum*, vol. 5, p. 559, 2021.

[79] S. Efthymiou, S. Ramos-Calderer, C. Bravo-Prieto, A. Pérez-Salinas, D. García-Martín, A. Garcia-Saez, J. I. Latorre, and S. Carrazza, "Qibo: a framework for quantum simulation with hardware acceleration," *Quantum Science and Technology*, vol. 7, no. 1, p. 015018, 2022.

[80] S. Efthymiou, M. Lazzarin, A. Pasquale, and S. Carrazza, "Quantum simulation with just-in-time compilation," *Quantum*, vol. 6, p. 814, 2022.

[81] Qibo team, "Qibo documentation, version 0.3.5," https://qibo.science/qibo/stable/, sections "Using multiple GPUs" and `qibo.gates.DepolarizingChannel`. Accessed 2026-10-06.

[82] T. Jones, A. Brown, I. Bush, and S. C. Benjamin, "QuEST and high performance simulation of quantum computers," *Scientific Reports*, vol. 9, no. 1, p. 10736, Jul. 2019. [Online]. Available: https://doi.org/10.1038/s41598-019-47174-9

[83] QuEST Developers, "QuEST v4.2.0 release notes," https://github.com/QuEST-Kit/QuEST/releases/tag/v4.2.0, 2025, accessed 2026-10-23.

[84] NVIDIA Corporation, "CUDA-Q, version 0.14.0, source code," https://github.com/NVIDIA/cuda-quantum, 2026, `PythonLauncher::launchModule` in `runtime/cudaq/platform/default/python/QPU.cpp` specialises the kernel and creates a new JIT engine in every call. Accessed 2026-10-31.

[85] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 73:1–73:12.

[86] Qiskit Development Team, *Qiskit Aer documentation:* `depolarizing_error`, definition of the depolarizing channel. Accessed 2026-10-31. [Online]. Available: https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.noise.depolarizing_error.html

[87] Qulacs developers, "Qulacs documentation: Python advanced guide," http://docs.qulacs.org/en/latest/guide/2.0_python_advanced.html, definition of the noise gates. Accessed 2026-10-06.

[88] A. Asadi, A. Dusko, C.-Y. Park, V. Michaud-Rioux, I. Schoch, S. Shu, T. Vincent, and L. J. O'Riordan, "Hybrid quantum programming with PennyLane Lightning on HPC platforms," 2024. [Online]. Available: https://arxiv.org/abs/2403.02512

[89] Google Quantum AI, “qsim: Choosing hardware for your qsim simulation,” https://quantumai.google/qsim/choose_hw, 2026, documentation of qsimcirq 0.22.1, accessed 2026-10-23.

[90] M. Barros, “Quantum simulator selector: a prototype of the two-level selection framework,” https://madalenarb.github.io/quantum-simulator-selector/, 2026, source code at https://github.com/madalenarb/quantum-simulator-selector. Accessed 2026-10-23.

[91] A. Yasin, “A top-down method for performance analysis and counters architecture,” in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2014, pp. 35–44.

# A

# Supporting Evidence for the Results

This appendix presents the measurements behind the results of Chapter 5 and the explanations they support. Each section answers a question that the timing data alone leave open, so the appendix is read alongside the section of Chapter 5 it refers to rather than on its own.

Section A.1 reports the spread between repetitions for the state-vector, stabiliser and tensor-network series. Sections A.2 and A.3 then support the single-thread landscape of Section 5.1, the first by showing where each backend spends its time and the second by following the instructions through the core pipeline. Section A.4 justifies the process layout chosen for the multi-node runs of Section 5.3. Finally, Sections A.5 and A.6 support the multi-GPU runs of Section 5.4, by estimating when the GPUs are active and idle and by isolating the transport that Qiskit Aer uses between them. The profiling in these sections is diagnostic. It identifies which execution path dominates in each backend and why, but does not divide the simulation times of Figure 5.1 into phases.

## A.1 Variation Between Repetitions

Table A.1 shows how far apart the repetitions behind the results of Chapter 5 lie. The spread of a point is the difference between its slowest and fastest repetition, relative to the fastest, over all the repetitions behind that point. Since the median and the mean of a point lie between its slowest and fastest repetitions, the choice among the statistics of Table 4.4 therefore changes the value of a point by no more than its spread. For the stabiliser and tensor-network series, the table covers the qubit counts up to $n = 38$ and $n = 80$, respectively, for which the slowest repetition of each point is stored.

**Table A.1:** Spread between repetitions for each experiment group of Chapter 5, with the number of repetitions per job. The spread of a point is the difference between its slowest and fastest repetition, relative to the fastest, over all the repetitions behind that point, and the median and the largest spread are taken over the points of each group. Among the points included, no spread exceeds 7%, and the 128-thread series have the highest median.

| Group | Section | Repetitions per job | Median (%) | Largest (%) |
|---|---|---|---|---|
| One thread | 5.1 | 10 | 0.8 | 5.4 |
| 128 threads | 5.1 | 5 | 2.7 | 6.0 |
| One A100-40 GPU | 5.1 | 6 | 1.9 | 6.7 |
| Thread sweep | 5.2 | 10 | 1.4 | 4.4 |
| Multi-node | 5.3 | 10 | 1.0 | 4.5 |
| Multi-GPU, one node | 5.4.1 | 10 | 2.1 | 5.8 |
| Multi-GPU, across nodes | 5.4.2 | 10 | 1.9 | 5.9 |
| Roofline | 5.5 | 10 | 0.1 | 1.1 |
| Stabiliser | 5.7 | 5 | 1.1 | 4.4 |
| Tensor network | 5.7 | 5 | 1.4 | 5.9 |

## A.2 Single-Thread Execution-Path Profiling

Section 5.1 attributes the ordering of the single-thread runs at small $n$ to the fixed cost of issuing each gate, which is low for QULACS and high for QISKIT AER (Section 4.3.2). To locate that cost, the single-thread runs on Deucalion were profiled with `py-spy --native`, which samples the native call stack and classifies each sample by execution path.

Samples fall into four paths. The first is the simulator-specific state-vector kernel, the native gate-application code such as `qulacs_core`, `libQuEST` or AER's `controller_wrappers`. The second is runtime and orchestration, namely dispatch, synchronisation and thread-wait frames, and the third is allocation and import support, `malloc`/`free` and runtime setup, which is most visible at small $n$. The fourth is the generic NumPy array path (`_multiarray_umath`, reshapes and strided copies), native code outside any state-vector kernel. A lower non-kernel share therefore means that more of the sampled time has moved into the simulator's own gate-application routines, so the qubit count at which a backend becomes kernel-dominated marks where its fixed cost stops setting its time.

Figure A.1 tracks the non-kernel share of the two backends across that region. That share falls as $n$ grows, which is consistent with the fixed overheads of orchestration, allocation and import becoming less important relative to gate execution. For QULACS, the fixed cost dominates only below $n \approx 10$, where its profile becomes kernel-dominated, so even at small $n$ most of its time is spent applying gates. For QISKIT AER, in contrast, the fixed cost dominates up to $n = 12$, where nearly all of its sampled time still falls outside the kernel; its profile crosses into the kernel-dominated regime between $n = 12$ and $n = 16$, and the kernel exceeds $90\%$ of it only from $n = 18$, just below the QFT crossover of Figure 5.1. This fixed dispatch cost is therefore what places QISKIT AER behind at small $n$ in Section 5.1. Once both profiles are kernel-dominated, the ordering of the two backends is decided by their kernels, which is where the gate fusion of QISKIT AER takes effect. The same qualitative trend holds for the HEA and for Clifford $d = 10$.

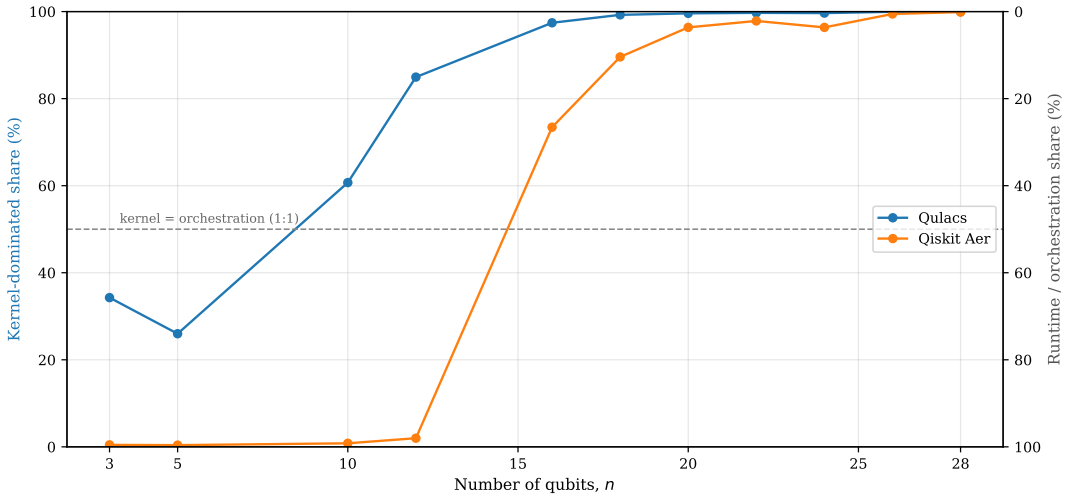



**Figure A.1:** QFT single-thread profile composition for QULACS and QISKIT AER on Deucalion, measured with `py-spy --native`. Each curve is the share of samples outside the state-vector kernel (runtime, orchestration, allocation and unresolved support), read on the right axis; the left axis gives the kernel share. The qubit counts span the region in which the QFT curves of the two backends cross in Figure 5.1.

Beyond that point, the kernel that each backend runs sets its time, and Table A.2 records the dominant path of each backend and how its hot routine shifts between the QFT and the HEA. The three backends with a native kernel differ in which routines that kernel runs, as QISKIT AER spends the QFT in diagonal and phase routines and the HEA in fused dense ones, in line with the fusion counts of Section 5.1. Default QIBO also spends most of its time in native code, although in the generic NumPy array path, which is the execution-backend effect that Section 5.1 gives for its offset in Figure 5.1.

**Table A.2:** Dominant single-thread execution path per backend on Deucalion, from `py-spy --native`, with the principal hot routine for QFT, HEA and Clifford $d = 10$. The first three backends run a native state-vector kernel; default QIBO runs a generic NumPy array path.

| Backend | Execution path | QFT hot routine | HEA hot routine | Clifford hot routine |
|---|---|---|---|---|
| QULACS | State-vector kernel (`qulacs_core`) | A few native gate kernels | Same path, over more kernels (rotations and ring) | Same few native kernels (same hot cluster) |
| QISKIT AER | State-vector kernel (`controller_wrappers`) | Diagonal/phase routines (`apply_diagonal_matrix`) | Fused dense (`apply_matrix_n`) | Fused dense, mostly 3-qubit (`apply_matrix_n`) |
| QUEST | State-vector kernel (`libQuEST`) | Controlled diagonal and dense | Balanced dense/diagonal mix | Controlled one-target diagonal, plus dense for $H$ |
| QIBO | NumPy array path (`_multiarray_umath`) | Tensor contraction, copy, reshape | Same NumPy path | Einsum contraction and iterator buffering |

## A.3 Top-Down Microarchitecture Analysis for the QFT

Section 5.1 draws two conclusions from the single-thread timings at large $n$ that the timings alone cannot verify, namely that the native kernels of QULACS, QUEST and QISKIT AER are sensitive to memory traffic, and that the memory writes of the NumPy execution backend of default QIBO contribute to its offset. This section tests both with hardware counters. Since Deucalion gives no access to them, the same single-thread

QFT runs were profiled on the i9-13900KS platform of the roofline study (Section 5.5) with the Top-down Microarchitecture Analysis (TMA) method [91] in Intel VTune 2026.0, and, as with the other measurements on that platform, the values are compared only with each other (Section 4.3).

TMA attributes each issued pipeline slot to one of four categories. *Retiring* slots commit useful work, *Front-End Bound* slots stall on instruction delivery, *Bad Speculation* slots are discarded after mispredicted branches and *Back-End Bound* slots stall on execution resources or memory. The last splits into *Core Bound* and *Memory Bound*, and VTune further estimates where the memory-bound slots stall, in the L3 cache, in DRAM or on stores, which is the split that separates the backends here. Figure A.2 shows the four categories at $n = 28$, where the $4$ GiB state vector exceeds every cache of the platform, and Table A.3 gives the values and the memory split at $n = 26$ and $n = 28$.

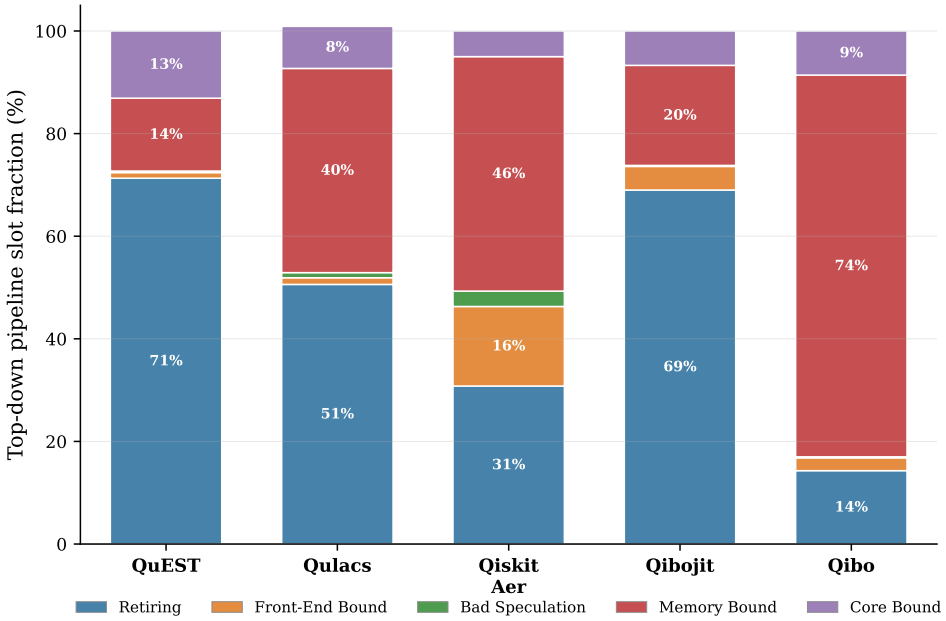



**Figure A.2:** Top-down Microarchitecture Analysis pipeline-slot fractions for the single-thread 28-qubit Quantum Fourier Transform on the i9-13900KS platform. Each stacked bar is one backend; the segments are the top-level TMA categories Retiring, Front-End Bound and Bad Speculation, plus Memory Bound and Core Bound, the two components of Back-End Bound. Heights are percentages of the available pipeline slots and sum to 100%.

The three native kernels show different levels and sources of memory-related stalls. Qiskit Aer has the largest Memory Bound share of the three, almost all of it in DRAM, so its core spends most of its stalled slots waiting for amplitudes to arrive from main memory, which is what streaming a state vector larger than every cache looks like in the counters. It also has the largest Front-End Bound share of all the backends, so a further part of its slots is lost before execution, waiting for instructions to be fetched and decoded. This is consistent with the fused path that Section 5.1 identifies behind its crossover with Qulacs, since a fused block executes more varied code per pass than a single gate, although the counters do not show whether fusion itself causes that cost. Qulacs, in turn, stalls on both the L3 cache and DRAM, so part of its traffic is

**Table A.3:** Top-down diagnostics for single-thread QFT on the i9-13900KS platform at $n = 26$ and $n = 28$. CPI is in cycles per retired instruction; all other columns are percentages of the available pipeline slots. *Memory Bound* and *Core Bound* are subcomponents of *Back-End Bound*; L3, DRAM and Store are VTune's estimates of the memory-bound stalls by source, computed separately from the top-level categories, so they need not add up to the *Memory* column.

| $n$ Backend | CPI cyc/instr. | Retiring % slots | Front-End % slots | Back-End % slots | Memory % slots | L3 % slots | DRAM % slots | Store % slots | Core % slots | Main signature |
|---|---|---|---|---|---|---|---|---|---|---|
| $n = 26$ | | | | | | | | | | |
| QuEST | 0.254 | 64.7 | 0.4 | 34.6 | 22.0 | 11.2 | 8.9 | 2.2 | 12.7 | High retiring; native state-vector path |
| Qulacs | 0.312 | 52.4 | 1.4 | 46.1 | 37.5 | 18.3 | 18.7 | 2.1 | 8.5 | Native kernel with stronger memory pressure |
| Qiskit Aer | 0.486 | 32.7 | 16.5 | 47.4 | 42.0 | 4.1 | 37.3 | 1.4 | 5.4 | Memory-bound plus front-end pressure |
| qibojit | 0.245 | 67.5 | 4.9 | 27.2 | 20.3 | 9.9 | 8.5 | 0.0 | 7.0 | JIT backend path; high retiring |
| Qibo | 1.030 | 16.8 | 5.7 | 76.5 | 68.5 | 6.4 | 19.7 | 39.3 | 8.1 | Store-bound NumPy array path |
| $n = 28$ | | | | | | | | | | |
| QuEST | 0.230 | 71.3 | 1.1 | 27.3 | 14.2 | 7.8 | 6.5 | 0.0 | 13.1 | High retiring; stable native path |
| Qulacs | 0.326 | 50.6 | 1.3 | 48.0 | 39.8 | 15.9 | 22.8 | 1.2 | 8.2 | Native kernel with increased DRAM pressure |
| Qiskit Aer | 0.518 | 30.8 | 15.5 | 50.7 | 45.7 | 5.2 | 39.2 | 0.7 | 5.0 | DRAM-bound; front-end cost persists |
| qibojit | 0.240 | 69.0 | 4.6 | 26.3 | 19.5 | 13.8 | 6.2 | 0.0 | 6.7 | Stable JIT profile across $n$ |
| Qibo | 1.264 | 14.3 | 2.5 | 83.0 | 74.4 | 5.7 | 13.7 | 50.3 | 8.6 | Store-bound worsens; CPI exceeds 1.2 |

still served from the last-level cache, and retires about half of its slots. QuEST retires the most of the three and stalls the least on memory, so its core does useful work in a larger share of its cycles.

All three therefore remain sensitive to memory traffic, as Section 5.1 treats them. CPI alone does not determine runtime, however, since a higher CPI can be offset by fewer retired instructions per gate. These profiles thus help interpret execution behaviour but do not explain the Deucalion ranking directly, and the Qulacs profile at $n = 26$ also comes from a build without the AVX2 kernels of the Deucalion runs.

Comparing default Qibo with `qibojit` shows how the execution backend changes the profile of the same workload. Default Qibo is Back-End Bound for at least three quarters of its slots, with stores accounting for up to half of them and fewer than $20$% retiring, the signature of the NumPy reshape, copy and contraction operations that write the state back to memory. With `qibojit`, no store stalls are recorded and about two thirds of the slots retire. This difference supports the interpretation of Section 5.1 that memory writes in the NumPy execution path contribute to the offset of default Qibo in Figure 5.1.

## A.4 Process Layout of the Multi-Node CPU Runs

The multi-node CPU runs of Section 5.3 compare each backend in the faster of its two process layouts, mpiQulacs and QuEST with one MPI rank per node and $128$ threads, and Qiskit Aer with $128$ ranks per node and one thread each, and Figure 5.3 shows Qiskit Aer in both. The question that this choice leaves open is whether mpiQulacs would also gain from the layout of Qiskit Aer. Table A.4 answers it by running both backends in both layouts with $28$ qubits per node, as in the weak-scaling series of Figure 5.4, on one and two nodes.

The layout of $128$ ranks per node shortens the runs of Qiskit Aer by a factor of more than $60$, for the reason given in Section 4.3.1. For mpiQulacs, in contrast, it lengthens the runs by about half, for two reasons. Its single rank already runs one thread on each core of its node, so the split leaves each rank $1/128$ of the node's share of the state and gates on seven more qubits need MPI exchanges within the node. Moreover,

**Table A.4:** QFT time of mpiQulacs and Qiskit Aer on Deucalion x86 with $28$ qubits per node, with one MPI rank per node and $128$ threads and with $128$ ranks per node and one thread each, and the speedup of the second layout over the first. Each time is the fastest repetition, as in Figure 5.4, and the mpiQulacs times with $128$ ranks per node rest on one job of ten repetitions. With one rank on one node, Qiskit Aer runs its distributed engine forced onto that rank.

| | | | Time [s] | | |
|---|---|---|---|---|---|
| **Backend** | **Nodes** | $n$ | **1 rank/node** | **128 ranks/node** | **Speedup** |
| mpiQulacs | 1 | 28 | 12.73 | 18.12 | $0.70\times$ |
| | 2 | 29 | 14.35 | 21.27 | $0.67\times$ |
| Qiskit Aer | 1 | 28 | 241.48 | 3.32 | $73\times$ |
| | 2 | 29 | 264.60 | 4.19 | $63\times$ |

its circuit optimiser emits about $1.6$ times as many gates in this layout, $618$ instead of $392$ at $n = 28$ and $698$ instead of $429$ at $n = 29$. Consequently, Section 5.3 compares each backend in the layout that its engine is built for, and the dashed line of Qiskit Aer in Figure 5.3 shows what the shared layout would have cost it.

## A.5 GPU Activity and Data Exchange in the Multi-GPU Runs

Section 5.4.2 links the slowdown from four to eight GPUs, which lengthens the runs of all three backends at every size (Table 5.7), to the exchanges that then cross InfiniBand. This section supports that interpretation in two steps. First, it estimates the time in which the GPUs are active and the time in which they are inactive; then, it measures the bandwidth of the exchanges themselves.

Every multi-GPU run of that section ran alongside `nvidia-smi`, which recorded the `utilization.gpu` counter of every GPU every $0.5$ s, and only the samples taken while the state vector was held in GPU memory were kept. This counter is the share of each sampling period in which at least one kernel was executing, so its mean over all GPUs of a run splits the median timed repetition into an active part, in which kernels execute, and an inactive part, in which no kernel executes (Figure A.3). The split is an estimate rather than a timing, since the kept samples also cover the untimed correctness check and the gaps between repetitions, the shortest runs on one node rest on two to eight samples each, and the counter records only whether a kernel was running, not what it computed or why a GPU was idle, so it cannot attribute inactive time to any exchange.

Within one node, the estimates of Figure A.3 place the GPUs of all three backends active for most of the run, those of CUDA-Q for $83$ to $95$% of it from $n = 32$, those of Qiskit Aer for $65$ to $86$% and those of QuEST for $86$ to $100$%. On two nodes, in contrast, the estimated active share falls to $5$ to $6$% for CUDA-Q, to $13$ to $17$% for Qiskit Aer and to $19$ to $24$% for QuEST. The step to eight GPUs changes both the active and the inactive time. At a fixed $n$, each GPU then holds half as much of the state, so its kernels update half as many amplitudes and the estimated active time shrinks. However, the exchanges between the two nodes now cross InfiniBand instead of NVLink, and the inactive time grows to most of the run, so the run lengthens even though its kernels finish sooner. That inactive time grows in proportion to the data, since on eight GPUs it

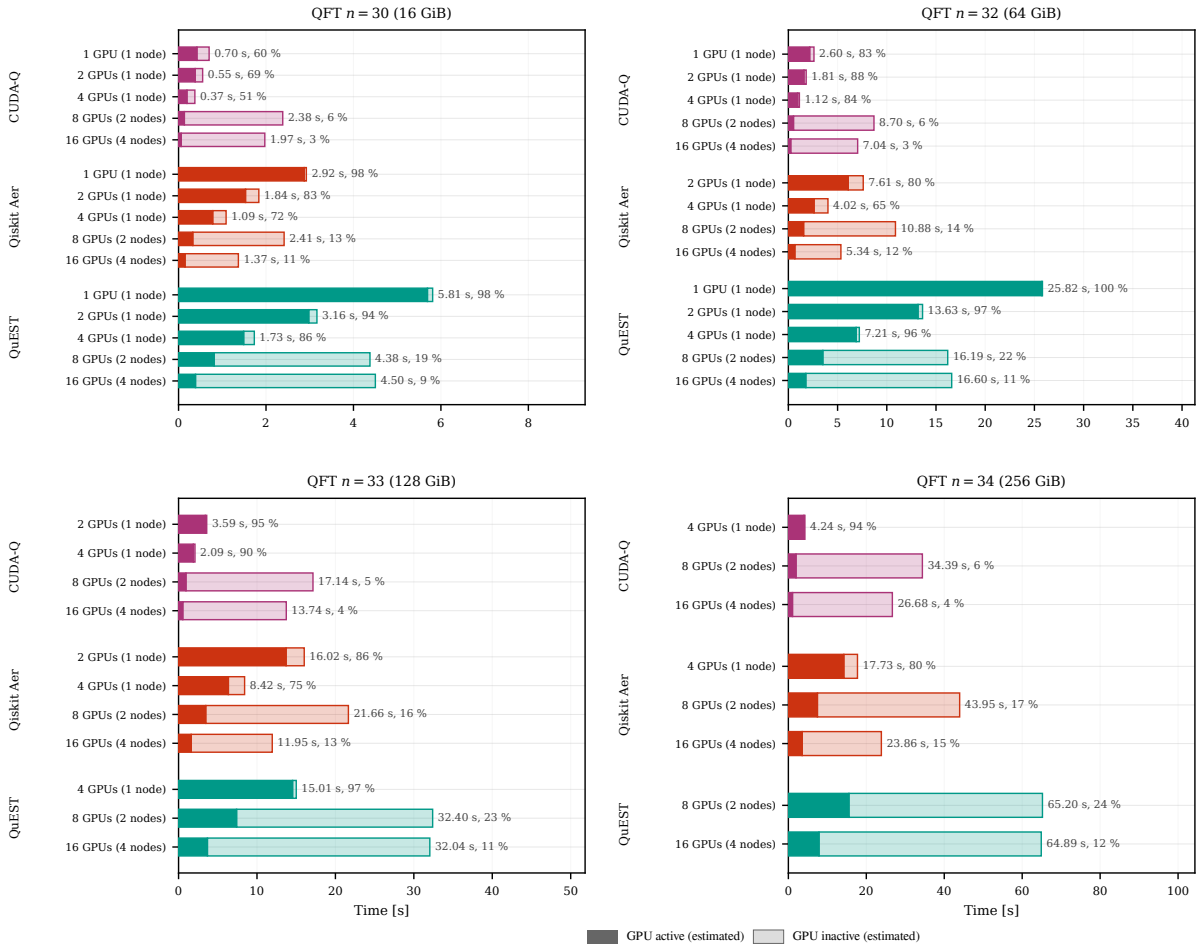



**Figure A.3:** Estimated GPU-active and inactive time of the QFT from $n = 30$ to $n = 34$ on Deucalion A100-80 nodes with four GPUs each, for CUDA-Q, Qiskit Aer and QuEST. Each bar splits the total time into the estimated part in which the GPUs execute kernels (dark) and the part in which they are inactive (light), from the GPU utilisation sampled every $0.5$ s during each run. Each label gives the total time and the estimated active share, and runs on up to four GPUs stay within one node. Inside a node Qiskit Aer uses the CUDA IPC transport in every run.

amounts to about one second per GiB that each GPU holds for CUDA-Q at every size, as a transfer limited by bandwidth would.

The backends differ in how that split is made up. The higher active share of Qiskit Aer across nodes reflects its longer estimated active time, whereas its inactive time is close to that of CUDA-Q, about $36$ against $32$ s at $n = 34$ on eight GPUs. QuEST has the highest share for the same reason, since its unfused passes make its estimated active time the longest, about $16$ s at that point, although its inactive time is also the longest, about $50$ s. For all three backends, the slowdown at the node boundary comes with a marked increase in estimated inactive time, which is consistent with the loss that Section 5.4.2 reads from the step ratios.

The bandwidth behind these transfers, which Section 5.4.2 quotes, was measured directly with the same Python environment, the same Open MPI and UCX build and the same one-GPU-per-process placement as the Qiskit Aer runs, by swapping buffers of up to $1$ GiB of GPU memory between pairs of processes in

both directions at once, as a distributed state vector does (Figure A.4). Inside one node the swap reaches $89$ GB/s per GPU and direction with the CUDA IPC transport of UCX, whereas without it the data are staged through host memory at $4.7$ GB/s for one pair and $2.5$ GB/s when all four GPUs of the node exchange at once. Between two nodes the transport plays no role, since CUDA IPC works only within a node, and the bandwidth falls for two reasons. The link itself is slower, since a single pair of GPUs swaps at $7.9$ GB/s over InfiniBand against $89$ GB/s over NVLink. The four pairs of a run on eight GPUs then slow each other down, since swapping at once leaves each GPU only $0.9$ GB/s, or $3.6$ GB/s in total, less than the single pair alone. Buffers in host memory, by comparison, swap at $16$ GB/s per direction over the same InfiniBand link, whose two adapters per node run at $200$ Gb/s each, so the link carries host memory faster than it carries any of the GPU buffers. Their limit lies in moving GPU memory onto the network under the default UCX settings, a path that the four processes of a node appear to share.

These bandwidths give an expected exchange time to set against the activity estimates. At $0.9$ GB/s, swapping half of the slice of each GPU takes about $4.7$ s at $n = 32$ and $19$ s at $n = 34$ on eight GPUs, against estimated inactive times of $8.2$ and $32$ s for CUDA-Q. The two agree to within a factor of about two, which is consistent with the few exchanges that the QFT makes on the qubit split between the two nodes and supports the exchanges as the main contributor to the slowdown. However, the bandwidth test uses the MPI and UCX stack of Qiskit Aer and no backend records its exchanges, so the comparison is an estimate.

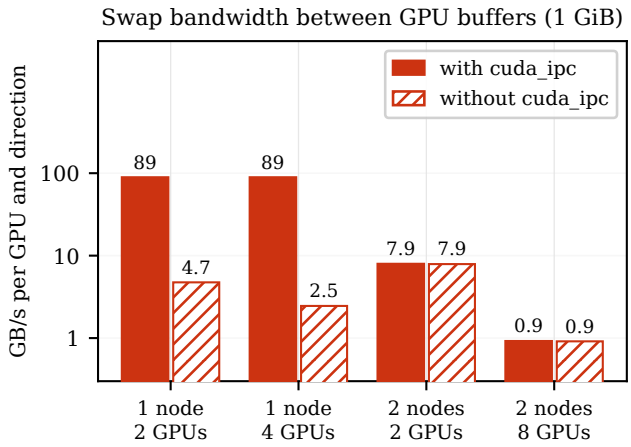



**Figure A.4:** Bandwidth of swapping $1$ GiB between GPU buffers in both directions at once, per GPU and direction, on Deucalion A100-80 nodes, with the MPI and UCX stack of the Qiskit Aer runs, with (filled) and without (hatched) the CUDA IPC transport. The groups are two and four GPUs in one node, and one and four pairs of GPUs across two nodes (two and eight GPUs). Logarithmic scale.

## A.6 The CUDA IPC Transport in the Qiskit Aer Runs

Section 5.4 runs Qiskit Aer with the CUDA IPC transport of UCX in every multi-GPU run within a node. This section records what the transport changes and why those runs start a separate process for each repetition from $n = 32$.

Qiskit Aer exchanges parts of the state vector between its MPI processes, one per GPU, through CUDA-aware Open MPI and UCX, which picks the transport for each pair of processes from the list in the `UCX_TLS` variable. With `cuda_ipc` on that list, two processes on the same node copy data directly between their GPUs over NVLink; without it, UCX stages the data through host memory, so every exchange inside a node is limited by Peripheral Component Interconnect Express (PCIe) and host memory (Figure A.4). In these runs, however, the GPU memory of a finished simulation stayed allocated while the process was alive and accumulated over the repetitions in one process until the GPU ran out of memory. The cache of mapped IPC buffers that UCX keeps is a possible cause, which was not confirmed. From $n = 32$, each timed repetition therefore runs in a new process, while at $n = 30$ the accumulated memory still fits.

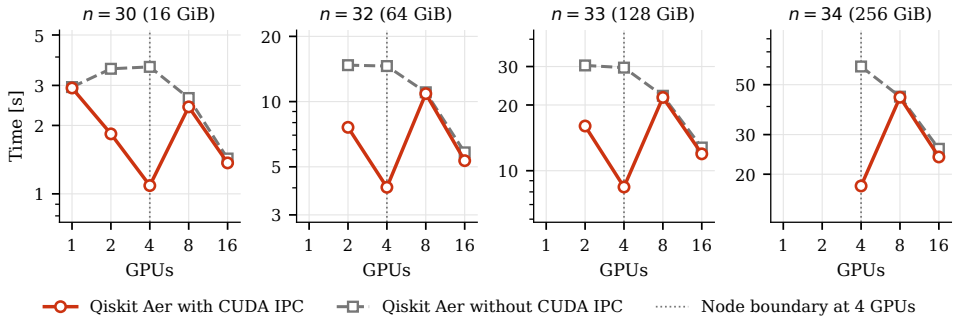



**Figure A.5:** Qiskit Aer QFT time with (solid) and without (dashed) the CUDA IPC transport of UCX on Deucalion A100-80 nodes with four GPUs each, for $n = 30$ to $34$ qubits, against the number of GPUs, on a logarithmic time axis. Each point is the median of ten repetitions in double precision with one MPI rank per GPU, and the dotted line marks one full node of four GPUs. Without the transport the time barely changes from two to four GPUs, and across nodes the two settings nearly coincide.

Within one node, the transport decides whether Qiskit Aer scales at all. With `cuda_ipc`, the run nearly halves from two to four GPUs, whereas without it the time barely changes (Figure A.5), so on four GPUs the transport makes the run $3.4$ to $3.6$ times faster from $n = 32$ to $n = 34$. On one GPU, where nothing is exchanged, the two settings take the same time, and across nodes they differ by less than $10\%$, since the InfiniBand exchanges then dominate the run. The activity estimates of Appendix A.5 indicate that the transport acts mainly on the exchanges, since at $n = 34$ on four GPUs the estimated active time is $14.2\,\mathrm{s}$ in both settings while the inactive time falls from $46.0$ to $3.5\,\mathrm{s}$. The comparison with CUDA-Q within a node in Section 5.4.2 therefore gives both backends direct exchanges between GPUs, so host staging does not account for the difference that remains between them, although their exchanges and their schedules of computation and communication can still differ.